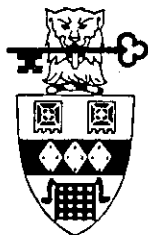
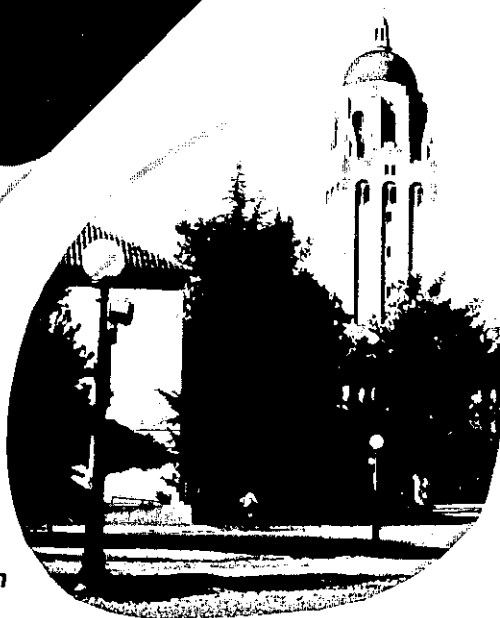


VECTOR

APL91
Reports

*The Journal of the
British APL Association*



A Specialist Group of the British Computer Society

ISSN 0955-1433

Vol.8 No.2 October 1991

Contributions

All contributions to VECTOR may be sent to the Journal Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be accompanied by as much visual material as possible. Unless otherwise specified, each item will be considered for publication as a personal statement by the author. The editor accepts no responsibility for the contents of sustaining members' news, or advertising.

Please supply as much material as possible in machine-readable form, ideally as a simple ASCII text file on an IBM PC compatible diskette (any format). APL code can be accepted as camera-ready copy, or in workspaces from I-APL, APL*PLUS/PC, APL*PLUS II, IBM APL2/PC or Dyalog APL/386.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1991-92

Category	Fee	Vectors	Passes
UK Private	£12	1	1
Overseas Private	£20	1	1
(Supplement for Airmail)	£8		
UK Corporate Membership	£100	10	5
Overseas Corporate	£155	10	
Sustaining	£430	50	5
Non-voting Student	£6	1	1

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Administrator using the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are £250 per full page, £125 for half-page or less (there is a £75 surcharge per advertisement if spot colour is required).

Deadlines for bookings and copy are given under the Quick-reference Diary. Advertisements should be booked with, and sent to, Alison Chatterton, whose address is given on the inside back cover.

DYADIC

have moved to,
new premises
as from **Monday**
30th July, 1990

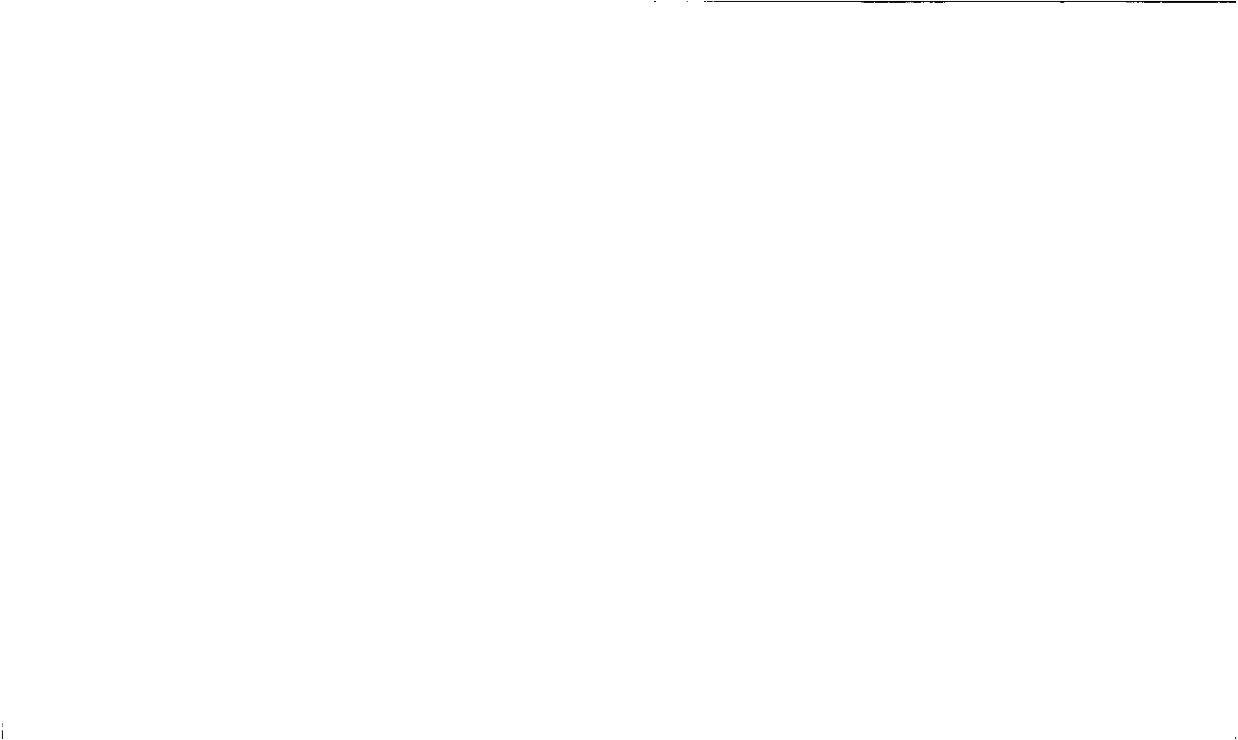
Please make a note
of our new address,
telephone and fax number

CHANGE OF ADDRESS

Dyadic Systems Ltd.

Riverside View, Basing Road
Old Basing, Basingstoke
Hants RG24 0AL, United Kingdom
Tel: 0256 811125 Fax: 0256 811130



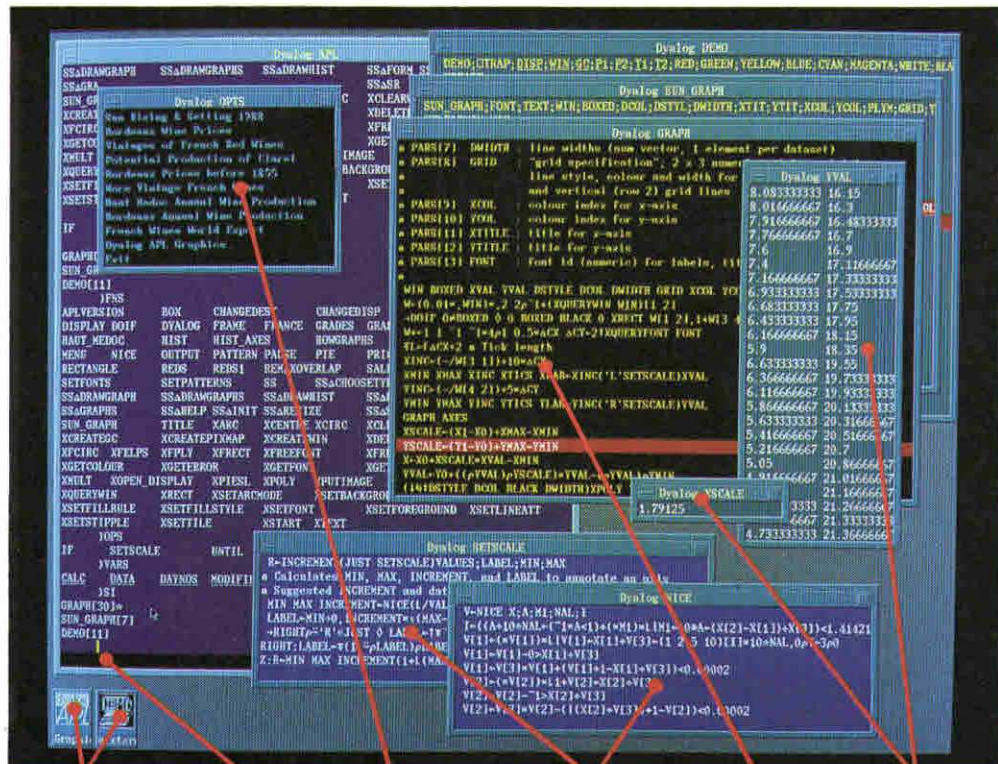


CONTENTS

Guest Editorial Wanted: Programmer Support	Gregg Taylor	Page 3
APL NEWS		
Quick Reference Diary		5
APL92 Program Statement		7
General Correspondence		11
British APL Association News: News from Sustaining Members	Alison Chatterton	13
The Education Vector	Alan Sykes	17
The Random Vector	David Eastwood	33
RECENT MEETINGS: APL91 at Stanford		
Conference Roundup		56
Sunday 4th Aug (Polivka and Zeidner Tutorials)		61
Monday 5th Aug		65
Tuesday 6th Aug		70
Panel "Is J a Dialect of APL?"		76
Wednesday 7th Aug		
Panel "25 Year Perspective"		81
Panel "APL as an Entrepreneurial Tool"		84
Thursday 8th Aug		87
Survival Strategies for APL Puddle-dwellers	Adrian Smith	93
GENERAL ARTICLES		
Power Reduction	Graham Parkhouse	96
The Legacy of the Typewriter	Richard Nabavi	99
Suggested Standards for a Component File System at a Commercial APL Site	Ray Cannon	105
ASWI Programming Standards	FHDv.Batenburg et al	111
TECHNICAL SECTION		
Hackers' Corner: Fast Filing with <i>Δpack</i>	Adrian Smith	125
A Case Story About Moving an APL-application	Allan Zenth	127
Desk Top Publishing on the Cheap	Richard Hill	130
Training the Mouse (in APL*PLUS)	Alain Delmotte	132
How STSC's <i>□MF</i> can help in testing APL workspaces	Ray Cannon	135
A Look at Residues	Stephen Wynn	137
Index to Advertisers		143

dyalog APL/X

GETS YOU THERE FASTER!



You can "iconify" and restore your windows as and when required. Here you see icons for an APL graphics window and a second Unix session.

You can execute expressions in the Session whilst Edit and Trace windows remain open.

You can edit character matrices and vectors of character vectors.

You can edit several functions and/or operators at the same time. Lines can easily be copied/moved between functions or to and from the Session.

The TRACE facility lets you step through your application line by line. As you do so, the APL stack is displayed and the current execution line is highlighted.

Watch your variables change as you run your code.

A 1-user licence for Dyalog APL/X costs just £1,500 and is available for a range of UNIX workstations including IBM RISC System/6000, Sun SPARCstations, and Hewlett-Packard HP9000-300/400 Series.

In addition to the unique Development Environment pictured above, Dyalog APL/X provides a powerful set of tools to help you produce good-looking, responsive and easy-to-use interfaces for end-user applications. Other important features include graphics support, external variables, a really fast component file system and a native file interface. You can easily access external functions written in C or Fortran, and an interface to Oracle is available on most systems.

All Trademarks, Servicemarks and Trade Names are the acknowledged property of the Companies concerned.

For further details contact:

Dyadic Systems Limited
Riverside View, Basing Road, Old Basing,
Basingstoke, Hampshire RG24 0AL
Tel: (0256) 811125 Fax: (0256) 811130

DYADIC

Wanted: Programmer Support

Guest Editorial by Gregg W Taylor

Extended APLs are no longer new products. By now, there should be evidence of a fair amount of proficiency in the use of nested arrays, user-defined operators, and the various new functions that manipulate nested arrays. Yet, I am compelled to say that the level of proficiency in these areas is disappointingly low.

I have not collected hard evidence in the form of surveys or extended research; this conclusion is based on observations I have made at several corporations, several user groups, and chats with other APLers. As a disclaimer, I will say that there is a good amount of code that is being written using extended features - by a few good programmers. But the majority (!) of people that use APL to program are still using a small subset of the capabilities available to them. WHY?

I think there are four basic reasons:

- the cost and effort of conversion
- the prices of the newer systems tend to be higher
- extended systems seem more complicated
- a lack of good examples

I will briefly cover these in the order listed.

Moving out of flat APL into the broader realm of ragged arrays and the each operator is really not that difficult. Migration tools are provided, and by and large, flat APL is a proper subset of nested systems. But on a practical basis, there are those nagging differences. IBM no longer supports the 124 processor; strand notation and ambivalent functions allow code that once failed to now successfully execute; and if you have never tested for complex numbers, you may be inclined to start. In short, it is a simple application that migrates trivially; the more sophisticated, the more considerations.

And I know of large corporations that are intimidated by the prospect of migrating hundreds, or even thousands, of workspaces out of a system that is working fine. This situation is worsened by purists (like myself) who have to improve the systems in the process by rewriting existing methodologies. All of this is for the better, mind you; but it is all at a cost.

The newer systems, mainly on the PC, tend to carry higher price tags. IBM has reduced the price of their mainframe product to approximately the VSAPL price, and their APL2PC product is still \$500, but compare STSC's two PC offerings. I am not suggesting that there should not be a price differential between flat APL and an extended system, but I do think the pricing should be accessible to the fellow who can afford an 80386

machine. When you can buy C compilers and assemblers for under \$200 (and even under \$100 in some cases), it's no wonder that even \$500 seems high. Is APL priced more for corporations than individuals?

But I can't blame the vendors for the programmer inertia which prevents (flatland) APLers from learning to use the extended systems. I remember my own misgivings about learning a new way to do APL - all the timings that had to be done, all the new idioms to be discovered, the pitfalls to be avoided. It is easier to just keep on doing things the same old way. For me, this is the most frustrating thing to witness - complacent programmers, no longer interested in growing in their professions.

Some have said that the new flavours of APL are simply too complicated. I even remember listening to one APLer saying that, since a printer could only print pages, why work in dimensions higher than two? More sophisticated data structures are intended to more closely model more complicated physical realities. Flat APL provided a method of organising data dimensionally; extended APLs provide a broader range of data structures, hence a close match with real data in more situations. Once the data structure is decided upon, the other new features are needed to manipulate that data.

The process of abstraction and generalisation leads to richer structure and function. To paraphrase Einstein: keep things as simple as possible, but no simpler. Some solutions are not adequate when over-simplified. Extended APLs provide more capabilities when dealing with non-trivial problems.

If the new APLs are more complicated, I think this is a result of the lack of understanding about them. Once you get acquainted with and use the new features, it becomes a lot more difficult to solve a problem using flat APL. Overcoming the reluctance to learn new tricks and the fear of not catching on quickly are the two real problems facing APL programmers, not the language itself.

But the learning process could be made a lot simpler if there were more samples of good coding techniques in the extended APLs. Not all of us will do the timings, discover the nifty algorithms, and warn of the nasty pitfalls. Yet publications like this one are much too rare, and there are only a few books on techniques in extended APLs. The New York APL users' group has undertaken, with the support of the SIGAPL, the compilation of a book of APL phrases that deal only with extensions in the various dialects. This promises to be quite good - hopefully combining good coding style with interesting idioms, all the while illuminating the differences in the several flavours of APL available today.

Really getting the newer APLs into proper use is not that difficult a task. But it will require the efforts of corporations, vendors, programmers, and (a few) writers. It's hard, and often expensive, to keep up with new technology. Hardware has typically outstripped software in these areas, but I dare say that extended APLs are in the software vanguard, waiting for more of us to catch up.

Quick Reference Diary 1991-2

Date	Venue	Event
25 October	IEE London	BAA Meeting
22 November	IEE London	BAA Meeting

Provisional dates for 1992

14 Feb 1992	IEE London	BAA Meeting
27 March	IEE London	Donald McIntyre: Tutorial Day on J
1 May	IEE London	Vendor Forum
12 June	IEE London	BAA AGM and meeting
25 September	IEE London	BAA Meeting
23 October	IEE London	BAA Meeting
27 November	IEE London	BAA Meeting

British APL Association meetings are held in the IEE, Savoy place. Nearest tube outlets: Temple or Embankment. Meetings are normally held on the 3rd Friday of the month throughout the autumn and spring.

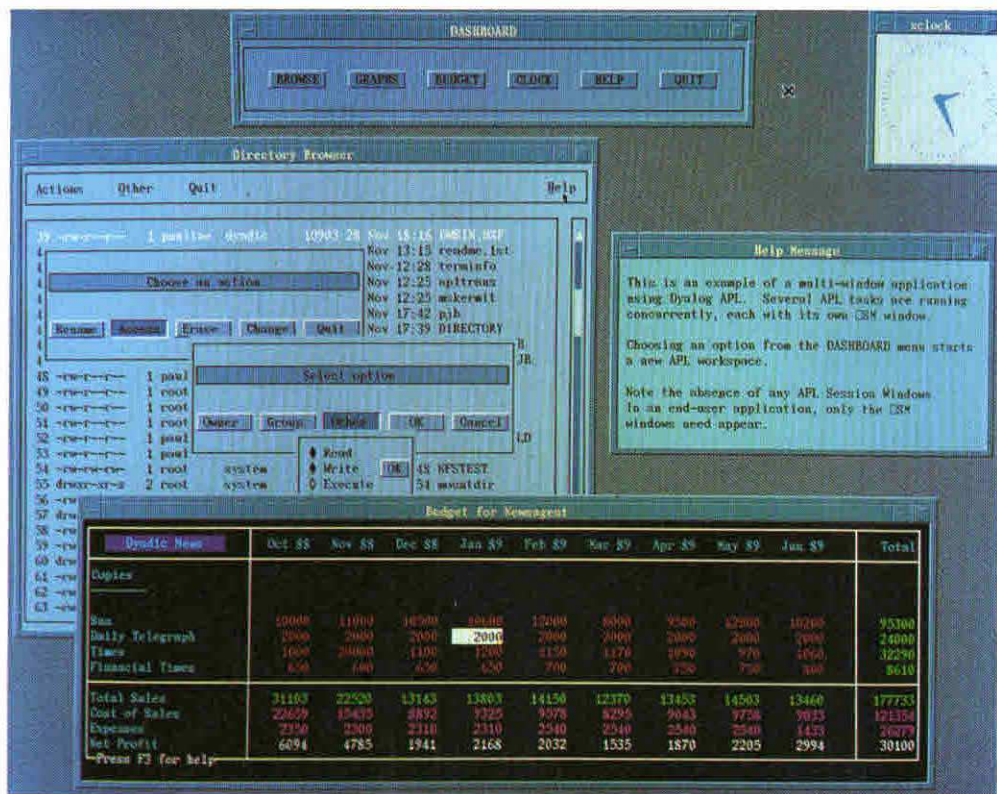
Dates for Future Issues of VECTOR

	Vol.8 No.3	Vol.8 No.4	Vol.9 No.1
Copy date	1st Dec 91	1st March 92	1st June 92
Ad booking	1st Dec 91	1st March 92	1st June 92
Ad Copy	12th Dec 91	12th March 92	14th June 92
Distribution	January 92	April 92	July 93

Overseas APL User Groups

The Vector working group are anxious to maintain contact with as many overseas associations as possible. If you are running an APL group and would like to receive Vector in exchange for your own newsletter or journal, please contact Alison Chatterton at the address on the inside back cover. We will endeavour to compile and maintain an accurate list of contacts of overseas groups, and publish this regularly with our APL Product Guide.

AS YOUR USERS SEE IT!



This photograph illustrates the sort of modern windows applications you can develop using Dyalog APL/X. At the heart of the user-interface is a system variable `□SM` which represents the layout, content and behaviour of a set of fields within an X window on the display. Users interact with `□SM` using the keyboard and/or mouse through a system function `□SR`. Together these tools provide an extremely powerful yet natural and easy-to-program user-interface for your APL applications.

Dyalog APL/X is just one of a fully compatible family of APL products for UNIX workstations, multi-user UNIX systems and DOS/386 PCs.

All Trademarks, Servicemarks and Trade Names are the acknowledged property of the Companies concerned.

For further details contact:

Dyadic Systems Limited
Riverside View, Basing Road, Old Basing,
Basingstoke, Hampshire RG24 0AL
Tel: (0256) 811125 Fax: (0256) 811130

DYADIC

APL - The Language in Common

APL 92 Program Statement

The APL Community has been gathering in international conferences since 1973. We have believed these conferences are the showplace of worldwide APL activities. Quite recently we have however become aware that we have been missing a major area of APL development and APL applications because the Soviet Union APL community has not been represented in our international meetings.

This long time ignorance and negligence was caused by political and language barriers and by missing contact between western and Soviet Union APL people. In the broadest sense we could say that we did not have a common language, we did not know how to communicate.

APL 90 in Copenhagen was a big door-opening event where the western and eastern APL communities met. It was found that the Soviet Union is one of the APL countries. The USSR Academy of Sciences, the Ministry of the Nuclear Industry, and other state and public organisations sponsor APL development in the Soviet Union.

The first GLOBAL Conference on APL

APL 91 in Leningrad is meant to be the first truly global APL conference where western and eastern APL communities, hitherto virtually unknown to each other, will meet and create fruitful new contacts. We would like to build a world of mutual understanding, but we remember the story about builders of the Tower of Babel, who failed due to the lack of a common language. We want to show in practice how APL is the **Language in Common** for people with different cultural and language backgrounds. To overcome still existing language barriers all conference presentations and speeches will be translated from English to Russian and vice versa and there will be bilingual helpers around to facilitate personal communication. The proceedings will be published in English.

The data processing field in the Soviet Union is in a strong expansive phase. The Soviet APL community is well prepared for the expanding market. The western APL community can help Soviet APL people to fight for a better position in this dynamic environment. The technology and the experience from the West combined with the "new blood" from the East can give a strong impulse to the success of APL world wide.

The APL 92 Program Committee expects a broad spectrum of papers dealing with various aspects of the language, APL implementations, supporting software, and applications. We also want to focus on APL in the mixed environment where multiple national languages, multiple currencies, and different standards are used. The integration of the world economies requires new solutions, and APL can show its strength there. We would also like to get papers concerning teaching technology and the role of APL in it.

Conference Overview

The APL 92 Conference will take place 6 - 10 July 1991 in Leningrad, northwestern Soviet Union. The Conference site is the Architecture and Building Training Centre of the State Committee of the Nuclear Energy Usage. Its own hotel can accommodate 270 participants at a reasonable cost and city international hotels will be available for those who wish to live near the attractions. The Centre is situated about 3 miles from the city centre near a metro station with fast service all around the city. In addition, a convenient taxi and special bus shuttle services will be provided at a fixed price. Two restaurants in the Centre will be in the service of the participants for lunches and dinners. A coffee shop for refreshments will be open all day.

Leningrad is the second largest city in the USSR. It was founded in 1703 by Peter the Great as the new capital of the Russian Empire. Leningrad is situated on the delta islands of the river Neva. There are more than one hundred rivers and canals with more than 300 bridges across them. Some of the bridges are drawbridges which are raised every night. Leningrad has a vast number of historical and architectural monuments, churches, palaces and museums, minutely preserved, all open to the public. The famous Leningrad "white nights" in the high summer make it possible to enjoy its beauty twice around the clock.

Activities: The main conference will run from 6 - 10 July, including: submitted papers presented in parallel tracks; sessions of posters addressing short or informal topics; opening and closing sessions featuring prominent speakers in fields of interest to APL users; vendor exhibits and vendor forum. Rooms will be available for special sessions organised by user groups, committees etc.

Tutorials: On Sunday, 5 July, tutorials will be given in two parallel tracks, one to give an introduction to APL language, the second on general computer science topics. Both are intended to focal students and other interested individuals, given mainly in Russian. The tutorial sessions are open to anybody even without registration.

Social events and cultural programme: A variety of social events will provide opportunities to meet other attendees informally. Leningrad is rich in cultural events like concerts, ballet and circus performances, theatres etc. The conference office will help with reservations. Special tours for accompanying persons will be offered during the conference and side trips will be organised after the conference. One of the planned post conference tours is an International Conference on Computers in Education on board a comfortable ship. It will cruise along the northern rivers of Russia and visit the centres of the Russian wooden architecture, the islands Kigi and Valaam. The participants of APL 92 are exempt from the registration of this second conference.

Start Planning for Participation!

Please, return the enclosed interest card for priority mailings.

Come to Leningrad to meet experts, share information, experience the Russian charm and exotic surroundings, make friends.

Write a paper. Use the interest card to indicate your topic. Draft papers are due by 1 November 1991. Paper drafts and abstracts should be sent to the contact nearest the sender. Authors will be notified of acceptance by 1 February 1992 and will receive an author kit. Final camera-ready papers are due by 1 April 1992.

Show a poster. Use the interest card to indicate your topic. Send us an abstract of the program by 1 March 1991.

Exhibit your products and services. Rent a booth, schedule a forum, sponsor a social event. Please contact the organisers or use the interest card to get information as early as possible.

Teach a tutorial or chair a session. Indicate your speciality using the interest card.

Join ACM SIGAPL and receive a discount for the conference.

Travel and Communications

Leningrad was chosen as conference site partly because it is near Finland and its capital Helsinki, served by all major international airlines. Leningrad is 40 minutes from Helsinki by airplane, six hours by train or by bus. In order to ensure reliable and speedy mail and telecommunications services, a conference sub-office is operating in Helsinki.

Official carrier of the APL 92 Conference is Finnair. Finnair is an expert in travelling to Soviet Union from the United States, Europe and Japan, and will provide a traveller kit with telephone numbers, schedules and other useful information to make your planning and travelling easy and economic.



The first meeting of the APL92 Program Committee



Paul Chapman & Lee Dickey booking for APL92 with Olga Kondrashev

General Correspondence

From: Richard Nabavi

6th September 1991

Dear Sir,

I am sure that lots of Vector readers will have found Vector 8.1, the "Windows Special", particularly interesting. But I fear that they may also have been unnecessarily disheartened by it. Eager to consider bringing their APL applications into the new world of X-Windows and the Motif windowing interface, they will have turned to the articles by Jensen & Beaty, and by Peter Donnelly - and they will have been horrified.

Take IBM's X-Windows interface, AP144. A 103-line function just to do a simple "Hello World" program? The window doesn't even appear until line 51! Is this really the kind of developer-friendly high-level software tool which is going to make life easier for us?

Maybe graphics is a bit hard. Let's turn to Peter Donnelly's article. After all, user dialog boxes are a pretty standard feature for windowing systems. His last example seems to address a fairly straightforward requirement - a selection list with four "radiobuttons" and two exit buttons. That shouldn't be too difficult. But look at the code! What is all this stuff about "XmATTACH_POSITION" and "XtManageChildren"? Things are even worse here than in the "Hello World" example - you get to line 72 before you "realize the widget tree" (which I think means show the dialog box).

The problem - as always with this sort of thing - is that in both cases, the work is being done at the wrong level. It is important not to fall into the trap of providing an APL interface which comprises a one-to-one correspondence between low-level language calls - in this case the C interfaces to X-Windows and Motif - and APL functions. Whilst this may very occasionally be necessary, for 99% of cases it gives you the worst of all worlds - code which is as long-winded as C, without the advantages of C's high speed as a compiled language and the documentation and type-checking advantages of including standard interface files.

The APL programmer does not want to know about "XtInitialize" or "XLookupString" - and he most certainly does not want to be bothered with mapping X-windows data types to and from APLs. He will not be very impressed by the fact that giving a wrong argument to a function can crash APL, as will certainly be the case if C pointers are directly manipulated from APL. He

will be concerned at the CPU usage of an APL function which takes 100 lines to do a simple part of a screen display. And he will be very worried about how difficult it will be to port these massive functions to other windowing systems.

Fear not - it does not have to be done this way. The X-Windows example can be done in about nine easily-readable lines of APL. Peter's Multiple Widget Example can be done in about eight, depending on exactly what functionality is desired (bringing up the dialog box takes three lines). It can be done using an APL system which provides sensible defaults for everything which is not explicitly specified, and which can be used without specialist X knowledge. And what is more, it can be done in such a way that the code will be portable from (say) Motif to Open Look without change. And it can be written using an APL system which itself conforms to the style guidelines of Motif, with the proper scroll bars and menus and dialog boxes built in, all controllable using the standard Motif resource mechanism.

You will not be surprised to hear that I am referring to MicroAPL's way of doing things, as refined into APL.68000/X for Unix machines such as the IBM RS/6000. But you might be surprised - as I was - that no less than five years after we first launched our Macintosh version of APL.68000, others are still at the level of transliterating C subroutine names into APL functions, providing clever solutions to the wrong problem. The problem is not "How can we call the X-Windows C interface from APL?", but "How can we avoid having to call the X-Windows C interface from APL?".

Richard Nabavi
MicroAPL Ltd

News from Sustaining Members

Compiled by Alison Chatterton

Cocking & Drury

Cocking & Drury is delighted to welcome Dave Ziemann back to the fold following his round-the-world (or halfway-and-back- again) travels.

We have had a busier summer than expected. STSC have been advertising APL*PLUS heavily in US magazines like Byte and Dr Dobbs Journal. This has led to a large number of leads from their UK readers.

We are also about to ship upgrades to existing APL*PLUS II customers to Version 3.5. This allows users to exploit the multi-tasking Windows environment and is the first step in a process which will lead to a full GUI version of APL*PLUS II, running under the true 32-bit Windows NT environment, together with a Windows toolkit and DDE support. This migration path will maintain compatibility with existing versions of APL at the same time as taking advantage of the Windows environment.

Version 4 - the next step in that development - is already in Beta-test and is expected for release early in 1992.

We also had a busy time at the Software Tools '91 exhibition in June. We met a lot of people - new faces as well as old - and collected a large number of leads both for APL*PLUS and for Smalltalk.

STSC have announced new pricing structures for Run-Time for APL*PLUS PC and II. The price of PC Run-Time falls by over 60% (!) and Developer Kits for both - giving unlimited distribution rights for APL*PLUS Run-Time applications - become more readily available and more affordable. We have also been able to reduce our pricing on most APL*PLUS PC and II System products by up to 34%; details in the next Product Guide.

There is also a limited special offer on APL*PLUS Mainframe add-on software with 50% off the APL*PLUS Compiler and 25% off most other add-ons. Bargain basement time!

Both consultancy and course business are recovering slowly from the effects of the recession with prospects for the final quarter already better than the previous six months.

MicroAPL Ltd

At APL91 in August, MicroAPL announced what is probably our most important new product for some years - APL.68000/X for the IBM RISC System/6000. APL.68000/X is the first version of APL.68000 for IBM hardware and the first implementation of our APL windowing interface to run under X-Windows, using OSF/Motif. This marks the culmination of two years' development work on our range of APL interpreters to upgrade them to include second generation features and to allow them to run on computers which use RISC technology.

APL.68000 now conforms closely to the APL2 language specification for second generation APLs. This means that a serious APL user can now standardise on one dialect of APL in the confident expectation of being able to run the same APL code on virtually any hardware platform - a consideration of key importance bearing in mind the current, entirely reasonable, drives towards software standardisation and the 'downsizing' of applications.

Our latest issue of a MicroAPL News, issued to coincide with APL91, discusses how we have managed to port APL.68000 onto RISC systems such as the RISC System/6000 and retain the advantages of Assembly Language source code. We still have one master set of source code for APL.68000, with all the advantages this brings in terms of source code control. As far as the user is concerned, it also means that APL workspaces written using any version of APL.68000 can be immediately loaded and run by any later version of APL.68000.

APL.68000/X is now completing its beta test program and we shall shortly start shipping the first release. We have also announced proposals to cover run-time variants. We shall be supplying developers with a 'packager' which takes an APL workspace and 'packages' it with an interpreter to make a single executable file. Having bought the packager, developers will be free to distribute as many packaged copies of their application as they wish without further licence fees being payable to MicroAPL. We believe that this is the first time such a scheme has been applied to a Unix-based APL. We are confident that, like our free runtime policy on smaller machines such as the Apple Macintosh, this approach will encourage professional software developers to use APL for developing general applications.

Dyadic Systems Limited

Those members of Dyadic who took part in the exhibition at APL91 reported a busy stand and a high level of interest from attendees.

Pride of place on Dyadic's stand was given to a PC running an early version of Dyalog APL/W for Windows 3.0. Although the planned release date for this new product is not until January 1992, visitors to the stand were able to gain a very good idea of what it will look like and how it will operate. In particular, the Dyalog APL session, editor and tracer were shown fully operational, popping up and down using genuine windows, and looking just like the X Windows implementation of the product. Being the first APL to be fully integrated with the Windows GUI, Dyalog APL/W generated some excitement and early orders.

Dyadic was also showing Dyalog APL Version 6.1 under DOS and the same version under X Windows on an IBM RISC System/6000 and a Sun SPARCstation 2. The new version includes the facility to display the value of a variable in a window. This lets you browse large arrays in a convenient fashion, and you can watch values change as you execute code. Coupled with the Tracer, this new facility is a great aid to debugging.

On the IBM RS/6000, Dyadic was demonstrating an Auxiliary Processor interface to OSF/Motif. This experimental AP lets you drive all of the Motif widget set and produce end-user applications, written in Dyalog APL, that conform fully to the OSF/Motif Style Guide. Dyadic originally developed this AP to investigate strategies for integrating the Dyalog APL Screen Manager ($\square SM/\square SR$) with Motif and other GUIs such as Windows 3.0 and Presentation Manager. Although integration with $\square SM$ is Dyadic's intended direction, the AP can be made available as a stand-alone product if required. Anyone interested in obtaining a copy should contact Dyadic.

Dyadic has announced an implementation of Dyalog APL for the Hewlett-Packard HP9000 Series 700 range. The HP9000-720 delivers 57 MIPS and 17 MFlops, while the 730 is even faster at 76 MIPS and 22 MFlops. In APL benchmarks, the "entry-level" 720 runs 3 times faster than a 25MHz 486 on general operations, and up to 10 times faster on floating-point calculations.

APL People Limited

We continue to work towards 1992 and the advent of the Single European Market by cultivating our contacts overseas - we have just helped a major financial institution in Switzerland to recruit an APL specialist from the UK for a year's contract. So come on, all you employers out there looking for skilled APLers in France, Germany, Switzerland, Sweden, Spain and elsewhere - give me a ring! I've got lots of really good people eager to widen their horizons by working abroad (some already have foreign language skills). The recession in the

UK means that many people are prepared to accept very modest rates or salaries at the moment.

However, I'm sure that most of you reading this will be pleased to read that we have detected signs that the economy is starting to pick up at long last. Several companies have been recruiting at graduate trainee level, although at the time of writing, I still have one or two graduates with APL experience yet to be placed, and would be keen to hear from any companies who might be interested.

On the software side, our sister company, APL Manufacturing, looks set to secure a contract to supply £70,000 of software to a manufacturer of Formula 1 Grand Prix racing cars - so we're all hoping to swap our boring old company Fords for something a bit faster!! Going from one extreme to another, PEFAC, the estimating system written in APL, has been sold to a company employing just 8 people! The last time this happened, using PEFAC enabled the company concerned to expand its workforce to 50 within the year! The software has also been sold recently to a furniture manufacturer, employing disabled people, which has resulted in a number of enquiries from similar factories elsewhere.



APL PEOPLE

Serving the International APL Community

Consultancy advice and assistance with all aspects of APL systems development

Recruitment unique placement service for companies and individuals

Software Competitive prices on all APL software tools and interpreters

Contact: Jill Moss, APL People Ltd.,
The Old Malthouse, Clarence Street,
Bath BA1 5NS

Tel: 0225 462602 during office hours
0225 333618 evenings and weekends

THE EDUCATION VECTOR

October 1991

Editor Alan Mayer

This Education Vector has been reprinted from VECTOR Vol.8 No.2. VECTOR is the Quarterly Journal of the British APL Association. For more information about the British APL Association, please contact: Anthony Camacho, 2 Blenheim Rd, St Albans, Herts AL1 4NR Tel: 0727 860130.

Contents

Editorial	Alan Mayer	18
Letters		19
Science Education in California	Zdenek V. Jizba	22
Recreation with Transcendental Numbers	Gerard Langlet	25
Getting to Grips with Local Variables and the State Indicator	Alan Mayer	27

Dr Alan Mayer
European Business Management School
University College of Swansea
Singleton Park
Swansea SA2 8PP Wales, UK

Ysgol Rheolaeth Busnes Ewropeaidd
Parc Singleton,
Abertawe SA2 8PP

Tel. 0792 205678 Ext.4274 Fax: 0792 295626 JANET: MAYER@UK.AC.SWAN.MS

Editorial

Welcome to Education Vector. The more observant reader (who bothers to look at the front page) may have noticed a change of four letters in the name and address of the editor. The bad news is that the change occurs in the surname: after three hard but successful years of entertaining and educating us, Alan Sykes has made his first big mistake and handed the job over to me. The good news is that I work in the office next door to Alan, and he has assured me that he will continue to take a lively interest in the affairs of Education Vector.

I know that I write on behalf of many of you when I thank Alan for what the journal has become under his leadership - I know, because I have inherited a whole file full of letters expressing your appreciation. Perhaps the aspect I appreciate most is that we have all been encouraged to take part by contributing our thoughts, our objections, our prejudices, our APL functions - so that others may benefit from our experience (and our mistakes!).

So how can we build on the foundation that has been laid? May I emphasise the word *we* - everything written in these pages is open to every reader to criticize (constructively if possible!). Personally, I identify two major objectives: to encourage beginners in APL, and to promote the use of APL in Education. I believe that considerable progress is being made on both of these fronts, and I know of several individuals who are using Education Vector for these purposes. Perhaps there is also a third objective: to create new "beginners". In other words, I want articles, letters, suggestions please - not for myself, but for the thousands of people out there who need APL, but don't know it yet.

Among those thousands, I suggest that a major target should be secondary school teachers. I have been told that many of them are too busy with the National Curriculum to spend time on "side-lines" like APL. Part of their problem seems to lie in the area of coordinating and manipulating marks from tests. Perhaps we should tell them about a computer language that makes that sort of task much easier?

I know that there is much more to the National Curriculum than coordinating marks - but don't let that stop you from telling me! How? Well, like your latest notebook computer, I respond to several modes of data entry: post, telephone, fax, email (I even talk to people sometimes!). The relevant addresses and numbers are on the title page. This edition contains contributions from several readers, for which we are all grateful. Why not add your name to the contributors' list for the next edition?

Letters

From: Robert L. Briechele, Cleveland, Ohio.

IF-THEN-ELSE (Education Vector, April 1991, p.27)

A slight modification to allow the simpler construct, IF-THEN without the ELSE to operate if that was desired:

```

      V R←A ELSE B
[1] A---FL1: IF <APL chrstr> THEN <APL chrstr>
      ELSE <APL chrstr> construct
[2] M←(ρA←,A)⌈ρB←,B ⋄ R←(2,M)ρ(M+A),M+B
      V

      V IF S
[1] A---FL1: IF <APL chrstr> THEN <APL chrstr>
      ELSE <APL chrstr> construct
[2] ⚡S
      V

      VR←C THEN A
[1] A---FL1: IF <APL chrstr> THEN <APL chrstr>
      ELSE <APL chrstr> construct
[2] ⚡(⌈~1+ρρA)⌈' +0,0ρR←(⚡C)/A' ⋄ R←A[1+~⚡C;]
      V

```

I write APL code entirely without interaction with any humans, relying heavily on VECTOR, SIGAPL and the like for guidance. If you can recommend a good screen data input facility, I'd sure like to hear about it. I have written four or five for specialized use myself and have looked at several packages and bought one of them. I do not wish to have my workspaces prostituted with all kinds of objects about which I know little and which are named to conventions different from my own. I want a single object, perhaps written in C, which can be invoked on a setup basis (giving it a map of the screen together with a table of fields and attributes) and then called to manage screen input, allowing functions of my own to be executed (optionally) upon leaving any field. I want the data to be automatically sent from the screen to be scrollable or pageable up and down as I need that. And I want function-key-callable dialog boxes and information messages to pop up as needed. I know there are products which meet some or all of these requirements. If you know of any which you can recommend, I'd appreciate it.

(Editor's comment: Any suggestions will be forwarded. If they are of general interest, they will be published.)

**From: Andy Pole, Duke University, Durham, North Carolina.
Labelling (Education Vector, July 1991, p.30)**

For people with APL*PLUS/PC there is a utility workspace provided called **FORMAT** which includes routines to centre a title, label rows, label columns, left and right justify arrays of strings. Details can be found in the documentation (for release 8 see Chapter 6, Aids to Formatting, in the Formatting section). For dealing with segmented strings, removal of blanks, text replacement etc. there is a collection of assembly language routines in the workspace **ASMFNS**. A useful feature of the documentation (see "Workspaces" in the manual) is that for several examples equivalent APL expressions are given. Another source of useful APL idioms for all sorts of operations is the **FINN APL idiom** booklet.

**From: Tony O'Hagan, University of Nottingham.
Let's Integrate - Using APL (Education Vector, July 1991, p.22).**

I would just like to make a technical point about Simpson's rule. Applied to 3 points it is unarguably a good thing. It integrates a quadratic exactly, and so on. You can easily justify the weight 4 in the middle because of smoothness of the function. Whichever way it curves, the area under a piecewise linear interpolant (trapezium rule) underestimates or overestimates the integral according as the middle point is high or low. So we want more weight than the value 2 given by the trapezium rule.

What is not sensible is applying the rule repeatedly on $2m+1$ points with weights 1, 4, 2, 4, ..., 4, 1. I know this is often found in textbooks, but think about it. That will integrate exactly a function that is piecewise quadratic. That is, a quadratic on intervals 1 and 2, then a different quadratic on 3 and 4, and so on. Why is this sensible? Why not a function that is quadratic on intervals 2 and 3, 4 and 5, etc.? That makes just as much sense. Why not use that, with a bit of trapezium rule on the first and last intervals? Then you get weights 1.5, 2.5, 4, 2, 4, ..., 4, 2.5, 1.5. If this is just as good as the other, why not average the two sets of weights, giving 1.25, 3.25, 3, 3, ..., 3, 3.25, 1.25. Now you have almost exactly got back to the trapezium rule with weights proportional to 1, 2, 2, ..., 2, 1.

There is no good heuristic argument for weighting the even points 4 and the odd points 2. Repeated use of Simpson's rule looks odd, and it really does not make sense. I always use the trapezium rule for a lot of equally spaced points in one dimension. (In high dimensions, when the sheer number of function evaluations is a problem, then we need other rules, but I still do not use Simpson's in practice.)

From Peter Branson, Uxbridge, Middlesex.

More String Manipulations (Education Vector, July 1990)

In this article, Alan Sykes gives a little function named *DAYOFWEEK* which, given the month and year, finds the weekday number (starting with Monday). Alan attributes *DAYOFWEEK* to Jerry Turner and professes to not even trying to understand it. As they say: "Me too, neither!". Instead, I give below an alternative called *WEEKDAY*, which I believe is rather easier to understand:

```

      VWD←WEEKDAY;CONST;YRMO;DAYS
[1] A Get day of the week      Peter B   Util   Apr 91
[2] CONST+0 8 25 5 1 9 5 13 21 17 25 21
[3] YRMO+(□TS[1]-1900)+CONST[□TS[2]]
[4] WD←7|□TS[3]+(7|YRMO)+1((28|YRMO)+3)÷4
[5] A
[6] A To return name of day, instead of no, add next 2 lines
[7] A DAYS←7 4p'SUN MON TUE WED THU FRI SAT '
[8] A WD←DAYS[WD;]
      V

```

Note the different calling syntax since it doesn't need any arguments at all. Also there are a suggested couple of fairly obvious extra lines which can be de-commented to give the name of the day, instead of the number, if preferred. Note also that I am a bit more conventional, so Sunday is Day 1 here.

On the same sort of theme, another sometimes-useful utility function, using a similar technique, is *JULDAY* which gives the Julian Day number:

```

      VJD←JULDAY
[1] A Get Julian day number   Peter B   Util   Nov 90
[2] JD←0 31,(28+0=4|□TS[1]),31 30 31 30 31 31 30 31 30
[3] JD←□TS[3]++/□TS[2]+JD
      V

```

Science Education in California

by Zdenek V. Jizba

The Chevron Corporation in La Habra has a programme of science instruction to 5th grade students. The programme, called Adopt-a-School is currently available to three grade schools in the La Habra (Southern California) area.

Volunteers are mostly employees of the Chevron research facility in La Habra, but there are also several of us who have retired in recent years. Under the programme, one day a week is devoted to one of the schools. The volunteer in charge for that particular day spends one period (about 45 minutes) with each of four or five classes, mostly 11 to 12 year olds. The subject of study is generally some aspect of science. Examples include the study of fossils, simple experiments in physics, and construction of simple objects.

At the time I found out about this program, I was involved in writing lessons for I-APL. It seemed a good idea to try to extend my work by volunteering in this project.

Clearly, 45 minutes (or as someone pointed out, one micro century) is too short a time to teach APL, but it may be sufficient, I thought, to create a favourable impression that could in time motivate some of the students to pursue a career in maths and science.

Right away, I bumped into a road block, when I discovered that the only personal computers available for instruction were the Apple IIe's. I proposed to give a lesson in elementary maths instead, but I-APL came to the rescue with the announcement of a version for the Apple IIe. Mr Rolf Levenbach was kind enough to send me an early copy of the version.

I was able to access the Apple IIe about one week before the sessions were to be started. The time allotted was much too short (about 5 hours), and my total unfamiliarity with the Apple operating system was a great handicap. Fortunately I had prepared a session in advance using I-APL on my IBM PC/XT. At first, my approach kept changing from session to session as I was getting feedback from students' reactions and from the extremely slow response of the computer.

On a typical day I have 4 to 5 classes (each class with different students). The room facilities include 12 to 15 computers, and a blackboard. This year two of the schools also had facilities to project the monitor display on the screen. I have an assistant volunteer from Chevron to help me, and there is usually, but not

always a teacher or a teacher's aide with each class. The size of the class is determined by the number of computers in the room. There can be no more than two students sitting at each keyboard. To complicate the problem, one of the schools has a group of students who do not speak English. This class requires an interpreter, and I must say that this particular session is somewhat discouraging.

Clearly 45 minutes is totally inadequate for an extensive training in APL. However, I have gradually developed an approach that seems to give the students some idea of the excitement that APL users have when exploring the power of this language.

When the class enters the room, I request that the students sit in front of me and the blackboard. Then I explain that computers are quite "dumb". They do only what they are told to do. To illustrate this, I pretend to be a computer, and ask them to tell me to draw a square on the blackboard.

The first class in the morning usually has a hard time figuring out that the first instruction to me should be to pick up the chalk. By the time I get to the last class of the day this exercise goes pretty fast.

My next step is to write on the board the number 2, and ask: "What is the square of 2?" We then discuss the graph of a square and the number square. Once the students accept the concept that squares are obtained by multiplying a number by itself, I ask whether it may be possible to get squares by adding numbers instead. Using graphs again, I show that the first N odd numbers add to N squared.

At this point I mention an 11 year old boy named Karl who lived about 200 years ago. (Of course I refer to Karl Friedrich Gauss.) Karl's teacher asked his class to add the numbers 1 to 100 (actually this is a simplification), and was astounded when Karl came up with the answer in seconds.

I tell the class that we will next go to the Apple keyboards and do some exercises. At the end of these, if the students pay attention, they will not only be able to solve Karl's problem, but some may even figure out how Karl did it.

In the I-APL session, the students are asked to enter their names. The names are reversed, and the students are given some problems in arithmetic. For example they are asked to add two numbers. If they do it correctly, they see a "happy face", and if they make a mistake they see a "sad face". Some students never get past this stage.

The next problem is to add a sequence of three numbers using reduction (+ /). If they get past that they see exercises in multiplication. First two numbers followed by a product with ($\times$ /). About half of the students get this far. In the next exercise they learn how to store a numeric vector and how to rotate it. They also learn to use the index generator (iota). Some students get trapped here spending the rest of the time displaying ιX where X is some large number. When $\iota 5$ is added to its reverse, they see a vector of constants. Very few students recognize this as an important clue in the solution of Karl's problem.

A few students with some help end the session understanding the expression $+ / \iota 100$. So far (after instructing over 600 students) no students discovered on their own Karl's solution of adding the vector to its reverse, reducing the result and dividing by 2. (Several got there with a little help.)

Invariably there are some students who finish the lesson early, but show no interest in arithmetic. These usually ask me for some other "game". I do have a second exercise which shows manipulation of character strings. An array of words is generated, and the students learn how to rotate it, transpose it, and are challenged to find a pattern that will produce valid words no matter how this array is displayed.

From my experience so far, the fifth grade (ages 11 and 12) seems ideal for the introduction of concepts in science and mathematics by individuals from outside of the school. I believe that even a short exposure, if it demonstrates the beauty and challenge of maths and science, will have a significant effect in later years of the individuals.

CPCUG APL Lessons Available for Several Interpreters

A series of 26 interactive self-teaching on-disk lessons is now available for TryAPL2, IBM APL2, STSC APL*PLUS and Pocket, Sharp, and I-APL. Based on the work of Z.V. Jizba, these lessons were transferred to multiple APL formats by the APLSIG of the Capital PC User Group (CPCUG) for use in their 1991 APL classes. Lessons were edited to make them more generic, and to incorporate classroom experience.

CPCUG lessons are downloadable free from the BBS\APL: 301-384-3672, 300/1200/2400 baud, N-8-1, 24 hrs. See File menu Y. Lessons may be ordered by mail from HRH Systems, Box 4496, Silver Spring MD 20914 USA, for \$US19, postage paid worldwide. Mail orders should specify disk size and APL version (.TRY, .ATF, .AWS, .SAW, or .IWS). Cheques accepted in any national currency. Lessons will be sent free upon request to Eastern Europe and the Soviet Union.

Dick Holt, HRH Systems, Box 4496,
Silver Spring MD 20914 USA.

Recreation with Transcendental Numbers

by Gerard Langlet

First, let us fix $\square PP \leftarrow 16$ to have the maximum precision. What is the shortest APL expression producing the Golden Number? Probably:

```
.5×1+5*.5
1.618033988749895
```

Then, what is the shortest expression without using $*$? I propose:

```
⊥112ρ'1+÷'
1.618033988749895
```

(This is an interesting use of Execute, that may be generalised to produce other transcendental numbers, summing continuous fractions. Moreover, in APL68000, the expression becomes as short as the preceding one, if one knows that the closing quote may be omitted in that implementation...)

This gives the following idea: What is the shortest APL expression to get the Golden Number using only 1 either as a numeral or as a character? I propose:

```
⊥'1',111ρ'+÷1'
1.618033988749895
```

Then, is it possible to obtain the same number without any figure or numeral, even as a character?

```
⊥⊥'','',I,'','',I,I,I,ρ'+÷','',I',ρ⊥I←▼'I'='I'
1.618033988749895
```

(The $\perp$ on the right is necessary in APL*PLUS II: ∇ of a scalar being a vector of length 1.)

Now, knowing that

```
⊥111ρ'1+÷1+'      ⍘ returns 2*.5:
1.414213562373095
```

we may try:

```
⊠⊠I,I,I,'pI,'1+÷'1',I,'1+1'1',p⊠I←v'I'='I'
1.414213562373095
```

Similarly, a good value of $10\pi \cdot 5$ is obtained by:

```
⊠1111p'1+1+1+÷1+1+ 1+'      A (Mind the blank!)
3.162277660168379
```

Exercise: find the expressions that return all the famous transcendental numbers without using numbers!



Two of the younger delegates at APL91 ...
"How to fly a paper aeroplane"

Getting to Grips with Local Variables and the State Indicator

by Alan Mayer

One of the most important things we learn when we start working in APL is to keep track of variables. A pitfall we all discover at some stage or other is the loss of perhaps an hour's work because of a moment's inattention when we overwrite some important set of data. Perhaps we have laboriously copied a few hundred values into a vector called *J*, and then we execute a function containing a statement like *J←1*.

Well, as they say, "it can happen to anyone" - and of course if we have any sense at all we will have saved the workspace as soon as we entered the data. We always do, don't we? (except when the phone rings, or someone knocks at our door...)

One way of reducing the risk in this area is to use local variables whenever possible. In fact, we meet local variables before we realise what they are: as soon as you use a function with an argument, you are creating local variables. For example, a simple function like *MEAN*:

```

      ∇ RES←MEAN X
[1] RES←(+/X)÷ρX
      ∇

```

The variables *RES* and *X* are local. To demonstrate this:

```

      X←5 6 7
      RES←43
      MEAN 1 2 3
2
      X
5 6 7
      RES
43

```

Note that, in order to calculate the mean of 1 2 3, inside the function *MEAN* the value 1 2 3 must have been given to *X*, and the result 2 must have been assigned to *RES*. In spite of this, the original values have not been overwritten.

Suppose we rewrite our function *MEAN*, calculating the number of values, *N*, before doing the summation:

```

)ERASE MEAN
VRES←MEAN X
[1] N←ρX
[2] RES←(+ /X)÷N
    ∇

```

We should still have the original values of *X* and *RES*, but we need to test the effect on *N*:

```

    N←77
    MEAN 1 3 5
3
    X
5 6 7
    RES
43
    N
3

```

The value 77 has been overwritten by the value 3, which was assigned to *N* inside the function when the statement *MEAN 1 3 5* was executed. This can be avoided by explicitly making *N* a local variable. This is done by adding it to line [0] (the header) of the function, separated from it by a semicolon. So edit the function:

```

[0] RES←MEAN X;N
    N←77
    MEAN 4 6 5
5
    N
77

```

As you can see, the data in *N* has been protected by the declaration of *N* as a local variable inside the function. Now let's try something a bit more difficult.

```

      ∇FUN1;X
[1] Y←7
[2] X←4
[3] FUN2
[4] Z←X+Y
      ∇

      ∇FUN2;Y
[1] X←2
[2] Y←5
      ∇

      X←1
      Y←2
      FUN1

```

OK - what values do you think are in *X*, *Y* and *Z* now? (answer and explanation at the end of this article.)

It's all very well making sense of what comes out of a function when it is working the way you think it should - but what if something goes wrong? Can we find out where we are, and the current values of the variables? Yes, we can if we consult the State Indicator by typing *)SI*. First, let's start with a clear workspace and enter a simple function:

```

      )CLEAR
      ∇RES←X DIVIDE Y
[1] RES←X÷Y
      ∇
      6 DIVIDE 3
2

```

Now, if your function has worked correctly first time, you can carry straight on. But if you had to have more than one shot at it, your State Indicator may not be clear. Type *)SI*. There should be no response. If there is a response, you need to clear the State Indicator. Unfortunately, different implementations of APL require different commands to do this. One of the following should do the trick:

```

      )RESET
      )SIC
      )SICLEAR
      □RESET

```

Our aim is to get no response from *)SI*.

Next, we shall deliberately cause an error to occur:

```
6 DIVIDE 0
```

At this point, we should get a report that a *DOMAIN ERROR* has occurred. We have tried to divide 6 by 0, and found it to be impossible. What does the State Indicator have to say?

```
    )SI  
DIVIDE[1]*
```

The function *DIVIDE* has been suspended (* means suspended, i.e. interrupted because of an error). The error has occurred in line [1] (fairly obvious in a 1-line function, but informative in general). What about variable values?

```
    X  
6    Y  
0
```

We should be surprised that these variables exist at all - they are local to the function, and would have disappeared if the function had terminated correctly. But local variables remain active when functions are suspended. Furthermore, we can change their values, and then continue execution of the function:

```
    Y+2  
    +1  
3
```

The last statement means "resume execution at line 1". The result is 6 divided by 2. What has happened to SI?

```
)SI
```

No response - SI is clear. In this rather trivial example, we have caused an error to occur, corrected the fault in the data, and resumed execution to a successful conclusion. Let's create a second function:

```

      ▽ RES←X SUMINVERT Y
[1] RES←(1 DIVIDE X)+1 DIVIDE Y
      ▽

```

This function should sum the inverses of its two arguments:

```

      2 SUMINVERT 4
0.75

```

(The inverses of 2 and 4 are 0.5 and 0.25, which sum to 0.75.)

```

      2 SUMINVERT 0

```

Once again, a *DOMAIN ERROR* is produced.

```

      )SI
DIVIDE[1]*
SUMINVERT[1]

```

This time there are two entries in SI. As before, *DIVIDE* is suspended (indicated by *). Below this we have the function *SUMINVERT* which is pendent (without a *). Pendent functions have called other functions which then failed to complete their execution - in this case, *DIVIDE* became suspended. What are the values of X and Y?

```

      X
1      Y
0

```

No, they do not have the values 2 and 0. Why? We must think about what is happening inside function *DIVIDE* - the function where the error actually occurred. It is the second call to *DIVIDE* that has gone wrong - the 1 *DIVIDE* Y bit. At this point, Y has the value 0, so *DIVIDE* is being passed the parameters (X,Y) = (1,0), and this is still the situation when it crashes. Let's put it right:

```

Y←4

```

Before we resume execution, just one other point:

```
)ERASE DIVIDE
```

(It doesn't work: you can't erase functions that are suspended or pendent.)

```
      +1  
0.75  
      )SI
```

Clear again. Note: execution is resumed in line [1] of the top function in the SI list - the last one to get suspended.

Two final notes:

- (i) a suspended function can be terminated (together with any consequently pendent functions) by `→` (with no argument);
- (ii) the State Indicator can always be cleared using `)RESET` (or one of the other commands described earlier).

If you have never played with local variables and the State Indicator before, try making a few more functions crash - functions calling functions calling functions... etc. Familiarity with the concepts will encourage you to use them "for real" when the opportunity arises. The answer to the local variable problem:

```
      X  
1      Y  
7      Z  
9
```

Initially, X is 1 and Y is 2. Inside $FUN1$, X becomes 4 and Y becomes 7, then $FUN2$ is called. Inside $FUN2$, X becomes 2 and Y becomes 5. On return to $FUN1$, as Y is local to $FUN2$, it reverts to the value 7, but X stays as 2. Hence Z is the sum of 2 (X) and 7 (Y), i.e. 9. We now have the values $(X,Y,Z) = (2,7,9)$. On return from $FUN1$, as X is local to $FUN1$, it reverts to the value 1 (its original global value). Hence the result $(X,Y,Z) = (1,7,9)$.

THE RANDOM VECTOR

The Newsletter of the APL Statistics Library

Editor David Eastwood

October 1991

Contents

Editorial	David Eastwood	34
ASL at APL 91		35
A Basic Statistics Volume Tutorial	Jake Ansell	37
Stirling Numbers: a Case Study	David Appleton	44
Gregynog 91		49
ASL - the next stages	Tony O'Hagan	49

Editorial

The copy for this issue of the ASL Newsletter is being compiled as preparations for the third ASL Statistics Conference near completion and I shall be discussing the topics that we hope to cover at the conference. A number of us who are involved in the ASL project have just returned from APL91 at Stanford University and, in common with other articles in this issue of Vector, we shall report on the ASL presentation at APL91.

We take the opportunity to reprint David Appleton's article on Stirling Numbers which was savaged by the printers in an earlier issue of Vector and finally Tony O'Hagan steps down as ASL Chairman with some parting thoughts.

As usual, I include the contact details of the current ASL Management Committee who are (unless voted out at Gregynog!):

Deputy Chairman:	Alan Sykes, Department of Management Science and Statistics, University College of Swansea, Singleton Park, Swansea SA2 8PP. Telephone: 0792-295296
Manager:	Jake Ansell, Business Studies, University of Edinburgh, William Robertson Building, 50 George Square, Edinburgh EH8 9JY. Telephone: 031-650-3806
BAA Representative/ Newsletter Editor:	David Eastwood, MicroAPL Ltd, South Bank Technopark, 90 London Road, London SE1 6LN. Telephone: 071-922-8866
BAA Representative:	John Searle, 4 Hawks Mews, Greenwich, London, SE10 8RA. Telephone: 081-858-6811 (home)

ASL at APL91

Members of the ASL project were invited to give a tutorial presentation at APL91. Alan Hawkes, Alan Sykes and Alan Mayer from Swansea and Jake Ansell from Edinburgh gave a 90-minute team presentation on the ASL project. The audiovisual team at Stanford obviously deeply resented being asked to do anything as demeaning as audiovisuals and managed to live down to their customary low standards, but despite this the large audience of early birds who attended evidently enjoyed the presentation and were able to see most of the material being presented.

Alan Hawkes gave an introductory overview which covered the original aims of ASL and the way in which the ASL project has taken shape over the last few years. He outlined why it was originally felt that another set of APL functions for Statistical work might be beneficial to the APL community - and mentioned some of the standard works in this area (see the references I reproduce at the end of this review). Alan felt that the relative lack of success of earlier works in this area might be due to the common preference exhibited by APLers to write their own code rather than using code written by someone else. The ASL project hopes to overcome this 'acceptance' hurdle by the production of a definitive range of APL functions and, perhaps most importantly, functions which are supported by a large group of APL statisticians. This will lead ultimately to a comprehensive range of APL Statistics functions for all types of analysis. In a sense ASL could be thought of as a statistical equivalent of the Fortran NAG library.

At that stage it was realised that the concept should go beyond these bounds. It was necessary to create not just another set of APL software for Statistics but a wholly new concept. A unified structure and philosophy was required. It was subsequently perceived that the ASL project was attempting to create, paraphrasing APL itself, a statistical language.

There was an important realisation of the need to define the market of future users of ASL which arose out of the discussions held at the first ASL Workshop. It was fairly easy to list those who might use or be encouraged to use ASL: APL Statisticians, those teaching Statistics, Statistical Research workers, users of Statistics, etc. The real question was which group had priority? For the sound reason of knowing the group it was felt that ASL should attempt initially to satisfy APL Statisticians. Next would be college teachers of Statistics with the aim of exposing undergraduate students to ASL and hence APL. In Britain this could well be all students except those in the Arts Faculties. Once exposed, it was hoped that they would remember APL and wish to apply it in the future over a diverse range of careers. A potential consequence is the further acceptance of

APL within colleges and hence its spread into other areas. This established in our minds the key aim of ASL, which is to increase the use of APL, not just within Statistics but as widely as possible. The production of a suite of good APL Statistics functions then becomes almost a by-product of the main objective.

Alan Hawkes cited a number of references to APL in statistics and I reproduce some here:

Anscombe, F.J. [1981], *'Computing in Statistical Science through APL'*, Springer-Verlag, New York.

Douglas, J. *'Stat APL'*.

Grenander, U. [1982], *'Mathematical Experiments on the computer'*, Academic Press, New York.

Hawkes, A.G. [1990], *'Markov Processes in APL'*, APL Quote Quad, Vol 20, No 4, 175-185.

Smillie, K.W. [1969], *'Statpack 2: An APL Statistical Package'*, 2nd Edition, Dept of Computing, University of Alberta Publication.

Jake Ansell then outlined the design philosophy that has evolved over the course of the ASL project and the way in which the software production cycle has been managed, paying a particular tribute to Maurice Jordan's work in the automatic testing of APL code. For more details on these two topics, see Jake's article in Vector 8.1 'Bottom Shelf: Basic Statistics) and Maurice's paper published at APL91 (APL91 proceedings, 'CATS - Computer Aided Testing of Software').

The main part of the tutorial was based around live demonstrations of the three ASL Volumes now available - Basic Statistics, Generalised Linear Regression and Basic Graphics. The first two, Basic Statistics and Generalised Linear Regression are available to run under APL*PLUS, Dyalog APL, APL68000 and APL2/PC. Basic Graphics is currently written for Dyalog APL with ports to other interpreters under way.

Alan Sykes ran the tutorial on Regression and Basic Graphics and Jake Ansell gave the tutorial on Basic Statistics. Alan Mayer managed admirably in typing the examples as fast as the presenters introduced them.

Of the three sessions, I have chosen to include the Basic Statistics tutorial here. The Basic Statistics volume of ASL has so far had least coverage in Random Vector and this tutorial should serve as a good introduction to the scope of the functions included in the volume.

A Basic Statistics Volume Tutorial

by Jake Ansell

The Basic Statistics tutorial gave a demonstration of some of the Bottom Shelf functions applied to two sets of data: the monthly American Stock Exchange Market Value Index (closing levels) for 1986, 1987 and 1988 and UK TV Franchise Data on 16 UK Regional TV Stations taken from 'The Guardian' April 1991.

First, consider the American Stock Exchange closing levels for the period 1986 to 1988, a period which includes the October 1987 crash. The first stage in any statistical analysis is to look at the data:

```

      M
243.87 257.35 270.03 268.97 282.6 284.2 261.56 273.85 260.69
265.59 264.81 263.27 300.47 321.76 332.66 325.19 326.39 338.13
358.03 361.35 356.45 260.36 242.39 260.35 269.1 288.46 296.43
303.14 294.19 309.25 306.18 294.8 301.63 300.95 294.36 306.01

```

The data itself is a simple numeric vector, and as you would expect we have a series of functions to compute some basic statistics from the data such as mean, variance, minimum value and maximum value:

```

      MEAN M          FIND THE MEAN
292.9
      VAR M          AND THE VARIANCE
1015
      MIN M          EVEN 'MIN' AND 'MAX':
242.4
      MAX M
361.4

```

We might also require a feel for the shape of the distribution of data. Hence we need frequencies:

```

      FREQTAB M      A FREQUENCY TABLE
240 2
250 1
260 9      ... 310 0
270 2      320 3
280 3      330 2
290 4      340 0
300 7 ...  350 2

```

A visual representation is an improvement on this, and hence we have a simple barchart of data using the frequencies:

*BARChart FREQTAB M A AND A VERY SIMPLE BARChart OF THE
TABLE. EACH □ REPRESENTS 1 UNITS
SCALE IS 1*

```

240.00 | □□
250.00 | □
260.00 | □□□□□□□□
270.00 | □□
280.00 | □□□
290.00 | □□□□
300.00 | □□□□□□□
310.00 |
320.00 | □□□
330.00 | □□
340.00 |
350.00 | □□

```

Significance Tests

Given the stock market collapse, we might be interested to know whether 1988's figures are the same as 1986's. If they are, has the market recovered?

If we are willing to assume that the data is normal and independent then we can use a parametric test. We first test the data to see if the variances are the same with a variance test. This is based on the ratio of the two variances:

```

M[1:12] VARRATIO M[23+1:12]
0.5579

```

The value has to be compared to the F Distribution to see if it is significant. Given the value is less than one, we can take the reciprocal test is against F on 11,11 degrees of freedom.

```

11 11 FTAIL1 ÷M[1:12] VARRATIO M[23+1:12]
0.1737

```

The probability is fortunately in the central region of the distribution and hence we can assume the data have similar variances. We can now carry out the two sample t test.

```

M[1:12] TWOSAMT M[23+1:12]
-5.092

```

The value obtained should be compared to the t distribution on 22 degrees of freedom.

```
22 TTAIL M[1:12] TWOSAMT M[23+1:12]
0.9999789111
```

The result indicates that the populations are different, since the probability is in the tail of the distribution. It may be that one is doubtful about the normal assumption? Therefore a non-parametric test might be more sensible. The Mann-Whitney test is an appropriate choice.

```
M[1:12] MANNWHIT M[23+1:12]
26
```

This value could be compared to the appropriate table but if the analyst is willing to take the normalised value, he could compare with the t distribution as an approximation. The sample is relatively large.

```
M[1:12] MANNWHITTS M[23+1:12]
-2.655811238
```

This produces the normalised value of the test statistic. The probability value would be

```
22 TTAIL M[1:12] MANNWHITTS M[23+1:12]
0.9927809917
```

Fortunately a similar result as before. We might consider comparing the three years assuming the values are random observations from the years, although this is a little doubtful. The one way ANOVA test can be used to test whether the three years differ.

```
J+(12 12 12/1 2 3) ONEWAY M
J
2      14651.35035      7325.675175      11.58166525
33     20873.27475      632.5234773      0
35     35524.6251      0      0
```

We have assigned the result to J so that we can test the variance ratio against the F distribution.

```
J[1 2;1] FTAIL1 J[1;4]
1.547062728E-4
```

A low probability, implying difference, as might be expected.

Time Series

The above analysis was possibly not wholly appropriate since we know the data is a time series. Hence it would be sensible to treat it as such. We can then obtain the Moving Average to see whether there is a trend.

```
12 MOVA M
266.4 271.1 276.5 281.7 286.4 290 294.5 302.6 309.9 317.8 317.4
315.5 315.3 312.7 309.9 306.9 305 302.4 300 295.6 290.1 285.5
288.9 293.2 297
```

This produces a moving average whose values fall between the observations since the period 12 is even. It is more usual to produce the Centred Moving Average for even data.

```
12 CENTMOVA M
268.8 273.8 279.1 284 288.2 292.3 298.6 306.2 313.9 317.6 316.5
315.4 314 311.3 308.4 306 303.7 301.2 297.8 292.9 287.8 287.2
291.1 295.1
```

With this it is possible to calculate the seasonal effects and hence remove them from the data. The seasonally adjusted moving average can be obtained for both an additive model for seasonal effects as well as multiplicative. In the case of the data we might feel that as the trend increases, so seasonal effects increase and so the multiplicative model is appropriate. Hence we use:

```
12 SEASAM M
254.4 251.9 257.4 257.7 275.7 267.1 246.6 252.5 248.5 296.3
307.3 297.2 313.4 314.9 317.1 311.6 318.5 317.8 337.5 333.2
339.8 290.5 281.3 293.9 280.7 282.3 282.6 290.5 287 290.7 288.6
271.8 287.6 335.7 341.6 345.4
```

Now consider the other dataset - 1991 UK TV franchise data, held as a three column numeric matrix with the columns representing, successively turnover, viewing households, and staff numbers. This will be rather more topical for UK readers in view of the current TV franchise changes being considered.

The descriptive statistics functions can be applied to arrays, so we have:

```

DATAF
12.2      262      140
26        483      290
250       2870     1100
143.5     2000      700
220       2500      800
44        635      307
5.1       50       97
250       4900      910
86.5      2.2E4     399
347       5300     1550
314       4100     1000
125       1600      711
201.1     3000     1137
63        3000      380
110       1400      600
20        442       208

```

```

MEAN DATAF
138.6 3409 645.6
VAR DATAF
1.257E4 2.726E7 1.743E5

```

Linear Regression

In this data we might be interested to see if there is a relationship between Turnover and Household and the number of employees. Obviously as prospective franchise buyers we wish to predict Turnover.

Can we predict Turnover from Households? The model would be a linear regression:

$$T = b_0 + b_1 H$$

The model is fitted using a set of routines *REGLINEST* and *REGLINRESID* etc., but we can use *REGLINREP* to get a brief report. We store the results again, to test against the F distribution.

```
FP+1,REGLINREP DATAF[,1 2]
REGLINREP DATAF[,1 2]
```

```
EST REGRN. COEFF(S). 126.51 3.5443E-3
EST STAN. ERROR(S). 34.508 5.6596E-3
EST ERROR VARIANCE. 13097
EST COEFF. OF DETERMINATION 0.02725
```

ANALYSIS OF VARIANCE:-

SOURCE	SS	DF	MS	F-RATIO
REGRESSION	5136.6	1	5136.6	0.39219
RESIDUAL	1.8336E5	14	13097	
TOTAL	1.885E5	15	12567	

The result is a little disappointing, apparently Household on its own does not account for a significant amount of variation in Turnover.

```
FP[2 6] FTAIL1 FP[4]
0.5412
```

Indicating that the model is not significant.

Multiple Regression

Is prediction of Turnover better if we add Staff Numbers to the model? The model is now a multiple regression:

$$T = b_0 + b_1H + b_2S$$

We have a set of regression functions which enable us to do this (*REGEST*, *REGRESID* etc). Again we can use the reporting function *REGREP*.

```
FP+1,REGREP DATAF,1
REGREP DATAF,1
```

```
EST. REGRN. COEFF(S). 1.1239E-3 0.25404 -29.243
EST. STAN. ERROR(S). 1.8005E-3 0.022514 17.587
EST. ERROR VARIANCE. 1306.7
EST. COEFF. OF DETERMINATION 0.90988
ANALYSIS OF VARIANCE:-
SOURCE          SS          DF          MS          F-RATIO
REGRESSION      1.7151E5      2          85755        65.628
RESIDUAL        16987         13         1306.7
TOTAL           1.885E5       15         12567
```

We have assumed a mean, hence the column of 1 in the matrix.

```
FP[2 6] FTAIL1 FP[4]
1.608E-7
```

The result is significant. Fitting both variables seems to account for a large proportion of the variability in Turnover.

Polynomial Regression

Instead of adding Staff Numbers, we could have tried using the polynomial model,

$$T = b_0 + b_1H + b_2H^2$$

A set of Polynomial Regression functions exist in basic Statistics - *REGPOLYEST* etc. Using *REGPOLYREP* to fit a quadratic, we get:

```
FP+1,2 REGPOLYREP DATAF[;1 2]

2 REGPOLYREP DATAF[;1 2]
EST. REGRN. COEFF(S). 4.0766 0.078944 -3.4013E-6
EST. STAN. ERROR(S). 22.096 9.7859E-3 4.2767E-7
EST. ERROR VARIANCE. 2404.7
EST. COEFF. OF DETERMINATION 0.83416
ANALYSIS OF VARIANCE:-
SOURCE          SS          DF          MS          F-RATIO
REGRESSION      1.5724E5        2          78618        32.694
RESIDUAL        31261          13          2404.7
TOTAL           1.885E5        15          12567
```

and testing against F, we get

```
FP[2 6] FTAIL1 FP[4]
8.478E-6
```

which is significant. The model explains a large proportion of the variability.

Stirling Numbers: a Case Study

by D R Appleton

Theme

Several different APL functions, for the evaluation of Stirling numbers of the second kind, are presented. Their advantages and disadvantages are discussed.

Exposition

A colleague doing some theoretical statistical work on the analysis of an experiment which had resulted in counts from the truncated Poisson distribution being observed, recently asked me if I had an APL function to evaluate the Stirling numbers of the second kind. I did not have such a function, but he told me they were defined for positive integers n and t by

$$S_t^n = \frac{1}{n!} \sum_{k=0}^n \binom{n}{k} (-1)^{n-k} k^t$$

a formula obtainable from Abramowitz and Stegun (1964). With APL it is easy to turn this expression into executable form, for example:

```

▽ Z←N STIR1 T;K
[1] Z←(÷!N)×+/(K!N)×(¯1*N-K)×(K+0,1N)*T
▽

```

However this function can be improved: it ignores the fact that the first term is identically zero, it does not make it clear that the result is always positive, and APL has a more concise way of summing alternating series. A better function is therefore given by

```

▽ Z←N STIR2 T;K
[1] Z←| (÷!N)×-/(K!N)×(K+1N)*T
▽

```

It may or may not be numerically preferable to factorise out the $n!$ and use

```

▽ Z←N STIR3 T;K
[1] Z←| -/(K*T)÷(÷!K)×!N-K+1N
▽

```

Of course APL's power comes from its ability to handle several different values simultaneously. We could easily have a vector for T .

```

      V Z+N STIR4 T;K
[1]  Z+|-f(K*. *T)÷q((pT),N)p(!K)×!N-K+1N
      V

```

However, it is likely that the values of T we would require, at least to form a table of the Stirling numbers (which might be useful in the long run to save calculation) would be all the positive integers up to T . This leads to

```

      V Z+N STIR5 T;K
[1]  Z+|-f(K*. *1T)÷q(T,N)p(!K)×!N-K+1N
      V

```

and we could have, had we wished, developed *STIR2* to this form instead of *STIR3*. If we wish to produce and save a table of the numbers we may build *STIR5* into a loop:

```

      V Z+N STIR6 T;I
[1]  Z+10
[2]  I+1
[3]  L:Z+Z,I STIR5 T
[4]  +(N≥I+I+1)/L
[5]  Z+(N,T)pZ
      V

```

It is now time to look at the table of numbers this function produces; this is shown in Table 1 for $N=6$ and $T=6$. It becomes very clear that a function which wastes its time calculating values for $N>T$ is unsatisfactory. This, of course, should have become evident if a little more algebra had been done before turning to programming, or if *STIR2*, say, had been properly tested, but the functions are so simple it hardly seemed necessary to test them all individually!

Table 1		t					
n		1	2	3	4	5	6
	1	1	1	1	1	1	1
	2	0	1	3	7	15	31
	3	0	0	1	6	25	90
	4	0	0	0	1	10	61
	5	0	0	0	0	1	15
	6	0	0	0	0	0	1

Development

It is important to define more coherently the problem we are tackling. We now want to produce a table of Stirling numbers in which the element at row n and column p contains $st_{n,p}^n$ and to solve the practical problem underlying the computing problem we would like n to take values up to at least 100 and p to reach at least $2n$ for as many values of n as possible.

This means rewriting *STIR5* as

```

      V Z←N STIR7 P;K
[1]  Z←| - / (K° . * N + 1 P) ÷ b (P, N) p ( ! K ) × ! N - K + 1 N
      V

```

Running this in a loop produces Table 2 for $N=8$ and $P=6$. The magnitude of the values appearing in the table gives rise to concern for the function: when will it overflow? This is certainly a matter which will have to be investigated, but there are other interesting aspects to the table.

Table 2

	P					
	1	2	3	4	5	6
1	1	1	1	1	1	1
2	3	7	15	31	63	127
3	6	25	90	301	966	3025
n 4	10	65	350	1701	7770	34105
5	15	140	1050	6951	42525	246738
6	21	266	2646	22827	179487	1323652
7	28	462	5880	63987	627396	5715424
8	36	750	11880	159027	1899612	20912320

From row 2 it is clear that $\sigma_{t+2}^2 = 2 \sigma_{t+1}^2 + 1$, and from row 3 it is not too difficult to deduce that $\sigma_{t+3}^3 = 3 \sigma_{t+2}^3 + \sigma_{t+3}^2$, and hence to discover the recurrence relation

$$\sigma_t^n = n \sigma_{t-1}^{n-1} + \sigma_{t-1}^{n-2}$$

This enables us to write a function without calculating the factorials or binomial coefficients in our original definition.

```

      V Z←N STIR8 P;I;J
[1]  Z←(N, P+1) p 1
[2]  I←2
[3]  NEWROW: J←2
[4]  NEWVAL: Z[I; J]←Z[I-1; J]+I×Z[I; J-1]
[5]  →((P+1)≥J+J+1)/NEWVAL
[6]  →(N≥I+I+1)/NEWROW
[7]  Z← 0 1 ↓Z
      V

```

But what is this? An APL function with a double loop? Surely that is unnecessary.

Instead of working a row at a time we must evaluate the table by columns.

```

      V Z←N STIR9 P;J
[1]      Z←(N,P+1)ρ1
[2]      J←2
[3]      NEWCOL:Z[;J]←+Z[;J-1]×1N
[4]      +((P+1)≥J+J+1)/NEWCOL
[5]      Z← 0 1 +Z
      V

```

Now we have a much faster function (taking about a quarter of the time *STIR8* does for $N=80$ and $P=80$), which is approaching its final stage, but let us look more closely at the table and the function we have written. Column 1 is just $+ \backslash 1N$ and column 2 is therefore $+ \backslash (1N) \times + \backslash 1N$. Indeed column P is $\pm 1 + (7 \times P) \rho' \times + \backslash (1N)$ an expression so concise it is hardly worth writing a function for. Notice that $\pm 1 + (9 \times P) \rho' \times \square + + \backslash (1N)$ prints out the entire table given by *STIR9*, although it transposes it and prints each line to a different format. This is avoidable, for small values of N and P , by $\pm 2 + ((12 \times P) \rho' \times \square + 10 \ 0 \mp + \backslash K')$, $' + \backslash 1N'$ but we wish to store the table, not print it, a task which is done by our final version

```

      V Z←N STIR P;K
[1]      ±((13×P-1)ρ'Z+Z,+K×(-N)†'),'Z←+K+1N'
[2]      Z←±(P,N)ρZ
      V

```

which is slightly faster even than *STIR9*.

Recapitulation

We have seen how it is worthwhile to look at the problem from a theoretical point of view so that we may obtain different algorithms for calculation. Even with an apparently obvious formula, simply translated into APL, it may be more appropriate to use a recurrence relation. This is not only for reasons of time, which are not of vital importance if the values obtained are to be stored in any case, but for reasons of numerical accuracy. In passing we suggested that *STIR2*, which involves the calculation of a factorial and a set of binomial coefficients, might have different numerical properties from *STIR3* which involves two sets of factorials.

To investigate this suppose $N=50$ and $T=60$, the sort of numbers which might indeed arise as the result of a small experiment. *STIR1* and *STIR2* give $\sigma_{60}^{50} = 2.02 \times 10^{25}$ while *STIR3* and *STIR4* give 5.06×10^{25} . The sensitivity of each function to the order of summing the series can be looked at by changing $- /$ in *STIR2* and *STIR3* to $- / \phi$; the results change to 1.71×10^{26} and 7.09×10^{25} respectively. With *STIR4* we can even alter the calculated value depending on which values other than 60 we include in vector T . *STIR* (and *STIR9* which uses the same arithmetic) gives 9.55×10^{24} for σ_{60}^{50} which I believe to be correct. Not only is the algorithm contained in $\pm 1 + (7 \times P) \rho' \times + \backslash (1N)$ remarkably succinct, it is numerically preferable to the others. *STIR* works whenever $350 > N + 2 \times P$ and occasionally outside that range.

When the function fails it is sometimes because the workspace becomes full, and sometimes because a domain error occurs.

The figure shows an isometric plot of the common logarithm of the Stirling numbers of the second kind, for values of n and $t-n$ up to 80. The values given above, except for that of 9.55×10^{24} , are system-dependent; the ones quoted were obtained from IBM APL on a Personal Computer, the same phenomenon occurs with different values using TRYAPL2. Other systems give a domain error instead of inaccurate results.

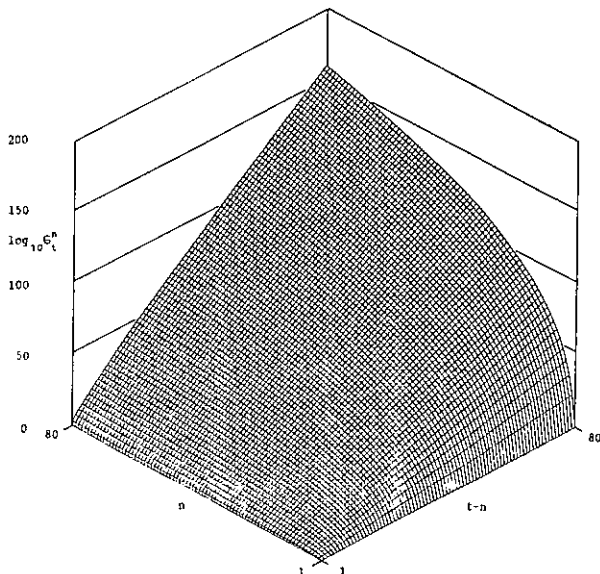


Figure: the Stirling Numbers of the Second Kind

Coda

The style of programming illustrated in *STIR* can be utilised in many instances of recursion, though the Stirling numbers of the second kind may be the most elegant. Readers might like to deduce what the following expressions evaluate.

```

1 ' + , ((9 * N - 2) p ' Z + Z , + / - 2 + ' ) , ' Z + 1 1 '
1 1 + ((16 * P) p ' x + \ ( - ( 1 N ) + K + K + 1 ) ' ) , ' + 0 * K + - 1 '
1 ' + , ((31 * P) p ' Z + Z + N p + \ ( ( 1 N + K ) , K ) p ( ( K + K + 1 ) p 0 ) , ' ) , ' Z + N p K + 1 '

```

Reference

- [1] Abramowitz, M. & Stegun, IA (eds). *Handbook of mathematical functions with formulas, graphs, and mathematical tables*. Washington: US Govt. Print. Off. 1964.

Gregynog 91

By the time you read this, the third ASL Conference will have just taken place at the University of Wales Conference Centre at Gregynog. We are expecting the usual 30 plus delegates of whom about half will be from the academic community. Our university delegates will include representatives from Swansea, Edinburgh, Birmingham, Newcastle and Monterey. The conference is primarily devoted to consolidating and advancing the ASL project.

Presentations are being given on testing ASL functions and also on proposals for ASL utilities. As the project gets larger and the software is released under more APL interpreters, this aspect of testing and management becomes more important. On the statistical front we have presentations which propose future volumes for the ASL:

- Time Series
- Smoothing and Density Estimation
- Contingency Tables
- Non-Linear Regression

and we shall be seeking to reach some collective consensus about the form and content of these shelves. We also will be seeking to review our experiences in working on the first three volumes of ASL, so that we can make life a little bit easier for the next volumes. Tony O'Hagan's discussion paper is reprinted below.

ASL - where have we got to and where do we go from here?

by Tony O'Hagan

As I step down from the ASL Management Team, I thought it might be useful to take stock of what has and has not been achieved so far with the project. This may help to set priorities and realistic objectives for the future.

What have we got? By the time of the third ASL conference, we should have formally released the first three volumes. Although they are late in coming, their actual release is a landmark in ASL. We must ultimately leave it to others to judge how good they are, but I believe they already form a very useful set of tools for any statistician. Volume 1 gives us a valuable collection of basic calculations. Even I sometimes want significance points of standard

distributions, or want to calculate a correlation matrix, and having those functions so conveniently to hand is great.

Volume 2 gives me a really good toolkit for building graphs. They are much quicker and easier to use than the basic low-level graphics functions supplied with most commercial interpreters, but much more flexible than the high-level suites of 'presentation graphics' functions that are often also provided. Of course, you cannot use Volume 2 yet if it is not yet implemented for your graphics system, but that is a good reason to call loudly for an implementation or port to your system (or do it yourself).

Volume 3 gives me a very powerful system for linear and generalised linear models. Many statisticians will need nothing more.

I think that we can be proud of what has been achieved so far, so if I now go on to dwell on what has not been done it is not meant as a criticism. The fact is, one can always do more and better. It is easy to focus on the 'more', which in ASL terms means starting on new volumes, but it is useful to think a little about the 'better' - how we can improve on what we have done.

Programming Standards

We set out with high ideals. ASL was to set the standard for statistical work in APL. Functions would be well designed, well programmed and well documented. I think we have achieved the first of these, because in my opinion the functions are well designed. We have not achieved the other two.

I know it is difficult to say what constitutes good APL code, but I am pretty sure that very little of what we have done would satisfy the best professional programmers. I know this to be a fact for Volume 2 because I wrote most of the final version! It was my first attempt at using second generation APL features in a serious application, and I am sure that I must have used them badly in many places. Alan Sykes would not claim to write consistently good APL either. Like me, he has learnt a lot through ASL, but Volume 3 grew out of a much older workspace, and probably looks patched and messy to any professional programmer. We talked about publishing our code, as part of 'setting the standard': with the current versions that would be unwise! It is debatable whether we should really try for good code, or settle for code that works. What should we do with future volumes?

Documentation

We also hoped to produce good documentation. A standard was set for internal documentation by comments. It was never written down and enforced, and I am afraid that under pressure to produce Volume 2 even this fell by the wayside. Did Volumes 1 and 3 manage to get this part right?

Documentation also means user manuals and technical manuals. This is something else that slipped through lack of time and manpower, but if ASL is to succeed it is vitally important. I believe that we must set as a high priority the production of high quality documentation of Volumes 1 to 3, in a consistent ASL-standard format, and that this must be maintained for future volumes. It is no good having good software if nobody can understand how to use it.

We also planned a book to introduce non-APLers to doing statistical analysis with ASL. This is a very big nettle that I believe must also be grasped. We know what we want to do; we even have a major publisher interested. But who will write it? And will it be done soon enough to be of any use?

The Library

ASL was expected to grow so big that good management tools would be needed for any user to draw into the workspace precisely the functions needed for the job. A Working Party was going to develop these, but did not deliver. As it happens there was no real urgency because ASL is not yet that big, but that is not a satisfactory view. We designed the functions carefully and we should design the library and its management carefully from the outset. Otherwise we will end up with volumes that need major revision to fit whatever format is ultimately chosen.

We talked endlessly about naming conventions and even appointed a 'custodian of names', but he was never given any names to take custody of! Who knows whether we have made any blunders? I know that in Volume 2 there is an odd hang-over from earlier conventions in the use of ASL as a prefix to names of global variables, but not as a prefix to function names. There was simply not enough time to resolve this.

Implementing WAGS

If Volume 2 is to be of any real value, a high priority must be its implementation on as wide a variety of systems as possible. We have a Dyalog PC version that works on screen and outputs to some printers, but is not tested on other printers because we did not have them available. We have an APL*PLUS PC version that

works on screen with the (obsolete) $\square G$ graphics but is not reliable for output on a printer. That is all. We do not need to do the development work ourselves if vendors are willing to take on implementations for their own systems. But we should take the lead in positively encouraging them to do so. Then what do we do about the ones that we know will not yield to that persuasion?

Future Manpower

I have spent some time on saying just how Volumes 1 to 3 fail to deliver at the level of our early high hopes. In thinking about the future of ASL, we should ask why. The reason is that wonders were achieved with the available resources, but the available resources were inadequate. Each one of the three volumes was developed by one or two dedicated people in their spare time. The work they put in dwarfs the input of any other person in ASL. Again, I am not really criticising. Whereas it was clear that without their efforts there would be no software, the effort of other people was always more marginal. Our working party on Standards and Implementation did relatively little not because of a lack of goodwill, but because the software could be produced (and was produced) largely without them.

So there are two manpower issues to face. How do we get new people to develop whole new volumes (particularly if they can guess how much work the first three entailed)? And how to we get goodwill to turn into a comparable amount of time and effort from those whose involvement is more peripheral? A lot of what I have said about the shortcomings of the current offerings revolves around the failure to bring forward this second kind of manpower - and make no mistake, this is just as important as getting in people to write new volumes.

We also need to decide whether we can make better use of a specialist programmer than we did do. Perhaps we were unlucky in the ways things worked out this time, but I have my doubts whether such a programmer can ever play a big part in the original development of a volume. There may be a compensating case, however, for a specialist programmer to come along afterwards to add any internal documentation, ensure adherence to standards (when we have them) and improve coding.

Not doing everything that the last minute

One thing is clear, software always ends up with a frantic rush to deliver by some date. No matter how many times the delivery date is postponed (which is usually somewhere near infinity) there is always a last minute rush. In fact, the

more postponements it has had the more bouts of frantic effort the poor finished product will carry the scars of!

Therefore we cannot expect the refinements of good documentation, thorough testing and good coding to be put in at the end. Either they are built in all the way through or they will not be there at the first release. I have a suggestion - we should try hard to build in these refinements all the way through but, when we inevitably fail, we should plan for an early second release in which they are all there.

So, give Release 1 of Volumes 1 to 3 a few months to find the main bugs, then go full ahead to give Release 2 all the good things that I have said they lack at the present. Now is the time to recruit an experienced programmer to work on that. Now is the time to get the Standards and Documentation teams up to speed.

Some final words

I am sorry I could not be at Gregynog this time. I hope to be involved with ASL in the future, through a continuing interest in Volume 3, as a user of ASL and perhaps as a contributor to new volumes. I would like to see ASL go from strength to strength, still trying to set 'the standard'. Although lofty ideals must always be compromised in practice, let us not lose those ideals. They are what drives us forward.

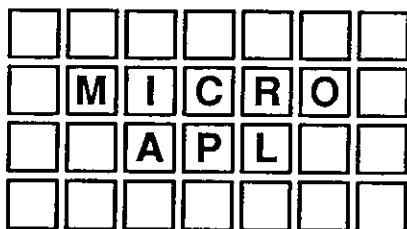
Some things are worth the wait

APL.68000 is not the only APL available for IBM's RISC System/6000. But no other APL running on the RISC System/6000 offers all these features:

- MicroAPL's acclaimed windowing interface
- Seamless integration with X-Windows
- Adherence to OSF/Motif style guidelines
- Very close compatibility with APL2/370
- Binary workspace compatibility across platforms
- High performance through MicroAPL's unique "Portable Assembler"™ technology

APL.68000 for the IBM RISC System/6000

*Fourth to announce.
First to get it right.*



MicroAPL Ltd.
South Bank Technopark
90 London Road
LONDON, SE1 6LN
Telephone: 071 922 8866
Fax: 071 928 1006

RECENT MEETINGS

This section of VECTOR is intended to document the seminars given at recent meetings of the association; it is of particular value to members who live away from London. It also covers other selected events which may be of general interest to the APL community.

If you would like to speak at one of the regular British APL Association seminars, please ring the Activities Officer (address on inside back cover) who will respond enthusiastically to your offer.

Acknowledgements

Notes on the sessions are credited by contributor ... many thanks to all those who helped the working group to document the conference so fully.

The photographs are by:

John Searle
Sylvia Camacho
Anthony Camacho
Ray Cannon
Phil Benkard
Adrian Smith

Many thanks for getting so many excellent prints to us so quickly, and apologies for not using more of them.

APL 91 REPORT

Conference Round Up

reported by Anthony Camacho

APL 91 was a social, financial and technical success. About 400 people attended, well over the break-even figure and all visible signs were that everything went to plan.



A Typical Stanford Arcade

We stayed in the student accommodation. We would not have paid \$110 a day for our non-air conditioned room with communal washing and lavatories if we had known we could get a good clean air conditioned motel room for \$41 a day. The \$69 a day difference would have paid to hire a car and eat grandly, leaving plenty over. The food, though not worth anywhere near \$69 a day, was excellent. I didn't hear a single complaint about it.

There was no formal banquet; I don't know whether it was replaced by the Exploratorium Evening or by the final barbecue picnic. The food and drink at both were excellent.

Sessions began early at 0830 and I felt we were worked really rather hard, but the quality of the papers I attended was as high as any conference I've been to.

The Conference Proceedings, edited by Jan Engel, were if anything of a higher standard than ever before. I am sorry that the Danish precedent could not be followed. I would have benefited greatly from a chance to read proceedings before the conference. I found I simply did not have time to look at papers before attending presentations and that too many presenters assumed that I had. Every evening was occupied, some by more than one event. Sunday evening there was a welcome reception, Monday an Exhibitors' reception and a Russian evening, Tuesday the Exploratorium, Wednesday a visit to Fry's and on Thursday, for some hours after the final session there was the final picnic.



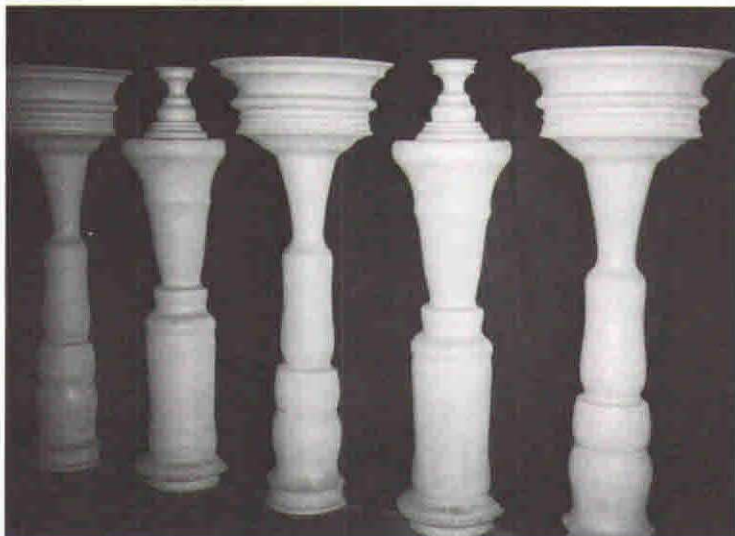
The Software Exchange

The panels I attended were better than most previous conferences. Each speaker seemed to have something prepared which was a reasonable length for the time allotted. The worst over-run was the 25 year panel - Ken only began to speak after it should have ended - but Roy Sykes was right to let it run.

The plenary speakers were Phil Smith of IBM on IBM's Personal Scientific Laboratory, Therese Myers of and on Quarterdeck Office Systems, Dr William Scherlis of DARPA and Ted Nelson of the Xanadu hypermedia project. Phil Smith did a neat demonstration and William Scherlis gave an excellent summary of DARPA's work on the computer side. Therese Myers, whose excellent products I use, seemed to me to do a straight plug for her company. Ted Nelson went on too long and had too few visual aids.

Tresidder Union was a good place to hold the exhibition. It was equipped with cafes and a bar. There was plenty of space to sit and talk or rest. The notice board, the enquiry desk and the software library were well arranged.

There was also a museum of historical APL exhibits - first editions, the first logon to a cleanspace and an interesting collection of APL ephemera - tags, stickers, labels, posters and so on. The museum exhibits were collected and displayed by George Mendonsa, who would be glad to hear of other items which could be given or lent to the museum for next time it is displayed.



People talking - at the Exploratorium

The Exploratorium describes itself as 'a playful museum where you make discoveries for yourself'. It has over 650 experiments which the visitor can take part in, arranged in 13 large subject areas. A good many APLers are overgrown

kids and this suited us perfectly. The experiments are designed with flair: you are sat on a freely rotatable stool before you try to turn a bicycle wheel gyroscope through 90 degrees; there are trays of soap bubble mixture and frames to dip into them; there are wind (with condensed water vapour) and water (with sand) vortices; lots of things to hit or listen to; dozens of computer keyboards controlling things and even frogs' legs and octopus nerves set up to respond to controllable electrical stimuli. Among all this were tables of several varieties of ethnic food and more tables giving out as much beer or wine as you wanted to drink. I never got to the beer, and it was good beer, because the wine was so good. It was the right recipe for an excellent evening. I had a wonderful time. Congratulations to all concerned.

The conference debriefing is really confidential, but I'm sure I don't reveal too many secrets by saying that when the Meeting Planning Associates staff left (before we went on to discuss the future) they were given a spontaneous ovation.

reported by Jonathan Barman

One of the best things about any conference is meeting people, and this conference was no exception. The social events were informal and relaxed, and it seemed easier than usual to mix and talk APL and other gossip.

Hiring the whole of the Exploratorium for the evening normally reserved for a formal banquet was a brilliant idea. The Exploratorium caters for children with enquiring minds which exactly fits the APLers attending the conference. There was so much to do and see that if I had tried to cover all the items there would have been no time to eat or chat.

There was an evening outing to a computer supermarket called Fry's. I experienced culture shock due to the sheer quantity and range of goods for sale. The store was laid out in much the same way as a conventional supermarket. There must have been over 100 different computers on display, and there was a complete aisle of shelves filled with different sorts of fax machines, and another with telephones, then another with memory chips, and it seemed to go on and on.



Charles Schulz, Therese Myers,
Tibor Horvath

The administration of the conference was excellent. The only area where there was a minor slip was for the tutorials. As the tutorials started at 8:30am on the Sunday, before registration, everyone who booked should have been notified when and where they started. There was a bit of milling about, but everything was soon sorted out.

reported by John Searle

APL91 was a very successful conference, with over 400 delegates, which I understand was the break-even point for financial success. Stanford University is a charming campus and a very lively place at all hours. I think that the Stanford Bookstore is the best technical bookshop I have ever been in.

The campus accommodation was adequate and clean, even though I did tend to bounce off my bed whenever I lay on it. Or perhaps that was the fault of the late night glasnost sessions. (For "glasnost" read "vodka".)

reported by Ray Cannon

As Ted Nelson said in the closing address "Stanford University is a city". Many a time while at Stanford, I thought back to my university days at Salford (part of Greater Manchester), and compared the two. Stanford won hands down (with the possible exception of cricket at Old Trafford). May I congratulate the organisers on their choice of site, the generally good weather (remember the Manchester 86 umbrellas?), the Welcome Reception (even if it was a bit windy), the visit to the Exploratorium (enjoyed by kids of all ages from 7 to 70, and I swear I saw the base of one of the Golden Gate towers through the fog on the way there), and the Closing Picnic (I was lucky, and found a table in the shade). So once again well done and thank you.

Once again, there were (as in previous conferences) 3 streams for the presentation of papers, so any one delegate could only hope to attend at best one third of the talks. At APL90 the proceedings were available well before the conference, and one had time to read the papers and choose which to attend. Unfortunately, this was not the case at Stanford, but the extract that appeared in the final program provided a good guide to deciding which to attend.

Sunday 4 August

Introductory Tutorial: An Introduction to APL by Ray Polivka

reported by Anthony and Sylvia Camacho

One of the most notable features of APL is the low threshold that newcomers need to cross to do useful work and the high ceiling they discover as they challenge APL with more complex problems. For a teacher this poses its own challenge: whether to reassure by demonstrating the low threshold or impress with a brief glimpse of the high ceiling. Ray refused all compromise and chose to give his audience a glimpse of the imposing height of a full nested array implementation. This required that he went beyond the ISO standard and be frank in his partiality for APL2. What resulted was a tour de force by a practised teacher, and notable for the ability of Speaker and Chairman to finish all sessions to specification and on time.

Ray Polivka's Tutorial was billed as an introduction to APL and was clearly not intended to impart skills to the participants. It was in two parts: a description and illustrations. The description covered Array Orientation, the Rich Set of Primitives, Syntax & Grammar, Programming, Operators and Control Constructs. Ray gave those aspects of APL which so often alienate newcomers, a hard sell. He described APL as a conceptually parallel, highly symbolic array-oriented language which had been 10 years in gestation. He showed Donald McIntyre's list of mathematical symbols and the dates each was first used. Perhaps it is not so odd that logical AND ($\wedge$) was only first used in 1933; it is amazing that PLUS (+) was not used before 1489.



Anthony Camacho and Ray Polivka

After this short toe-dipping session Ray pushed his audience into the deep end. He started with heterogeneous arrays, concepts of dimension, vectors of vectors, nesting and definitions of 'depth'. For the programmers he made the point that APL is self-describing and that the data carries with it a description of its own

data type, shape/rank and structure. Complex numbers were mentioned in passing as an example of a vector.

In what amounted to a digression, the relation of APL (as a computer language) to its environment was given a slightly breathless overview covering system commands, libraries, workspaces and the nested array display function.

Now was the time to 'taste the beef' with a review of APL2 language pure and not simple. With a reminder that 'APL manipulates whole arrays as well as items within them', Ray gave his gloss on the job of a programming language, saying it must do simple things simply, such as data transformation, array construction, array enquiry, data selection, data rearrangement, and data replacement.

With this as a framework the primitives were then presented, including such APL2 extensions as string search and enlist. Passing from primitives to definitions was then made a little more challenging by moving from defined functions to defined operators. 'There is no difficulty', said Ray, 'functions use variable data, operators have variable functions'.

Within two minutes of the appointed time the pupils were dismissed to afternoon coffee break with the promise of a final session to look at the solutions they had found to the problems they had been set to keep them from idleness over lunch.

This hard day's work was advertised as 'free'. Those attending might, if they wished, buy, for \$25, a package of papers and software demonstrating various aspects and implementations of APL.

This contained Ray's slides, leaflets from vendors, and three free APL interpreters: TryAPL2 (manual and disk), I-APL (manual, tutorial and disk) and APLISW (the shareware version of Sharp APL/PC now being produced by Iverson Software Inc). This last does not need a manual as it includes extensive help files on disk and some of the start-up options are no longer the problem they were with Sharp's version.

They certainly had their money's worth!

Introductory Tutorial: An Introduction to APL by Ray Polivka

reported by Lee Ann Estep

During the APL91 conference I enrolled in the Introduction to APL Tutorial. My expectation of a beginning APL class was that it would begin at the beginning and proceed at some reasonable rate toward, perhaps, the beginning of the

middle. What I needed was the elementary school version of beginning APL, but found myself in the graduate school version.

Several minutes actually were spent on the beginning but by the lunch break we had covered the beginning, middle, and the far end of APL. The majority of the material went so far over my head it never even ruffled my hair. No enquiry was made as to the level of ability or expectations of the students. We were treated to a pre-packaged script delivered at lightning speed. Questions were not encouraged.

There was no sign of a computer, either for the students, which I did not expect, or for the instructor, which I certainly did expect.

At the beginning of the session I knew nothing about APL. I still do!

Advanced Tutorial: Computer-Aided Software Engineering (CASE) by Larry Zeidner

reported by Jonathan Barman

I found the initial sessions of the CASE Tutorial rather difficult to understand. I am still not sure just what CASE is supposed to be, apart from the initials standing for Computer Aided Software Engineering. It looks as though most of the APL tools and utilities that people usually create for themselves could be described as CASE. The demonstration of the new editor in IBM's APL2/PC seemed to be irrelevant to CASE, though the editor itself looks useful.



Larry Zeidner and his Multiprocessor

The afternoon session turned out to be very interesting. Dr Zeidner gave a scintillating presentation of his CASE tool, which enables the APL programmer to create a system which will run numerically intensive computations on several machines at once. The particular application for which the CASE tool had been used was in the calculation of the complex path a numerically controlled machine tool has to traverse when machining an arbitrary shape. The Conference Proceedings contain a paper by Dr Zeidner which describes the details of the system.

After the official tutorial was over many of us went to a demonstration of the software. It was fun to see a partly machined golf club head, which Dr Hazodny brought along, and then see the system calculate the path that a machine tool would have to traverse to make some of the initial cuts. The number of calculations involved is enormous, so farming out the job to several machines at once is essential if the job is to be finished in a reasonable time. Even with multiple processors the full golf club head machining calculations take all night to complete.

I feel sure that this CASE tool will be applied in many other disciplines.



**The Welcome Reception
4 Smiths and Ray Cannon in the Background
Ed Cherlin and The Cat in the Hat in the Foreground**

Monday 5 August

Invited Speaker: Phil Smith.

Reflections on Magic, Dreams, and Programming

reported by Jonathan Barman

Phil Smith came onto the platform with electronic sensors attached to his forehead. These were hooked up to a PC display so we could see his brain waves in action during the presentation. The first 10 minutes were pre-recorded, with both voice and PC display. I could not make out if he was controlling the recording by changing thought patterns, or if he was just well rehearsed. Later on he gave a demonstration of choosing an item from a menu by thought control, but again I could not make out if it was real. I feel sure it was not a hoax.

I thoroughly enjoyed the display of capturing data from simple devices, and then using APL to display the results. One of the devices was based on the infra red measuring instrument used for focusing a Polaroid camera, and could measure distances very fast and accurately. The PC captured the stream of distance measurements, and then APL functions were used to display the data. Phil walked towards the device and then displayed the speed and acceleration of every footstep. He put it on the floor and flexed a bouncy spring over it and then showed the resulting damped sine wave. Another device was a freely rotating arm with an accurate measure of the angle traversed. Demonstrating that the frequency of a pendulum depends on length and not on amplitude could be carried out in minutes. When I was at school the same thing took days of preparation and computation, and at the time seemed quite tedious. There must be some very lucky students who benefit from Phil's work.

MicroAPL Vendor Forum

reported by Adrian Smith

This was probably the most interesting vendor session, as MicroAPL had taken a good deal of trouble to keep their product under wraps until the conference. It was however slightly marred by the fact that the conference Barco stoutly refused to accept the signal from the RS/6000.



MicroAPL's strategy is similar to IBM's

David Eastwood and Richard Nabavi

in that they have one source code copy in a relatively low level language (68000 Assembler / Series-1 assembler) which can be transliterated into other assemblers, as long as these have a broadly similar structure. With the SPARC and RISC instruction sets, the mapping is typically 1->2 or 3, but the structural fit is quite good, so the end product runs quite fast. The other respect in which MicroAPL resemble IBM is in their total commitment to the APL2 standard. This leaves the user in the fortunate position of being able to choose a totally compatible APL on any of the PC, Mac, Unix or mainframe platforms; whatever your views on APL2, this has to be good news.

The user-interface is fully Motif/CUA compliant, with all the standard cut&paste actions, an easy printer interface and the complete APL manual available as on-line help. The interface to Motif was designed from the outset to be easy not to use! The problem with managing 1034 C-calls from your APL application is that you have to turn yourself into a C programmer to do it. MicroAPL have tried to make a reasonable subset of the facilities available to APL with a simple set of functions (e.g. *ALERT* to put up a warning box) which are designed for APLers, not C-folk. The Motif resource mechanisms are used to fill in the details, e.g. if you want a dark blue button you set that up in Motif, and APL will automatically use it.

The next steps are towards SPARC (MIPS and HP machines), more optimisations, a clean implementation of *WNA* and an OpenLook version. MicroAPL intend to offer a free runtime policy with a packager available 'at a modest price'. They are beta-testing through September, and are aiming to launch on September 26th.

Panel: Modes of Thinking and Their Implications for APL

reported by Jonathan Barman

Hal Macklin has a special interest in the way the brain processes APL. Research with people who have suffered brain damage has shown that Japanese Kanji characters take a different route through the brain compared with normal script. Mathematical symbols seem to bypass language areas of the brain, and so does music.

Devon McCormick stressed the difference between parallel and serial modes of thought. APL needs parallel thinking which seems to be more complex, but APL enables one to cope with complexity. Someone coding a string search routine serially usually fails to notice the overlapping match problem, but coding in APL in parallel highlights this problem.

Phil Smith has had personal experience of the way APL changed his way of thinking, and has analysed the nature of this change.

Stan Kelly-Bootle felt that APL was not the only area where special thought processes must occur. He was impressed by the fact that the topologist who finally solved the problem of turning a sphere inside out happened to be blind.

Hal Macklin wondered why APL provoked such strong reactions from people. Could it be that an extraordinary number of the APL creators and enthusiasts are left handed?

Stan Kelly-Bootle thought that APL is not dying, but is just a minority cult.

Psycho-Biographical Analysis with APL, by A V Kondrashev & A A Kronik

reported by Sylvia Camacho

The subject matter of this application of APL is intriguing: the investigation of differences in subjective assessment of life experiences and expectations of longevity between populations assumed psychologically balanced and those requiring therapy. We were told that the prototype for this model was produced in one day and then developed in spare time. The model allows data to be captured, transferred to a database, subjected to statistical analysis and the output to be presented graphically.

One finding of the research is that expectation of a long life is a good indicator of psychological well-being. A recent analysis of differences between groups of men and women in their attitude to perestroika showed, on this assumption, that soviet men are considerably more optimistic than soviet women.

Supply-Chain Management at Rowntree: Critical Success Factors for APL, by Adrian Smith

reported by Anthony Camacho

The presentation was a first-rate guide on how to survive with APL, disguised as a workmanlike summary of the advantages and disadvantages of APL in the Rowntree-Mackintosh environment.

I extracted the following precepts: don't use any vendors' special features; they tie you to the vendor and make your APL non-portable. Don't use any vendor-supplied workspaces. Only use code you fully understand and can maintain.

Write cover functions for machine-dependent features such as graphics to simplify porting. Prefer PC work to mainframe; the PC user interface is far better. Don't use enhancements. Nested arrays and defined operators greatly increase the amount staff have to learn before they become effective. Buy faster PCs to make such simple code fast enough for your needs.

Collect a well designed and tested, centrally maintained set of utilities. This will improve robustness, reduce training time for new staff, reinforce coding standards by example, and greatly reduce development time on new systems (in Rowntree 85% to 90% of a system is utilities). Bear in mind that the scope of packages will grow. APL's advantage is as a notation for expressing hard sums effectively; use APL in circumstances where this advantage is decisive and don't fight the battles you might lose.



Adrian Smith

reported by Ray Cannon

Adrian Smith's talk on Supply-Chain Management was the most amusing, and all done without mirrors or any other visual aid.

Invited Speaker: Therese Myers.

Simplifying the View of a Complicated Technology

reported by John Searle

Therese Myers is President of Quarterdeck Office Systems of Santa Monica, California, the software company responsible for DesqView, which is a "multi-tasking" organiser for DOS machines. Her talk was an aggressive marketing pitch for her company and its forthcoming DesqView/X product, which will allow PCs to simulate X-terminals in a Token Ring or Ethernet network. This will enable existing investment in DOS machines to be carried forward into a Unix workstation environment.

The basic theme of her talk was that Quarterdeck is better than Microsoft, and whatever we do they copy about two years later. Now that Microsoft is the top of the heap, I suppose they are the prime target for criticism, but I felt there was too much emphasis on the put-down in her talk.

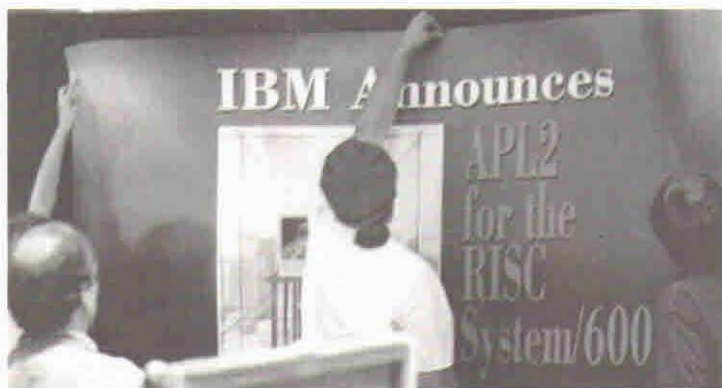
In technical terms, Quarterdeck have ported X11 Release 4 to DOS, and the user interface will be able to look like either Motif or OpenLook. Networking will be available through TCP/IP, Decnet or PC/NFZ.

The only reference to APL in the talk was that an APL Application Programming Interface is available "if there is an interest in it". (For "interest", read "profit".) It has been written by one of her APL-enthusiast colleagues. Unfortunately, her view of the limited APL market will probably prevent it from emerging.

The Exhibition Area

notes by Adrian Smith

Suddenly, all APLs are starting to look the same! Even Digital were there, running under DECWindows, to merge with the crowd of RS/6000s. STSC were oddly placed in the exact centre of the room, and seemed to be ignored by the majority of delegates who followed a natural path around the four walls, and thus found themselves back at the exit without ever visiting the little island in the middle!



Getting the IBM Stand Ready

Dyadic were actually showing their APL under Windows-3, but I was advised in no uncertain terms that it was more than my life was worth to even think about touching the mouse. They were right ... even under the careful guidance of John Scholes it still crashed with glorious unpredictability at the worst possible moments. IBM and MicroAPL were there too, but I couldn't get their stuff to break ... how boring!

Tuesday 6 August

Mastering J, by Donald McIntyre

reported by Anthony Camacho

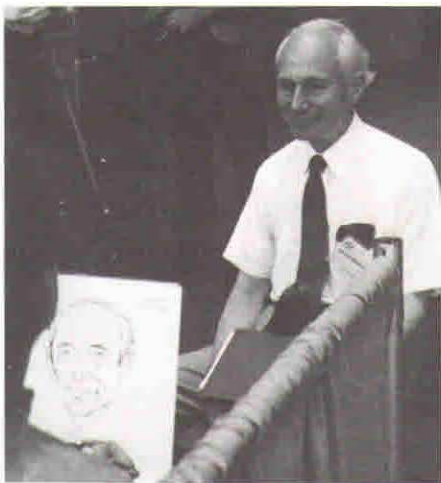
APL, which nearly got called Iverson's Better Math, encounters resistance because it requires people to learn new symbols. Now, ironically, J is encountering resistance among APLers because it does not use the symbols they know. Donald admitted he himself suffered distress making the change.

It is, in any case, the ideas that matter and Donald McIntyre has no doubt that the ideas in J are an improvement on APL. J is the best as well as the latest version of Iverson's notation.

Donald is still Mastering J and feels that his difficulties may be encountered by other non-theoreticians. He needed more examples than are given in the terse Dictionary of J and he sets out to provide some in this paper.

Through the good offices of Vin Grannell this session was videorecorded. With luck, if the recording was successful, Donald's presentation may reach a wider audience.

One of Donald's specialities is illuminating asides. I learned that we owe the rank operator to Arthur Whitney and that a conjunction in J, is like a dyadic operator in APL. I assumed that a conjunction and two verbs produce a verb. Then one of his examples produced a verb from a conjunction and nouns! I didn't note the phrase; I hope it is on the tape.



Donald McIntyre with Peter McDonnell's Drawing

Tacit Definition, by Roger Hui, Kenneth Iverson and Eugene McDonnell

reported by Anthony Camacho

I felt it fitting that Gene McDonnell presented this paper because, in *The Four Cube Problem* (APL Press 1981) he presented to the world the best example I've seen of how to write programs without variables. It convinced me that there was something superior about direct definition.

Within the direct definition, however, one did refer to arguments as α and ω . Gene pointed out that there is no ambiguity about the meaning of the expression $+/$ even though there is no argument. Further expressions may be added which specify what to do with the result returned by $+/$. In J, such collections of expressions can be assigned to a name. Some of the features of J make this extremely terse and elegant style of programming much easier to use than it was in APL; such things as *fork*, *hook*, *with*, *atop* and *commute*.

Explicit definition in J uses $x.$ and $y.$ where APL used α and ω . In J one can define average explicitly, thus: `average=. (+/y.)%#y.` or, using tacit definition, thus: `average=. +/%#`, which uses a *fork* (q.v.).

The later versions of J contain a primitive (`:11`) which converts an expression from explicit to tacit form. This is an extension of the parser. A tacit definition does not need to be parsed but can be executed at once so tacit definitions are, in effect, compiled and are executed significantly faster. Gene said that the parse tree also indicates what can be done in parallel and what has to be done serially. This augurs well for a J implementation on parallel processors.

A questioner recalled that Al Rose, in 1966 or '67, proposed the high minus as a suffix should be the symbol for commute with the mnemonic 'the dashing commuter operator'.

Nuclear Power Plant Diagnostics in APL, by Alexander Skomorokhov

reported by Sylvia Camacho

This paper, in the proceedings, is a sober professional piece of work. The paper as presented was a pleasure to attend and a triumph over the odds. Alexander began by thanking his audience for attending despite the rival attraction of a paper by Iverson. He followed this by a cartoon show that produced appreciative laughs for every overhead he showed. This made his apology for his English pronunciation 'you should consider it as noise on the line' quite superfluous.

Mention of a model of a nuclear power plant was accompanied by a picture of Madonna and Child followed by a reference to Chernobyl and Three Mile Island and a picture of the Old Man with the Scythe.

The timescale for the application is interesting. Their first introduction to APL was via Gilman and Rose in 1979 and they realised their own interpreter in 1985. The incentive to use APL to try for early warning of incipient problems within nuclear reactors was explained by a cartoon of a Greenpeace Protest.

The first version of the system was in 1989 and, as a result of a meeting with Per Gjerlov, Erkki Juvonen and Jim Brown, this was converted to APL2 with a consequent reduction of two thirds of the code. In the way of Management world-wide this was met by a question from the boss 'Why do we have to pay more for less code?'

The final 'thank you for listening' was displayed on the screen including a seven replicate of 'very' much and a good round of applause for a good Show Man.

Questions revealed an interesting concern among the audience that APL should be used in such security-critical circumstances. The response was that APL is currently an investigative tool and prototyper and would probably be converted into other languages for use in production.

Invited Speaker: William L Scherlis, DARPA Software Activities and Opportunities for APL

reported by Anthony Camacho

Dr William Scherlis of the Defence Advanced Projects Research Agency (DARPA) told us that the agency's mission was to develop advanced technologies for the Department of Defence: it has 160 staff and a budget of a billion dollars a year.

There are two kinds of activity, military-specific (with a known military aim), and multiple-use which covers computer systems and scientific research.

On the computer systems side there are four main efforts:

1. High performance computer systems - teraflops is the goal.
2. Advanced software technology - the grand challenges here are:

major problems such as the numerical simulation of thunderclouds;

the component-based approach to managing large software systems (a million lines of code);

automatic conversion from serial to parallel.

3. The national research and education network - the aim is gigabit speed networks. Arpanet is part of this effort.
4. Basic research and human resources.

Software has to cater for a long life cycle of twenty years or more, much higher reliability than for commercial work and must be organised so that requirements that change during development can still be satisfied.

The component-based approach to managing the development of large systems must allow each component to be developed by an appropriate method; not all necessarily in the same language or even the same computing environment.

Dr Scherlis ended his survey by listing the opportunities he saw for APL.

- as one of the systems for development
- as a fast prototyper with a wide spectrum
- as a rich source of ideas for array-specific designs

and he wondered whether it was possible to scale up the interactive experimental APL approach so that large systems could be developed faster.

DARPA is a technology investor - not a policy-making organisation. Its prototyping project is unlikely to lead to any language being specified as the prototyping language for defence.

reported by Jonathan Barman

It was interesting to hear that APL can fit into the mega software projects that DARPA initiates. The main requirement is that APL should be able to interface smoothly with other languages and systems, for example, an interface between APL and Oracle.

The primary opportunity for APL is in the rapid prototyping of systems. Some concern was expressed from the audience that one of the software projects is to identify a common prototyping language for DOD. Apparently there won't be an 'official' language.

Plenary Session. APL Standards Report

reported by Anthony Camacho

Professor L J Dickey brought us up to date on the latest meeting of the Standards Committee. About ten people attended, from USA, Canada, France and Belgium. IBM, DEC and STSC were represented: others were users or consultants. The meeting agreed to add enlist, pick, depth, match and each (on floating nested array systems) and are moving in the direction of adopting strand notation. They may even make 1 2 3[2] an error to allow them to adopt strands. [Your reporter expected a chorus of boos at this point: there was not one.] Anything added will be as addenda to the standard. The group hopes to speed its work by greater use of electronic mail.

ISO group 11 is interested in language-independent data types and arithmetic.

There are two groups standardising character sets. Unicode is supported by about a dozen of the major players and aims at a single character encoding. ISO 10646 is a competing effort. The APL standards group is trying to persuade both groups to include all the APL characters. If they succeed it will simplify moving functions between platforms.

The User Command Processor, by Jim Weigang

reported by Ray Cannon

Jim Weigang's paper on STSC's user command processor was, to me, the least interesting talk that I went to, but that may be because over the last year, I have implemented and now support over 150 commands using the User Command Processor.

The paper was a blatant advertisement for a useful but badly implemented product. Fortunately it is possible to improve the implementation, and Dave Ziemann and Peter Cyriax have done exactly this.

Pure Functions in APL and J, by Edward Cherlin

reported by Anthony Camacho

Ed Cherlin made a few concessions to people who did not know something about combinatorial logic, but not enough for me; all my logic was learned before the seminal book *Combinatory Logic* was published. Concealed among the logic was the proposition that all expressions in J can be written without explicit reference to variable names. In some sense, if this is true it makes J functionally complete.

Ed proceeded to prove this proposition to his satisfaction, but I did not follow the proof.

The advantage offered for functional completeness did not seem very great. If all expressions can be written as abstracts (i.e. without variables) then this simplifies the construction of expressions under program control and the application of operators (adverbs) to expressions. I had the feeling that I had missed something; in my bones I feel that the elegance of abstracts (the same things, I am sure, as tacit definitions) must bring other extra advantages. But my bones are not exactly a reliable oracle.

Gerunds and Representations, by Bob Bernecky and Roger Hui

reported by Anthony Camacho

Someone (Arthur Whitney?), at APL 89, when asked to give the three most desirable improvements to APL answered (1) speed, (2) speed and (3) speed. Bob Bernecky and Roger Hui are interested in compiling for speed. "The real reason for disliking GOTO is that it makes compiling difficult." Bob wants to be able to compile for parallel processors. Most languages are Single Instruction, Single Data (SISD). APL is Single Instruction, Multiple Data (SIMD). Bob wants to find ways to compile APL so that it becomes Multiple Instruction, Multiple Data (MIMD).

His slide on motivation read:

No syntax change

Generalised adverbs

Improved reliability

Easier compilation

Partial support for MIMD

Parallel control structures: CASE, IF, DO.....

Makes verbs first class objects (I think this means that verbs can be the subjects and results of computations)

A gerund is a form derived from a verb which can be used as a noun: for example the word programming in the sentence programming is an art. This paper is a proposal that an operation done to a verb in J should convert it to an atom. This atomic form can be handled like a noun. Another operation could

convert the noun back to verb and as a result various interesting control structures can be created.

The verbal nouns thus formed can be put into arrays and some of the benefits that various people have suggested might arise from arrays of functions can be obtained. A design goal is to make compilation possible without limiting the expressive power of J.

APL Technology of Computer Simulation, by A Boozin and I Popselov

reported by Sylvia Camacho

Both authors of this paper are academic mathematicians and their use of APL is just where one might expect it to be: in modelling linear differential equations and stochastic Markovian processes.

Questions put showed that they first used APL 15 years ago on terminals to what was described as 'a big old computer'. Currently they use APL*PLUS/PC. They have set up an environment in which they can split the screen and display data graphically with some zoom facilities. This environment can be used in conjunction with raw APL while exploring the results of a model, or can be used in conjunction with data derived from APL or some other source.

In fact it is often used by Pascal programmers. Most recently they have been setting up economic models to try to come to terms with the consequences of decentralisation and privatisation. They use both Roman and Cyrillic alphabets.

Panel: Is J a Dialect of APL?

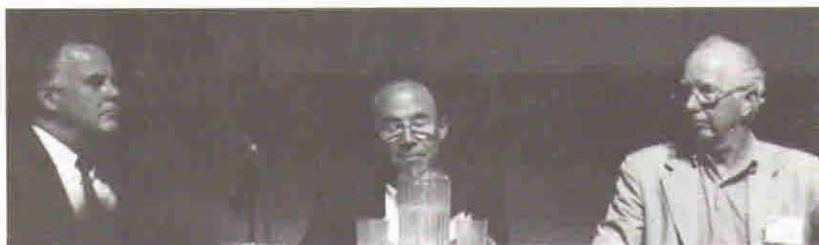
reported by Jonathan Barman

Eugene McDonnell - The question is irrelevant. Surely proponents of J would not be thrown out of the APL community?

Garth Foster - J is quite definitely not APL. J is spelled differently from APL and has different syntax. There are many J constructs which cannot be translated to APL, so it can hardly be called a dialect. ISO APL allows extensions to the language, but J cannot possibly conform. Speakeasy and Q'Nial were never considered dialects of APL.

David Steinbrook - What is a dialect? Comparing SAX and J shows many similarities. J gives us the power we all wanted for the last 20 years. APL is really a dialect of J!

Phil Benkard - This is a political decision, but political decisions affect our lives. Many aspects of J are different from APL. Functions are referred to as Verbs, box is different from nesting, hook and fork are new in J, and strand notation is different. No formal decision can be made today, but what political decision should be made?



**Is J a Dialect of APL - the "I don't cares"
Joey Tuttle, David Steinbrook, Gene McDonnell**

Joey Tuttle - Who cares if J and APL are different? Hopefully new insights will come from J and SAX which will enhance APL.

Richard Nabavi - Is US English a dialect of English? Richard cannot understand J. The academic view of a language is different from the commercial view, and sometimes the technically best solution does not win. For example DOS is technically poor but has wide support. The main objective should be to reduce the dialects of APL so that it can be promoted to a wide audience, and can be standardised. Will there be a J92 Conference?

David Weintraub - A standard is now available for APL, but will there be one for J? J seems to be more of a write-only language than APL. Could you implement APL in J?

Bob Bernecky - APL and J ideas need to be disseminated to the larger world of computing, and it does not matter what language is used. The character set inhibits APL. J is more compilable than APL, and has simpler syntax. The semantics of J are totally regular. Several mistakes were made in APL, and J is a new start where these mistakes have been rectified. J is not a dialect of APL, it is a functional language.

Garth Foster - Don McIntyre took a long time to learn J. Perhaps J is a successor of APL, but may not be a success.

Richard Nabavi - What versions of J are available? We should agree on the portrayal to the outside world.

Donald McIntyre - Will the ISO Standards committee decide whether J is a dialect of APL?

Phil Benkard - The APL2 syntax is simple, and the syntax and semantics are separated. There were mistakes in APL. It was disappointing that there was nobody present at the last Standards meeting representing the Sharp APL or J community.

Jim Lucas - APL and J form a family of languages.

Garth Foster - Everybody should buy and learn J. There is a history of language development, for example Pascal gave birth to Modula 2 and ADA. Let the market place choose whether APL or J is best. Teachers should use J, not APL.

Ed Cherlin - It is interesting that we are discussing the question at all. Why is this the one topic we want to argue about? Papers on J have been accepted at this conference, and will continue to be accepted.

Doug Bohrer - How many in the audience are new to APL? A show of hands indicated that there were very few.

Leroy Dickey - The J tutorial takes a page a day to complete.

Ken Iverson - The dictionary of J contains an introductory comment that J is a dialect of APL, so in a sense the whole debate is Ken's fault! He is flattered to think that he has actually created a new language.

Phil Benkard - Many of us are amazed that Ken has invented a new language, as he invested heavily in the special symbols, which are no more in J.

Gerard Langlet - In order to use J one will need to be able to choose from many J interpreters. Also J must be a stable language, not subject to change.

Bob Bernecky - Surely APLers will not drum out the J community. The popularity of APL and J will only increase if we all aim to publish articles in the big circulation magazines and journals.

R G Sellinger - Learning J one has to go back to first principles. It is not a dialect of APL, but it is derived from APL. The APL community is unique in being able to look at and accept new languages and ideas.

Joey Tuttle - In his experience one did not need to go back to first principles when learning J.

Jim Lucas - One does not have to go back to the beginning when learning J. The arguments about floating or grounded enclose took place 4 years ago. Ken Iverson has now given us a new name for the grounded system. A power struggle is quite unnecessary, and we should all stop the arguments and get down to the business of promoting APL and J.

reported by Anthony Camacho

Gene McDonnell said that he doesn't care whether J is or is not a dialect of APL and asked 'What are the consequences?'. Had I been organising the debate and known he was going to say this I would have not have included him on the panel. It was a bad way to begin, because there plainly are consequences. Debates are a poor substitute for an argument because debaters are at liberty to ignore each others' utterances. In this debate they did so to an astonishing degree. I would have preferred a medieval disputation where each sentence from one disputant is responded to by the other and the argument progresses or not according to the skill of the disputants.



Is J a Dialect of APL - the "better if nots"
Garth Foster, Phil Benkard, Richard Nabavi

Garth Foster said that J is spelled differently and has a different syntax and semantics so it is a new language, not a dialect.

David Steinbrook said that J feels like APL.

Phil Benkard had asked Howard Peelle, who had told him a dialect ought to be a subset. J is spelled out, not in single symbols. Phil thought it was a political decision.

Joey Tuttle said, whatever the answer was, it would not affect his life, so he was inclined to agree with Gene. He suggested a language is a dialect that has an army and a navy. He thought that what Ken is doing is really valuable and wants to see it publicised and discussed at conferences. After all Michael Jenkins spoke about Nial at an APL conference because APLers were interested.

Richard Nabavi admitted he doesn't understand J and felt it a pity that Jim Brown wasn't on the panel "Is he too busy writing J2?". He felt more variations in APL would diminish rather than magnify the chance of APL succeeding. APL is not increasing its proportion of computer users. In general it isn't the best solution that wins - look at MSDOS. He would like a single standardised APL which everyone could support and which would help to remove the doubts of potential customers. He snidely looked forward to being invited to J92.

Dave Weintraub said that he keeps encountering three objections to APL - it's write-only, it uses a different character set, it doesn't stick to one standard - and this dialect makes things worse. He also thought J would be capable of implementing any variety of APL.

Bob Bernecky wanted to get the ideas behind APL into the greater world of computing. He had found that with J he doesn't get the negative responses he used to get with APL. J has more regular semantics than APL, simpler syntax than APL, is more compilable than APL and omits the known errors in APL.

Garth Foster objected that simpler syntax wasn't necessarily easier. He recalled Bob getting lost during his presentation and having to ask Roger "Where am I?".

Donald McIntyre said that APL conferences without Iverson would be like Hamlet without the Prince of Denmark.

Jim Lucas rejected Richard Nabavi's arguments, saying the Catholic Church tried to standardise the rules of the universe for Galileo. He hoped APL conferences would include J. APLers (of all people) should be suspicious of any claim that J is unreadable or write-only. Even the symbol argument is suspect: Serbian and Croatian are very similar languages though written with different alphabets.

Garth Foster said Modula isn't called a dialect of Pascal even though Wirth developed both.

Ed Cherlin said that the argument was odd: there are innumerable dialects of APL with four or five in every conference. The Personal Array Transactor, hailed as the first APL implementation, is further from modern APL than J is. Surely the clincher is that J has the interesting ideas that APLers want to discuss.

Wednesday 7 August

Panel: 25 Year perspective

reported by Anthony Camacho

This panel was moderated by Roy Sykes. Vin Grannell paid for it to be recorded on video; I fear the recording will reveal many deficiencies in this report.

Adin Falkoff began. He put up a foil of IBM APL Systems and went through some 15 versions of APL, beginning with the Personal Array Transactor on the 1620 and ending with APL2.

John McPherson was the man who made IBM's first personal computer, the 5100, an APL machine. Not many people know that the modern PC started life as product no 5151. John ran the Systems Research Institute in New York and recalled Ken and Adin coming there to teach systems engineers. John Lawrence gave APL its first exposure in the IBM Systems Journal. Phil Abrahams, here at Stanford, described an APL machine in his 1969 thesis. Phil worked with Larry Breed on a ForTran punched card version that we would recognise as APL.

Another early version mentioned was one by Joe Harry, a part time programmer, but it never got finished.

In 1963 someone (Herb Hellerman?) argued he should write an APL for personal computing and was given the use of a 1620 with 20,000 digit memory and selectric I/O. He introduced scalar extension but only achieved a small subset with the restriction that each line could only execute one function. Lyle Heath wrote a batch version for the 7090. Iverson taught evenings and brought a student to help: it was Larry Breed. Herb Hellerman left IBM in 1969 to join SUNY: he is now at Amdahl.

Larry Breed introduced Roger Moore to computing on the IBM 650. George Forsyth of the Math Dept took people to a seminar at IBM's Systems Research Institute and remembers Roger 'muttering for days afterwards how wonderful it would be . . .'. At Stanford Mike Montalbano was enthusiastically teaching Iverson notation and introduced Larry to Ken. Gene Amdahl was at Stanford in 1964-5: he was IBM's lead architect on the IBM 360 and his article on 360 architecture was in the same Systems Journal that included the APL description of the 360.

Larry went to Yorktown and got to play with PAT. The notation was just moving from paper into the computer and having to write on a single line was a new problem. Function definition was introduced by an F overstruck with a tilde.

Larry recalled that IBM was scared of timesharing and they instructed the programmers to put in a minimum response time of five seconds on the grounds that less would frighten the users. Roger Moore was recruited as a consultant from I P Sharp. Larry mentioned three ideas the implementers resisted:

1. use of max and min symbols instead of AND and OR.
2. use of monadic less than and greater than for sine and cosine (Larry thought this was an idea of Ken's).
3. implementing vector constants delimited by spaces (originally they had to be laminated/catenated with the comma). [I think this was where Larry said the implementers admitted they were wrong.]

Larry said "Ken Iverson loves arguments: I think I got hired by arguing with him". He also remembered a soft spoken guy called Gene (who didn't get into the arguments) who surprised him by remarking that apl is the Greek root that means simple but not yet fully formed, and who joined him playing with the golden mean. During this time Larry felt he merged APL with the most delightful part of mathematics. He said, wistfully, that he began his career at the top.

Gene McDonnell was a salesman for the IBM time-sharing service. Roy Sykes reckons he has contributed more functions in less characters than anyone else in APL.

Gene said that when Dante was 35 in 1300, he wrote the divine comedy: when Gene was 35 in 1961 he began writing APL and hasn't stopped yet. He still has Ken's hectograph notes from the Systems Research Institute's course he attended. He found APL congenial and saw that it addressed fundamentals beyond the scope of the typical accountant's imagination. He remembers Ken teaching on 26 September 1962 when the first box of copies of A Programming Language was delivered to the class room; he reckons he got the first copy out of the box. Gene claims his contribution to APL\360 was the signoff command)OFF. Arthur Watson was exposed to BASIC at Hotchkiss (a private school at Lakeville Connecticut) and went back to Armonk saying "We want something like this." and by good fortune Iverson's notation was the obvious candidate; it WAS 'invented here'.

Gene recalls the quality of the implementation was really high because success depended on its reliability.

Roger Moore's real claim to fame was the six space prompt on the time-sharing system.

Roger said that when he joined (working on ForTran) the time taken to compile one statement was about the same as the mean time between failure. Timesharing was the obvious way to use up the expensive computer processor's spare capacity while it was printing core dumps. He looked at the printer driver with a view to timesharing it and was so appalled that he decided APL should be stand-alone, i.e. complete with its own operating system. One cylinder of the disk drive was 36K and so that was the original maximum workspace limit. The terminals (1050 and 2741) were half-duplex and the test for response was, after clearing the tabs, to press TAB, 2 RETURN - if the response was good the 2 would print before the carriage had returned to home. Roger said that Luther Woodram was responsible for the grade up and grade down overstrikes and Dick Lathwell for the design of the shared variables.

Before SigAPL, before STAPL there was Garth Foster. In 1966 Garth was hired by Syracuse to be a service engineer and was given the job of liaising with Yorktown Heights on APL. Garth was invited to Share by Karl Korn and volunteered to do a newsletter SHARE APL\360. Number 1 was April 1969. In 1968 he wrote a position paper recommending Syracuse to buy a 360 provided they could have APL on it. Otway Pardee placed the order with perfect timing so that it could not be reviewed or countermanded by the university top brass.

Roy Sykes introduced Ken Iverson with the words of Otway's successor at Syracuse: "There's little I can say that Iverson won't".

Ken said he clearly couldn't take time to refute all the errors in the stories we'd heard that morning. He recommended we consult the sources: A Source Book in APL, Quote-quad, the Early Years, Gene McDonnell's article on Vector notation, John Clark's film and George Mendonsa's museum.

But he couldn't allow Larry Breed to get away with the ludicrous assertion that the implementation team ever admitted error.

Ken applauded the far-sightedness of Howard Aiken who, realising that computing had to move out of universities into the real world, began to run an Automatic Data Processing course to train the people who would make the move. Ken was one of four graduate students who helped to run the course. He found that the first thing he needed was a matrix algebra language. After three years teaching and inventing notation as he went, he tried to reduce the bulk of the notation by generalising it. Herb Hellerman encouraged him to be ruthless culling dispensable material. Another culling followed during implementation. "But success brings paralysis: you get to the point where you can't get anything changed. Even switching companies doesn't help much." Ken said time does cull things ruthlessly and people earning a living at APL are particularly reluctant to cull. As Peter de Vries wrote "Time wounds all heels."



Ken Iverson

We were asked to remember three dead pioneers: Reyner Cogan, Mike Montalbano and Bill Bergquist. Ken felt it was users such as Stephen Jaffe and Donald McIntyre who would ensure the survival of the notation by using it to conquer new fields. He recommended us to be motivated by practical problems, implement them in APL and only then consider whether more change was needed.

Panel: APL as an Entrepreneurial Tool

reported by Jonathan Barman

The session started with each panel member telling us about their experiences in using APL for their business.

John Estep - E/Step Software sells a decision support package for inventory management, and has been in existence for 8 years. APL shortened the development time and cost, so that the product can be sold for \$25,000 less than the competition and still be a very full and competitive implementation.

Generalised code means that it has been possible to restrict the provision of special features for individual customers. Very few customers ask what language has been used, which is mainly due to the fact that PC users do not expect to get source code, and are therefore not interested in the source language.

Axel Holzmüller - Ditttrich & Partners is a software house with 50 employees of whom 20 are engaged in APL. Their business covers a wide range, including consultancy, APL support, software development, APL training, local area networks and PC distribution. Most software development is in APL, but they also use C, Clipper and fourth generation languages.

Donald Pittenger - The Demographics Laboratory provides population surveys for market researchers, and is a one man band. APL enables the numbers to be cranked out quickly, so increasing the number of projects Donald can deal with, and therefore the income he can generate. Almost all the APL is used for internal purposes. There is a problem in selling APL externally as run time licences from STSC are expensive.

Ameziane Mourad - Batisoft consists of about 70 people and sells a CAD package for architects. Most of the code is in APL, but C and Assembler are used where speed is critical. They are currently developing a GUI so that the product can be marketed in the USA, where customers are more discerning.

Nigel Harris - Saladin sells a decision support product for the oil industry and has about 40 employees. The package is written entirely in APL, and has dial-up links to public sources of oil industry data. Reuter File is used for distribution of data to 120 users in 20 countries. Venture capital was obtained to fund the start of the company. Five oil companies are design partners.

Georges Brigham - The APL Group sells a product to manage Electronic Data Interchange (EDI) of routine business documents such as invoices and purchase orders. Venture capital was raised in 1986 in order to speed up the launch of the company, and there are now 15 employees. The product contains 25,000 lines of APL code, and is continuously updated by sending out new versions.

The following questions were put from the floor:

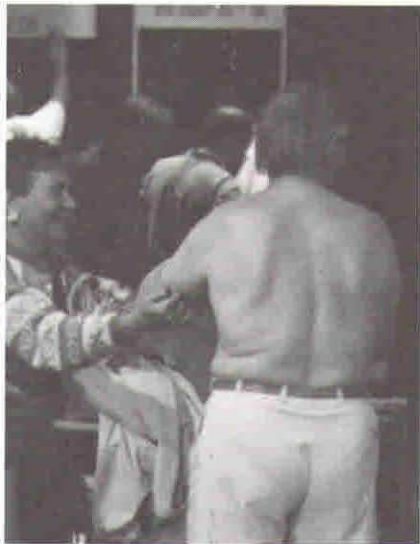
Did the panel experience any problems with customers refusing to deal with a one man band? Donald Pittenger said that he had no problems. Georges Brigham hit that problem in the beginning but the company concerned bought the source code, and there have been no problems since. If John Estep gets a problem he quotes the big companies he deals with.

How does one get venture capital? Nigel Harris said that you have to put together a business case and sell it to prospective investors. In their experience it takes about 6 months' full-time work for 2 people.

Is there a problem maintaining up to date code? Georges Brigham said that it was a tough problem which they solve by sending out code updates. Nigel Harris said they developed their own version control procedure.

What about run time licences? Donald Pittenger finds this a problem as APL*PLUS PC will only sell run time licences at \$100 each, and there is no VGA for APL2/PC where run times are free. (James Wheeler of STSC said there would be an announcement about run time pricing at their product forum, at which it transpired that a one-time payment could be made.)

Does low cost make a product look cheap? Donald Pittenger said that budgets were often important and that companies looked for cheaper solutions. Adam Osborne said that the linking of low cost to low quality was a myth.



A quick change - Erkki Juvanen preparing to advertise APL92

Thursday 8 August

A0 System - APL in Construction Planning, by Alexei Miroshnikov *reported by Sylvia Camacho*

The speaker's presentation was a summary of his paper. The paper shows the APL application to be very ambitious although the project has been under way for little more than a year. The impression given is that this system is more closely related to western expectations of commercial programming systems than those described by other soviet delegates. The overall approach to the work sounds very professional and your reviewer will not be alone, I am sure, in wishing it well. Any improvement in the facilities available to industry in the USSR is welcome and the construction industry must be a priority.

During the questions, attempts were made to discover how construction planning was done in the days of the centralised economy. Maybe most of the audience could not get over the language barrier but the answer appeared to be that there had been no significant computer involvement before perestroika.

reported by Jonathan Barman

A0 is a system which calculates construction costs based on detailed requirements for materials and labour. The system is written in APL*PLUS/PC, and took 3 developers 3 years to complete. The prototyping of the system was carried out with the customers.

APL made the development quicker, and the package came second in a competition up against packages written in other languages. However, there were disadvantages in using APL. Discipline in the programming of APL seems to be difficult, the facilities for interfacing with other languages is poor, and programmers found it difficult to debug packaged code.

Automatic Synthesis of the Inverses of APL Functions, **by Alvin Surkan**

reported by Ray Cannon

This was the most fascinating paper on the use of APL that I came across in the conference. In this paper Surkan describes how neural networks are trained to solve complex problems, these solutions are then analysed and the results written in APL. By writing the APL in a particular fashion (following some "special standards" such as only allowing "monadic" functions) the resulting

code can be "inverted" by reversing the "polarity" of functions (for example, dividing by a factor rather than multiplying by the factor) and reversing the order of execution. The process is some what similar to how DNA produces an inverted copy of itself (messenger RNA). (See Douglas R. Hofstadter's book "Godel, Escher, Bach: an Eternal Golden Braid" on self replication.) Try doing that in C!

Invited Speaker: Ted Nelson. Sealing Our Data

reported by John Searle

Ted Nelson is Distinguished Fellow at Autodesk of Sausalito, California. He is the inventor of hypertext, and is the visionary behind the Xanadu hypermedia project. The project has the simple aim of completely revolutionising the way in which data is stored and organised in computer systems.

In this new scheme, files will be replaced by data "puddings", and Xanadu will provide the structure for interconnecting items of data. According to Nelson, cohesive ideas emerge from good organisation of data - this is the manner in which the human brain works - and this is how we should design intelligent systems. Nelson has a "20/20 Vision", believing that by the year 2020 computer systems will operate in this way. Public systems will use a Universal Electronic Publishing system, in which the author of a contribution will pay to lodge it, but will obtain royalties every time it is used. Intelligent use of "pointers" will minimise the need to move and duplicate items of data.

Nelson's thesis is very much at the global thinking level, and he is not too concerned about the impracticalities and nitty-gritty details of his scheme at this stage. His talk was full of glib remarks and quotable quotes. "The moving storm of computer progress leaves many puddles, in which teem interesting life forms."

Nelson also believes that we are gathering and storing data at a faster rate than we can process and organise it. NASA has so much information from space and orbital probes, that it has appointed a Data Archaeologist to manage it.

Nelson urged us to recognise when we are faced with Paradigm Confrontation. We only understand the boundaries of our belief systems when they are challenged by new thoughts, which cause our hackles to rise. His challenge to us is to accept the limitations of our data architecture, and to be prepared to change it.

The APL91 Museum

reported by Dave Ziemann

APL91 was the first International APL Conference, as far as I am aware, to include an APL museum. A room was set aside to display artefacts from the long history of our favourite language, now a quarter of a century old. In one corner, an IBM 5110 was eagerly (yet slowly) playing Tic-Tac-Toe, announcing its progress via a speech synthesiser connected to it. The 5110 was one of the first APL personal computers, and is a direct ancestor of the rather more famous IBM PC line.



George Mendonsa's Museum

I was strangely attracted by the perverse beauty of an APL punch card - a regular Hollerith card where every punchable position was overprinted by an APL symbol. This is how the first APL interpreter, developed here at Stanford by Larry Breed and Phil Abrams on an IBM 7090, was programmed. In the words of the "APL Blossom Time" song, vinyl copies of which were also on display:

Well a FORTRAN batch interpreter's a mighty awesome thing
But while it hums a pretty tune it doesn't really sing.
The thing that we all had to have to make our lives sublime
Was an interactive program that would let us share the time.

Oh... They got Roger Moore and they got Dick Lathwell,
And they got Gene McDonnell with his caret and his sticks,
And you should have heard the uproar in the Hudson river valley
When they saved the first CLEANSPACE in 1966.

With brilliant foresight, Larry Breed saved that very early APL printout. It is a slightly faded piece of lineprinter paper which includes a dialogue between Larry and the machine operator, followed by the successful saving of the first ever clear workspace, called CLEANSPACE. The system save date was 27 November 1966.

The museum contained a variety of APL books and delegate gifts from previous conferences. Ken Iverson's original book, "A Programming Language" was there. A copy of the first edition of "APL - An Interactive Approach" by Gilman and Rose was represented. That famous edition of the 1964 IBM Systems Journal, which contains a paper that uses APL to describe the architecture of a new line of machines - the IBM 360 series - was also there. The "Iverson Notation" inside that journal looked very different from the APL we know today.

A number of attempts to keep track of the genealogy of APL have been made, and SIGAPL's effort is included in the museum - a database of who learned APL from whom.

Original 2741 golf balls and the paper tape that Al Rose used to feed into his machine to demonstrate APL when the mainframe went down were popular items. One of the best ideas was a little book where conference delegates could enter details of any historical APL objects which they possessed. There were two boxes to tick, one if you wanted to keep the item yourself, and another if you were willing to donate it to the newly formed APL Museum.

If any Vector readers have memorabilia which they feel the APL Museum should know about, please write to the Editor who will forward the information. If you have something you would like to donate to the museum, even better. One rich area of exploration could be old APL language manuals, particularly those from unusual APL systems.

I'll finish with the latest verse of the "APL Blossom Time" song, created during the conference, and reflecting events of recent years in the APL world:

Now Bob Brown said we need a faster APL,
So Arthur Whitney wrote one, but his bosses would't sell.
And some planners said interpreters no longer were in style,
So they sat down at their hardware, and they started to compile.
Well ... we got Roy Sykes and we got Clark Wiedman,

and they ran it just as fast as all the hardware would allow.
We don't have compilers on our personal computers,
But our PCs all run faster than the mainframes anyhow!!

Quotable Quotes

"That's either a typo or a thinko, I'm not sure which." *Bob Bernecky*

"The Dashing Commuter. (In 1966 or '67 Al Rose suggested the high minus suffix as the symbol for 'commute', and suggested this mnemonic.)" *E E McDonnell*

"We have one function called *X* ... that's probably a bad name to use."
John Jensen

"I don't get the negative responses [with *J*] that I used to get with APL."
Bob Bernecky

"IBM specified the minimum response to a timesharing response was to be 5 seconds - faster would frighten the user!" *Larry Breed*

"Ken Iverson loves arguments; I think I got hired by arguing with him."
Larry Breed

"I am flattered that at my age you think I could have the imagination to come up with a completely new language, unrelated to APL." *Ken Iverson* commenting on the *J* dialect issue.

"Always ask the customers what they want, and then don't pay any attention to it ... that's how we designed APL." *Adin Falkoff*

"I was exposed to APL but didn't catch it." *Andy Kinslow* quoted by *Gene McDonnell*

"Oh my goodness - an Iverson Ball!" *Gene McDonnell* on seeing the very first APL Selectric typeball

"Consult the written materials ... they're suspect too, but at least they stand still while you compare them." *Ken Iverson*

"The sad part - success brings paralysis. You soon get to the point where you can't get anything interesting changed. Even changing companies doesn't help much!" *Ken Iverson*

"Time wounds all heels." *Peter de Vries* quoted by *Ken Iverson*

"I am not of your church, ... but I greatly respect it, and I am here in the interest of ecumenism." *Ted Nelson*

"This is a palette of fabulous power, which I wish every kid had access to." *Ted Nelson*

"For 27 years I was considered a worthless maniac ... now I am considered a worthy maniac." *Ted Nelson*

"WYSIWYG - what you see on the **screen** is what you get on the **paper**! You are going to use that magnificent machine as a *paper simulator*." *Ted Nelson*

"In the traditional system, the user is just a subroutine of the application." *Alexander Skomorokhov*

"A culling needs to take place, and time will do it." *Ken Iverson*

"There is nothing complex about complex numbers, unless you don't understand them." *Donald McIntyre*

"My arithmetic is very poor, but then I'm a geologist ... 12 and 24 are near enough the same thing if you are talking about the age of the earth." *Donald McIntyre*

"If you don't like the news, go out and make some of your own." *Tibor Horvath*



The Final Picnic

Selected Survival Strategies for APL Puddle-dwellers

by Adrian Smith

One enormous advantage of spending the six weeks following APL91 globetrotting is that everyone else has already written up their notes when you get back! This (a) saves you a lot of work and (b) gives you the chance to reflect a little on how things went. It bothers me a little that almost without exception my colleagues have reported the entire event in the kind of glowing terms that the Nestlé News uses to record company progress towards ultimate supremacy in the chocolate world. Yes, of course as a conference it was a success: it was professionally run (I wonder at what expense if the breakeven was as high as 400?); the Stanford campus was all you could expect of it; the Exploratorium was brilliant; the picnic was great; but

- did you learn any new APL?
- where were all the papers proposing interesting and controversial language extensions?
- apart from J, were there any new ideas at all?
- was it a Conference or a Reunion?

To repeat that rather apt Ted Nelson quote:

"The moving storm of computer progress leaves many interesting puddles, in which teem interesting life."

As one of the 400 or so puddle-dwellers gathered at Stanford, let me offer some final thoughts on desert life, and the survival strategies it engenders:

1. Everyone swims round and round in ever-decreasing circles, talking in loud voices about nothing in particular. Any attempt to turn the conversation towards dangerous topics ("I wonder if it will rain today?") is quickly lost in the general hubbub. This is probably the least successful strategy.
2. The Australian frog approach is to gulp as much water as you can (becoming almost spherical in the process), burrow into the mud at the bottom, and wait. This works quite well as long as it eventually rains again, but it does rather commit you to a long slow death otherwise.

3. You could try walking on your fins, gulping air at a frantic rate, and setting off in search of another puddle. Of course you will very likely fail, but there is just a chance that you will evolve from a crude amphibian into a primitive reptile into the most successful species the world has yet known (well, the dinosaurs did run the show for 150 million years, so they must have got something right).

The J debate was a classic example of the first strategy, the second is well illustrated by the Sharpies (who are already well buried in the mud) and the APL2 developers (who are settling gently to the bottom). Ken Iverson and Don McIntyre were among the few who were brave enough to get up on their forefins and hobble off across the desert in pursuit of Ted Nelson's retreating storm. I think we should all wish them luck.



Stanford Library, with the Hoover Tower behind it

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers.

Power Reduction

by Graham Parkhouse

How many large numbers there are that my computer cannot represent.

```
* / 2 2 2 2 2
DOMAIN ERROR
* / 2 2 2 2 2
^
```

The number I could not calculate is the next in this sequence:

```
* \ 2 2 2 2
2 4 16 65536
```

Its number of digits is:

```
[65536 * 10 ^ 2
19729
```

What is the number of digits in $/6p2$?*

```
'1E', \ [19729 * 10 ^ 2
1E5939
```

The number of digits seems to escalate as quickly as the numbers themselves. Working with 10 rather than 2 gives us a clue to behaviour:

```
* \ 10 10
10 1E10
```

An extension to the commonly used exponent notation would give the result of $/4p10$ as 10 1E10 1E1E10 1E1E1E10. The number of noughts in these 4 numbers is 1 10 1E10 1E1E10.*

The numbers we encounter in the world are small by power reduction standards: even the number of atoms in the universe is currently estimated as 1E80 or $/10$ 80 or $*/47.7$ 47.7 or $*/3p3.74$ or $*/4p2.28$. Where does this progression lead?*

```
□ PP + 2
* / 3p3.74
3.5E79
* / 4p2.28
9.4E78
* / 10p1.543
2.9E90
* / 50p1.44873
4.5E91
* / 100p1.4456983
4.6E80
```

There is a hint of convergence in this sequence: as if there is some number X beginning 1.44... for which $/VLpX$ would give 1E80 for all values of VL that were very large. What happens if I take a slightly smaller number than 1.44... and see what effect VL has.*

```
□PP←5
  *√10p1.3
1.3 1.4065 1.4463 1.4615 1.4673 1.4696 1.4704 1.4708 1.4709 1.471
  *√100p1.3
1.471
```

Power reduction on a large number of 1.3s appears to converge.

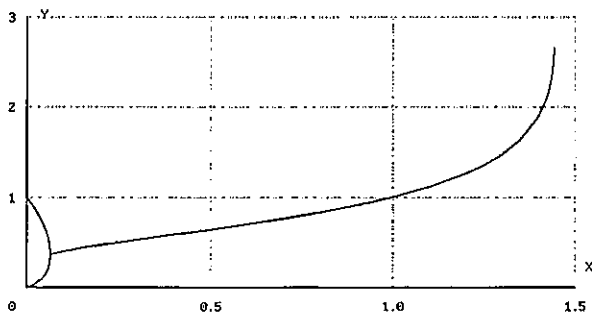
```
□PP←12
  *√100p1.3
1.47098896009
  *√1000p1.3
1.47098896009
```

I estimate that a power reduction on an infinite string of one number will converge provided the number is less than 1.4446678. If the number is less than 0.06599 there is another surprising result:

```
□PP←4
  *√10p0.01
0.01 0.955 0.0123 0.9449 0.01289 0.9424 0.01304 0.9417 0.01308 0.9415
```

There is convergence to two different values, a smaller value for odd-lengthed strings.

I can state what I have demonstrated like this: provided X is within the range 0.06599 and 1.44467 and VL is large enough, $/VLpX$ defines a number, Y , say. Plotting Y versus X produces the curve below. It includes the bifurcated piece where X varies between 0 and 0.06599.*

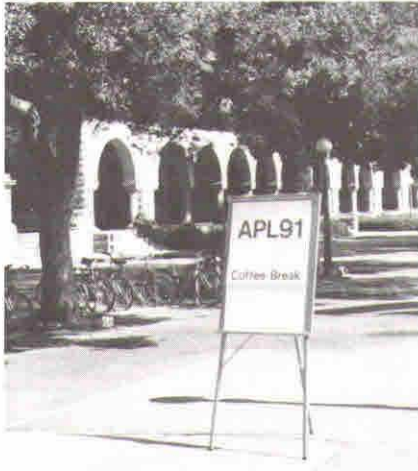


Power reduction also converges on long repetitions of groups of numbers, such as 0.1 2 0.1 3. It is an algorithm for separating sequences of numbers into 3 categories: non-convergent, convergent to just one number, and convergent to two numbers.

PS. These results are new to me. In APL they were easily explored and explained. Had I been thinking in conventional mathematical notation I would have been writing expressions like:

$$1.3^{1.3^{1.3^{1.3^{\dots}}}} = 1.46958$$

instead of `1.46958=**/6p1.3`. Another confirmation of the power of APL as a tool of thought!



(above) A quiet Coffee Break at APL91



Stanley Jordan at the opening concert playing APL (right)

The Legacy of the Typewriter

by Richard Nabavi

The mechanical typewriter has had a significant impact on computing. The QWERTY keyboard, which is the most frequently-used input device for computers of all kinds, is a throwback to the ergonomics of early touch-typing. More significant, however, has been the impact of the typewriter on font design.

By its nature, the typewriter is a fixed-font device. Every character takes up the same width on the page, and every line height is multiple of a fixed base. For the first three or four decades of the history of digital computers, the same restrictions applied to almost all output devices connected to those computers; early printers were in fact modified electric typewriters, and the same restrictions were carried over not only to line printers, but even to video displays.

Nowadays, of course, we have more sophisticated output devices which do not suffer from these restrictions. Although daisy-wheel printers in the Seventies had some proportional font capabilities, it is only recently that it has become economical to print a full range of fonts on a page. Modern desktop laser printers can handle a wide variety of fonts and effects, and these include proper proportional fonts in a wide variety of sizes and types.

At this moment, it remains the case that most computer screens still display fixed-size characters. This is true of most PC-based systems, as well as most terminals connected to mainframes and minis. But this is rapidly changing - the success of the Macintosh, and the increasing acceptance of graphically-based interfaces both on PCs (e.g. Windows and Presentation Manager) and on multi-user systems under Unix, has begun to make a significant impact on the market. With this impact comes the realisation that things have to be done differently in a non-fixed-font world.

The ubiquity of the typewriter for most of the twentieth century, and the years of fixed-font output from computers, have obscured the fact that fixed-size fonts are very unnatural. Until the typewriter came along, virtually all written material would use variable fonts, be they hand-written or printed. Newspapers, books, magazines are always printed using proportional fonts, except for a few amateur or semi-amateur publications where the typewriter is used to save production costs.

How Word Processing has Coped

The move to use proportional fonts, and fonts of variable size, has of course first impacted word-processing software. In fact, the impact has been slower than might have been expected; it is often amusing to see how secretaries react to powerful word-processing systems which can output to laser printers. All too often, the first thing they do is select Courier 10, and never use more readable fonts. At college, secretaries are still taught to count spaces and lines to lay out pages. It is symptomatic that the Hewlett Packard LaserJet - for some years the leading desktop laser printer - was originally sold only with fixed-size Courier fonts, despite the intrinsic capabilities of the technology.

All the same, the change has now occurred. Take, for example, WordPerfect, one of the most widely used PC word processing packages. Until Version 4.2, the software was still based on lines and columns, but so many facilities had been put into the software to vary font sizes and to handle proportional fonts, that it became clear that the approach would have to be changed. Specifying margins in columns, for example, is a nonsense even with a single size of proportional font, but it becomes quite absurd when there are four different font sizes on a page. As a result, WordPerfect Version 5 radically changed the approach used; margins were expressed in inches or centimetres, as were tab settings and all of the other dimensions used in the software. Other word-processing software packages have had to alter in similar ways - nothing else makes any sense.

But not just word-processing is affected. Modern spreadsheet packages now include powerful font-handling capabilities. There is an increasing demand for database software which can directly produce high-quality reports which take advantage of laser-printer features. Increasingly, if a software product outputs text, it needs to handle fonts. The legacy of the typewriter is receding - at last.

And what about APL?

APL, it is often said, is a notation first, and a programming language second. One of its strengths is that it is therefore not restricted by the limitations of particular hardware architectures. It does not, for example, suffer from the Pascal 'strings are limited to 255 characters' syndrome.

Whether APL is regarded as a notation or as a programming language, one thing is clear - its design should not be influenced by the mechanical limitations of the typewriter carriage mechanism. Yet that is precisely what has happened! The original implementations of APL were expecting to output to a 2741 golfball device (i.e. a modified electric typewriter). We have become so used to fixed-size APL fonts that we have failed to notice how deeply the assumption that all

characters have the same width has permeated the design of the language. There are four areas which are affected:

- Default output rules
- Monadic format
- Dyadic format, and other formatting features such as `□FMT` and format-by-example
- Data structures

Default Output Rules

Here is a typical example of how APL2 formats an array.

```
(3 2p.2×16) 'SOME TEXT' (3 2p110)
0 0.2 SOME TEXT 0 1
0.4 0.6          2 3
0.8 1           4 5
```

User input, of course, is indented six spaces. The nested array includes a leading space, and columns are lined up according to well-defined rules.

Let us re-display the same text in a typical “real-world” font, Times Roman (the font most commonly available on most laser printers):

```
(3 2p.2×16) 'SOME TEXT' (3 2p110)
0 0.2 SOME TEXT 0 1
0.4 0.6          2 3
0.8 1           4 5
```

Even with this simple example, it is clear that we cannot just apply APL2's default display rules and expect a comprehensible result. And yet, there does not seem any intrinsic reason why APL should need fixed-width fonts.

Let us look a little more closely at what the APL2 display rules are intended to achieve. Clearly, the aim is to give some order to the placement of sub-arrays on the page, so that columns line up. This is achieved by inserting the correct number of spaces into the text between the elements on each line. On a non-fixed-width font, this results in the effect shown above, which will be familiar to anyone who has used a word-processing package to edit text which includes tables of data which use spaces to achieve the lining up. The solution, in the

word-processing case, is to use tabs (expressed in centimetres or inches) to line up columns. (The problem is simplified for those fonts which have a fixed width for the digits 0 to 9.)

Suppose, then, that an APL2-like interpreter was designed as follows. Instead of counting the characters in each part of an array, and padding out the sub-arrays with spaces, it would measure the space taken up by each part of an array (in centimetres or some other absolute unit), and use this to work out the spacings needed as an absolute distance. On the Macintosh, for example, this would involve calling the stringwidth system call, which returns the width in graphics units of a given string in the current font.

A multiple of a fixed distance (I call it the offset distance) would need to be added between arrays to correspond to APL2's addition of extra spaces between sub-arrays according to the rank of adjacent items and other factors. Such an APL interpreter would be upwards-compatible with the existing APL2 rules. For, if the current font were a fixed-width APL font, the distances calculated would all be multiples of the fixed width, and provided the offset distance was equal to the fixed character width, the output would look identical to current APL2 output.

At least, the above is partially true. But consider printing width ($\square PW$). Just as Word Perfect needed to move away from expressing margins as multiples of a fixed-character width, so $\square PW$ in its current form only has a meaning as long as there is a single fixed width font to be considered. In the new world of non-typewriter-like interfaces, display widths should to be expressed in centimetres or inches.

Monadic Format

So far, the above discussion has not touched on the APL language definition in any significant way. Changing the default output rules to handle non-fixed fonts is a cosmetic change (albeit an important one). But what about monadic format?

The IBM APL2 *Programming Reference Manual* says that monadic format 'creates a simple character array whose appearance is the same as the display of [the right argument] (if $\square PW$ is set sufficiently wide).' It does this by following the same output rules as default display, creating a two-dimensional character array and inserting spaces as required. But this will no longer work if we make the change described above; a simple character array cannot express the lining up of columns of the source (nested) array, once we abandon fixed-width fonts.

What is needed is a monadic format which retains the structure of the original data, converting each numeric element to an enclosed text string of the format of the element. This is equivalent to format-each (▼") for arrays of depth 2. Such a formatter would have the pleasing property that the shape of an array would not be altered by the format. It is interesting to note that such a formatter could usefully be the inverse (on arrays containing only numbers) of a modified execute primitive which worked on arrays of any shape or depth.

Dyadic Format, $\square FMT$ etc.

An even more radical re-think is necessary for the built-in formatting routines which allow a detailed specification of how text should be formatted. These include dyadic format, APL68000's α formatter, $\square FMT$, and APL2's format-by-example. Diverse as these formatters are, they all return a simple character matrix (or vector) where spaces are used to line up the columns. As we have seen, this just won't do for the Nineties.

Since we already have so many different formatters, it is tempting to address the problem of designing something more sophisticated by ignoring the whole lot and starting again. As well as all the options which the existing formatters give, we need to be able to specify, for each 'chunk' of text:

- Font family (Times, Helvetica etc.)
- Font size (10 point, 12 point etc.)
- Font style (plain, bold, italic etc.)
- Justification
- Spacing
- Text orientation (e.g. left-to-right, vertical or following a curve)
- Position on the page (in cms or inches)
- Action to take if the text won't fit (e.g. use a smaller font)

Such a formatter needs to handle changes of font within a justified paragraph.

There are tricky problems of portability and generality here, but there is another even trickier problem - what data type would such a formatter return? Or would it simply output the text, thereby losing the advantage of being able to use APL to manipulate and store the formatted result?

Data Structures

We can now see that a simple statement of some of the formatting needs for outputting text leads inexorably to consideration of APL's data structures. Because, if our formatter is to return a formatted array which we can manipulate using APL, then maybe we don't need a formatter at all - maybe we need a way of associating format information with the existing text and numeric data types. You could consider this as a kind of 'object-oriented' formatting - each array would carry with it information which would ensure that its default-display appearance was in the correct set of fonts, styles and so on. Obviously, a set of defaults would apply if the appearance had not been defined, and these defaults might well be designed to be upwards-compatible with APL2's display rules.

Such an approach would certainly be hierarchical in that a sub-array would inherit its format information from its parent. For example, if the text:

The cat *sat* on the mat

were represented as a length 6 vector of vectors, then the font for the whole array would be defined once, with only the third element having the attribute 'italic'. Thus if it were later required to display this array in a different font, this could be done by changing the font for the whole array, but the 'italic' attribute of the third word would be preserved, and would automatically select the italic version of the new font.

In a future article, I shall outline my suggestions for implementing this approach to formatting, but perhaps readers of Vector might like to offer their suggestions?

Suggested Standards for a Component File System at a Commercial APL Site.

by Ray Cannon

Need for File Standards

At any commercial APL site using a multi-user component file system (such as the ones offered by STSC, I.P.SHARP or Dyalog) there is a need for a set of minimum standards and guide-lines for files. It can be argued that if all the files adhere to the standards, user and programmer productivity may be increased along with other benefits. This document will attempt to set out such standards and guide-lines, and the reasoning behind them.

Areas Covered by the Standards

Naming Convention. The standards relating to APL file names are constrained by the File System Software and existing applications. However, some guide-lines can be given, along with do's and don't's.

Naming conventions for new applications systems will be suggested, as they often simplify the users' task of choosing the right file.

Component use. The most important part of this set of standards will relate to the use of standard components across all systems. This gives all files a similar "feel".

Use of standard components will allow the development of utility software (and "meta-systems") relating to APL files, which in turn should have benefits in the areas of maintenance, housekeeping and personal interchange (that is end users changing jobs in the company).

Access security. By standardising the access security of APL files (which itself can be implemented with the use of utility software) benefits relating to maintenance, housekeeping, and personal interchanges can again be achieved.

Naming Convention

Purpose of a file name. The purpose of an APL file name is to uniquely identify that file within the APL file system. The file name however may also be used to convey additional information to the user of the system (such as the year the file relates to).

Conflicts between making the file name unique, the limited format of the file name imposed by the system, and making the name as meaningful as possible, is mitigated by the use of a naming convention. Typically, the information a file name can be expected to convey includes:

- **Owner.** This information is often (but not always) contained in an account or library number, being the first part of the file name.
- **Application system.** The system the file belongs to is often encoded into the file name via a standard prefix or suffix.
- **Type.** The type of data (e.g. documentation, tables, temporary storage, variable data, back-up or archive data etc.).
- **Date.** The date the data in the file relates to in systems that run in cycles.
- **Version.** The version of the run producing the file, if run more than once.
- **Release.** The release version of the software if the files are not compatible across all versions.
- **Status.** The status of the data (e.g. Test or Real, Checked or Unchecked etc.)

By adopting the use of standard components, then some of the above information may better be conveyed within the file, via standard utilities.

Do's and Don't's. (Some of these may apply in some environments but not others, e.g. under CMS but not under UNIX.) File names should contain only (uppercase?) alphanumeric characters. Use of "delta" or any underlined character is discouraged. These non-ASCII characters may cause problems with some hardware (e.g. non-APL keyboards/printers). Remember that file names may well have to be manipulated by the end user outside the APL environment.

System prefix. If file directories are not available, do use a 2 or 3 character prefix to identify the system to which it belongs.

Date. If the date is needed, do use a simple format that can easily be read e.g. YYMMDD.

Type. To convey the type of data in a file the following suffixes (extensions) are suggested: DOC for formatted documents; TEXT or TXT for ASCII text files; TEMP or TMP for temporary files; TEST or TST for test data; DATA or DAT for the main production data; CODE or BIN or HEX for machine code and other "unreadable" files.

Note. After a run has been completed, no "temporary" files should be significant, and users should be free to delete any such files found "lying about".

A copy of the name of the file should **NOT** normally be held within the body of the file, unless all utilities to "copy" and "rename" files know of this convention and can take appropriate action.

Components

With the widespread use of standard (reserved) components within an installation, it becomes possible to write "system utility software" to manipulate files. This should ease the maintenance and housekeeping workload. Also it means that standard tasks needed in most systems (e.g. copying, access control, archiving etc.) need not be built afresh for every system.

Reserved components. Most importantly, all files should be numbered from 1 upwards. Components must **NEVER** be dropped from the front end. Reserved components should occupy a block at the front of the file and a suggested set up is defined below.

Component 1. File type identifier. This component should contain either a simple text vector, or a nested vector with a simple text vector as its first element. This text vector should identify the file application type. The meaning of the remaining elements (if any) should be defined in component 2. Suggested uses however might include sub-identifiers, version number, index offset, etc.

Component 2. Text description of the file. The second component should contain a description of the file in an easily readable format (e.g. a text matrix). This component should define exactly what is contained in the reserved and other components. The index method (if any) should be explained.

Component 3. Top level index component. The exact format of this component will be defined to the software by the "file type" in component 1, and to people in component 2. The main point of this reserved component is to prevent "partial updates" occurring by providing a single component to be updated which "fixes" any changes made to the file.

Components 4 to x (where x is defined by information in component 1). Other system level information, such as free component lists, secondary indexes, audit trail, etc., and SPARE COMPONENTS. Every system's file layout should have some "slack" or spare capacity in the form of unused spare components. A few empty components in every file take up very little space, but they can be extremely useful when the system undergoes major modifications.

Indices. With all but the simplest data files, during updates file data integrity may be lost due to a system failure causing a partial update. One method of preventing partial updates is to never replace data in-situ during an update, but

to write the changed components out to new components (multiple writes) and then REPLACE the index pointing to the old data by the updated index pointing to the new data, in a single disk write. If the system fails before the index is replaced, the file holds an intact copy of the data as it was BEFORE the update started. If the system fails after the index has been replaced, the file is again intact but also updated.

When the file contains more than one sort of data (e.g. control information as well as program data) it may be useful to have a separate index for each data type, with a single high level "master-index".

Index offset. The "index offset" allows an extra level of flexibility in a file design. The idea is that every index on the file holds component pointers rather than absolute component numbers. The "index offset" is used to convert between the two.

With this extra level, files can easily be expanded to allow for extra reserved components, merely by creating a copy of the files with the extra component inserted after the existing reserved components before the data pointed to by the index(indices) and changing the index offset value.

Access Security

Note. The *DATA PROTECTION ACT* may have implications in this area. These suggested standards DO NOT cover this at all. However, these standards may provide pointers to problems raised by the Act. In-house file standards manuals should go into this area at a more appropriate level.

In some environments such as UNIX, access is controlled by the operating system, rather than the APL file system. Most of what follows may be not applicable in such environments.

As far as security is concerned, there are two types of file, **Private** and **Shared**. In most systems the default security provided by the system for private files is sufficient.

For APL file systems that employ "access matrices" the type of problems that one encounters with shared files include:

- The use of "Generic user IDs" (non-specific logon accounts used by a group of users);
- The difficulty of maintaining complex access matrices with the addition of new users or movement of users within the company;

- Utility software unable to handle password protected files.

Types of access - "WHAT". It is suggested that there should be 5 pre-defined levels of "shared" file access controlled by passwords known to the system. These levels are:

NONE No access allowed to the file.

READID Allowed to read the 1st component (plus owner, date, time and file size information) only.

READ Allows reading of the data, but not changing the data on the file.

WRITE Allows reading and writing the data. May not change the access matrix.

STEWARD . . . Complete unrestricted access.

Groups of users - "WHO". It is often desirable to set the access level for a group of users. This group may well be dynamic during the life of the file. This may be most easily achieved by applications "owning" their own files. In so doing, the application decides who can do what to one of its files via passwords in the access matrix known only to the system. Such systems must provide access to all the housekeeping utilities needed (e.g. archive, copy, rename, back-up, delete, etc.).

The application can then maintain (in a relational data-base?) lists of file owned, along with users' access authority to different classes of files.

File utilities - "HOW". A single set of file utilities should be developed, through which ALL file accesses are made to shared files within application. These utilities would be common across all applications within a site. Once written, the application programmer's task would be much simplified, and if the application platform changed (e.g. from CMS mainframe to DOS PC) only these utilities would need to be rewritten for the new platform.

Other Notes and Guidelines

Always "share" tie a file if possible. Tied files reduce system restartability but may increase efficiency. Don't "HOLD" files for longer than is strictly necessary.

Within a system use "system-wide" cover functions to access the files via lower level utility functions.

System passwords. The availability of a system password, which the system CALCULATES from the file name, via a secret algorithm, can be extremely useful, even if this password only allows "READID" access.

Summary

This section will attempt to highlight the pros and cons of implementing these standards. Please note that the benefits and disadvantages are not nearly as "black and white" as shown here.

Benefits:

- Use of standards reduces training times of new users to systems, when those users are already familiar with other systems using the same standards and utilities.
- Reduced development, testing and documentation time for new systems, since standard utilities can be used in multiple systems.
- Easier housekeeping and other maintenance. (Standard utilities can read files from many applications.) Rather than changing possibly hundreds of file access matrices when a member of staff changes job, only a single data-base entry may need to be changed.
- Better integration of systems. Use of "Umbrella systems".
- Tighter control of security. Compliance with the "Data Protection Act" may be easier to achieve and prove.

Disadvantages:

Strict adherence to the standards (working to rule) may be counter-productive. Standards may stifle the "best" solution and create inefficiencies.

Imposing new standards on old systems costs money, (rewriting, retraining etc.) which may never be recovered in the life of the system.

"Standard files" may take up more disk space and be less efficient than "optimised" non-standard files.

New packages acquired from "external" sources will not follow these (or your) standards and thus cause difficulty.

ASWI Programming Standards

*by FHD v.Batenburg, TRC Bonnema,
L v.Geldrop, H v.Loon, B Smoor*

Introduction

This paper describes a set of standards proposed by the committee of the "APL-Share-Ware-Initiative", ASWI for short. These standards are an integration of the various standards applied by BSO/Management Support (Utrecht, Netherlands), Philips/PASS-PC'S (Eindhoven, Netherlands), and the Institute for Theoretical Biology (Leiden, Netherlands).

The objective of this standard is two-fold. The first objective is to define a set of rules according to which APL programming should be carried out, in order to improve program maintainability. It is recognized that in a professional APL programming environment, where different people work for different clients, a need exists for a concise and standardized use of APL.

The second objective is to smooth the introduction to new programs for potential users. To fulfil this objective a set of standard names is proposed which should be available in every workspace and which act and inform according to preconceived, generally accepted notions. This ensures that a novice will always know how to move around in the on-line documentation of a foreign workspace.

We ask all programmers participating in ASWI to adhere to these standards for the following reasons. First of all, standards are a boon for not-too-experienced programmers because - hopefully - standards reflect the accumulated wisdom of experts. Secondly, standards are useful because the writers can fall back to automatisms and can focus their thinking on the real hard nuts in the program. And finally, readers can access the information faster when they don't need to puzzle things out but can rely on standards instead. This holds for good standards, but it holds for mediocre standards too.

So, even non-superb standards are often better than very clever incidentally used constructions. For this reason, we urge programmers to deviate from this standard in exceptional cases only and then to document these deviations carefully in order to ensure program maintainability.

The remaining text will cover the following topics:

- naming conventions
- branching conventions
- documentation conventions
- comments
- messages
- miscellaneous

Naming Conventions

This paragraph outlines the various standards for naming of systems, workspaces, functions, variables and labels.

Systems. An APL system is a collection of one (or more) APL workspaces and files which conceptually form a whole. Together they perform the functions described in the functional specification of the system. The system is identified by a 3 character identifier (no more, no less). Each of the characters should be alphabetic capitals. Valid identifiers are in the range of 'AAA' to 'ZZZ'. We will refer to the system identifier as "*SID*" hereafter.

Examples:

DLG - Dialogue system
PLT - Plotting system
RNA - RNA analysis
SCR - Screen utilities
TED - Text editing system
WHL - Whaling population growth simulation

Workspaces. The workspace is identified by a name of 3 or more characters of which the first three are the *SID*. All characters should be capitals and digits may be used (not in the *SID*). It is advisable not to use names with more than eight characters.

Preferably a system is contained in 1 workspace. If this is not feasible (for example for reasons of memory requirements) the system should be divided into functionally cohesive subsystems distributed over different workspaces. The names of those workspaces all start with the same *SID*.

Example:

WHLDATA - file which contains all data of whale simulation.
WHLSIM - workspace which performs population simulation.
WHLANAL - workspace which statistically analyses results.

Functions. We distinguish between user functions, sub-functions and imported functions.

1. USER FUNCTIONS

By user functions we mean:

- driver or main functions (the ones that start up the system).
- functions to be activated by the end-user directly.

User functions can be recognized by:

- an easy-to-remember name.
- error-proof and with error messages in the language of the end-user.
- sometimes no parameters.
- sometimes no explicit result.

A user function has a name of at least 5 characters of which the first three are the SID and the next two are alphabetic capitals. The remaining characters may be alphanumeric capitals.

Examples:

1. *PLTHISTGRAM, PLTPIE & PLTGRAPH*

2. *WHLSETPARMS, WHLSIMULATE, WHLGRAPH*

2. SUB-FUNCTIONS

Sub-functions are functions that are not used by the end-user directly. In general this type of function is called from other APL functions. A sub-function has a name of at least 6 characters of which the first three are the SID, followed by "_". The rest of the characters may be alphanumeric capitals.

Try to recognize functionally related sub-functions, and characterize them with common characters in position 5 and 6 of their names. (For systems with many functions, you can apply this principle recursively for remaining characters.)

Example:

<i>SIM_...</i>	all simulation programs of system with SID=SIM.
<i>SIM_DS...</i>	all discrete sub-functions.
<i>SIM_CN...</i>	all continuous sub-functions.
<i>SIM_CNAS...</i>	all continuous sub-functions for "assignment".
<i>SIM_CNTR...</i>	all continuous sub-functions for "transfer".
<i>SIM_CNSP...</i>	all continuous sub-functions for "splitting".

3. IMPORTED SUB-FUNCTIONS

Sub-functions which are copied from other workspaces are "imported sub-functions". They retain their original name. Because their names don't start with the proper SID, their foreign origin remains obvious.

If you introduce any change in the code of that function, the name should be changed in accordance to the new workspace in which it resides. (If necessary, you can explain the origin in the comment.)

Standard Names. The following niladic functions should always be present in the workspace whenever applicable.

1. *SIDLX* in □*LX*+ '*SIDLX*'

Niladic program *SIDLX* will either start the main program *SIDBEGIN* or present the most general information which is provided by *ΔSIDABSTRACT* (see paragraph DOCUMENTATION).

For example in workspace WHLSIM:

```
□LX+ 'WHLX'
```

```
□FX 'Z+WHLX' 'Z+ΔWHLABSTRACT'
```

and in workspace DLGUTILS:

```
□LX+ 'DLGLX'
```

```
□FX 'DLGLX' 'DLGBEGIN'
```

2. *SIDBEGIN*

If a workspace has one user function only, then this function is invoked by function *SIDBEGIN*.

In workspaces with more than one user function or with sub-functions (utilities) only, function *SIDBEGIN* only presents those alternative functions.

For example in workspace WHLSIM:

```
□FX 'WHLBEGIN' 'WHLSIMULATE'
```

3. *SIDRECOVER*

Any system which might be corrupted by unexpected errors or crashes should provide a function *SIDRECOVER* to repair the damage as well as possible. If automatic recovery is awkward or impossible, *SIDRECOVER* should direct the user to the proper information, or describe the best procedure to do so.

For example in workspace WHLSIM:

```
□FX 'WHLRECOVER' '...' '...' '...'
```

Variables. This paragraph will outline the conventions for shared variables, global, sub-global and local variables.

Global Variables. A global variable is a variable which is not local to a function, or a variable which is stored in an APL file. So a global variable is always available for reference by every APL function in the workspace.

The name of a global variable is at least 5 characters long. The first three characters are the SID. The next characters are lowercase alphanumerics.

The use of global variables must be restricted as much as possible! Examples:

1. *SCRappl, SCRfrs, SCRsubgibls, SCRmiss.*
2. *RNAenergyrules, RNAsequence, RNAstructure, RNAparams.*

Sub-global Variables. A sub-global variable is a variable, which is not a global variable but is accessed by more than one function. The name of a sub-global variable is at least 6 characters long. The first three are the SID, followed by "_". The remaining characters are all lowercase character. Examples:

1. *WHL_menu, WHL_params, WHL_select.*
2. *RNA_options, RNA_choice.*

The use of sub-global variables must be restricted as much as possible!

Local Variables. A local variable is a variable which is local to that function. The name of a local variable may be A-E or H-K or M-Q, S-Y. Or alternatively formulated, all capitals except F, G, L, R, or Z. For clarity longer, mnemonic names could be used up to 4 characters in order to improve the readability of the programs.

The name of the arguments and the explicit result of a function are L, Z and R or short mnemonic names which start with L, Z or R.

- Z: the variable name Z is reserved for the explicit result.
- L: the variable name L is reserved for the left argument of a function.
- R: the variable name R is reserved for the right argument of a function.
- F: the function name F is reserved for the left argument of an operator.
- G: the function name G is reserved for the right argument of an operator.
- I, J, K: it is advisable to reserve I, J and K for counters and important indices.
- T: We suggest using T for temporary line-locals (a line-local is a variable which gets its value somewhere else in the same line).

The variables F, G, L and R should not be assigned a value in any function.

When mnemonic names are used for the arguments or for the explicit result, the following rules must be obeyed:

1. the names may be up to 4 characters long (longer names might conflict with names of user functions).
2. the names must be alphanumeric capitals.
3. F, G, L, Z and R may never be used.

4. the first character should be L, R, Z if that variable is the left argument, right argument, or explicit result.
5. the standard names for labels should not be used for variables.
6. comment lines explaining the meaning of the arguments and the explicit result, must still be in the function and conform to the current standard.

Shared Variables. The names of shared variables are still under discussion. We welcome any suggestions.

Labels. The only permissible label names are B#, E# (with # a number), BE, XX and EXIT. See next paragraph.

Branching

There are numerous ways in APL to branch to function lines. But it is also true that this excessive number of possibilities has given APL a bad name. Imagine an APL function in which different ways of branching are used (for example: $\rightarrow L \times 1 C, \rightarrow (C) \rho L, \rightarrow C+L, \rightarrow C+L, \rightarrow (C)/L$). It is clear that in reading such a function, a substantial amount of time is needed to solve the flow in that function.

For reasons of structured programming one should not think in branches but in conceptual blocks of code which are executed once, multiple times or not at all. Each block is marked by a label B# at its first statement and by a label E# at its last statement, with # a unique number for each block.

So, we suggest new programs adopt the following 4 types of construction:

1. Condition:

B3: $\rightarrow (\dots) / 1+E3$

...
E3: ...

2. For-loop & Repeat while:

B1: $\rightarrow (\dots I \leftarrow I+1) / 1+E1$

...
...
E1: $\rightarrow B1$

B8: $\rightarrow (\dots) / 1+E8$

...
...
E8: $\rightarrow B8$

3. Repeat until:

B3: ...

...
...

E3: $\rightarrow (\dots) / B3$

4. If/else & Case:

$B4: \rightarrow (\dots) / B4a, B4b, B4c, 1+E4$

$B4a: \dots$

$\dots$

$\rightarrow 1+E4$

$B4b: \dots$

$\dots$

$\rightarrow 1+E4$

$B4c: \dots$

$\dots$

$E4: \dots$

Remember that the numbers identify different blocks in one function (not different types of construction).

Short conditions or repetitions can be coded as:

$BE: \rightarrow (\dots) / 1+1 \square LC \diamond \text{short conditional expression}$

$BE: \rightarrow (\dots) / 1+1 \square LC \diamond \text{short repeating expression} \diamond$
 $\rightarrow \square LC$

$BE: \text{short repeating expression} \diamond \rightarrow (\dots) / 1 \square LC$

$BE: \rightarrow (\dots) / 1 \square LC \underline{\Delta} \text{short repeating expression}$

Here " $\underline{\Delta}$ " stands for "lev" or for " $\square FX \ 'Z+L \underline{\Delta} \ R' \ 'Z+L'$ ".

Never branch to results of a calculation or to absolute line numbers!

To exit a function prematurely, do not use " $\rightarrow 0$ ". Use label "EXIT" instead which is at the end of a function. So apply the following type of construction:

$XX: \rightarrow (\dots) / EXIT$

$\dots$

$\dots$

$XX: \rightarrow (\dots) / EXIT$

$\dots$

$\dots$

$EXIT: \text{last and empty(!) line of program}$

Documentation

This paragraph outlines the proposed on-line documentation for programmers and for end-users. We suggest implementing the following documentation in functions which yield the text as their resulting vector, with each line separated by a return-character. Use uppercase and lowercase characters as in normal text (not uppercase only).

Our convention is simple and straightforward: document anything you like by using an APL function which yields a character vector, and give this function the same name as the item you are documenting, prefixed with " Δ ". For example, if you document the global variable "*WHLxyz*" the documentation is generated by function " Δ *WHLxyz*".

Programmers' Documentation. By on-line programmers' documentation we mean a description of all components of an APL system in the workspace or on file so that it can be used rapidly at all times by a programmer. This documentation is intended for system maintenance and development.

The following items should be documented and on-line available:

1. **SYSTEM.** If any general information exists which should be available for the programmer, then it is documented in variable Δ *SIDSYSTEM*. Example: if important information exists for system *WHL*, it is documented in Δ *WHLSYSTEM*.
2. **GLOBAL VARIABLES.** Global variables must be described briefly (1 line per variable) by function Δ *SIDGLOBALS*. If more extensive information is useful, then that particular variable is also described separately. Example: global variables *RNAenergyrules*, *RNAsequence* could be described extensively in Δ *RNAenergyrules* and Δ *RNAsequence*.
3. **SHARED VARIABLES.** Shared variables must be described briefly (1 line per variable) by Δ *SIDSHARED*. Similar to the global variables, more extensive information can be presented separately. Example: shared variable *CTL* can also be described in Δ *CTL*.
4. **VERSION-CONTROL.** If you don't have a proper version-control system we suggest you apply the variable Δ *SIDHISTORY* to document any changes. Example:

```

      □← $\Delta$ RNAHISTORY
Problem      |Date   |Author  |Change
-----+-----+-----+-----
-File selector without|031089|V.Batenb|RNA_FIL[5 7.1 11]
  folders      |      |      |RNA_BOX new
               |      |      |RNA_curfile new
-Absolute energyvalues|211189|JJ.Riet |RNArule new values

```

A component file is documented in its 0th component.

User Documentation. On-line user documentation is useful for experienced users who have forgotten some particular details and want to refresh their

memory. This requires that the information is short and compact and can be found easily by following standard procedures.

On-line user documentation is also useful for the novice, but here it has two different purposes. First the user wants to learn if the workspace will attend to his needs (investigation process) and second how to use it or where to find more elaborate information (application process). The first purpose -investigation - also requires short compact information; only the second purpose needs more elaborate information.

Both types of user are properly served with standard names to find their information. For this the following names serves:

1. $\Delta SIDABSTRACT$

$\Delta SIDABSTRACT$ returns 1 (or at most 2) lines which describe the system very briefly (compare this with the more extensive descriptions in $\Delta SIDDESCRIBE$ and $\Delta SIDHOW$). If function $SIDLX$ does not start function $SIDBEGIN$, then it activates $\Delta SIDABSTRACT$. For example:

```

□FX 'Z+ΔWHLABSTRACT' 'Z+' 'Simulate whale hunting
for...'
□FX 'Z+WHLLEX' 'Z+ΔWHLABSTRACT'

```

2. ΔSID

ΔSID generates 6 lines with the following basic information:

1. SID .
2. complete name of the workspace.
3. version number, release and/or last modification date; anything which uniquely distinguishes this workspace from earlier or newer versions.
4. copyright holder.
5. name, address or telephone number of person or company to be contacted for information.
6. manuals, books and other documentation of the system.

3. $\Delta SIDDESCRIBE$

Program $\Delta SIDDESCRIBE$ describes what the Workspace will do: the purpose of the Workspace. Program $\Delta SIDDESCRIBE$ should inform the reader in a nutshell if the workspace is appropriate for his problem. It does not teach the reader *how* to work with the workspace (that is the function of $\Delta SIDHOW$), but only tells *what* the workspace does. This extensive information is summarised in $\Delta SIDABSTRACT$.

4. $\Delta SIDHOW$

Program $\Delta SIDHOW$ describes *how* to work with the workspace. It describes the various options of the user-functions and the required syntax. If the Workspace contains several user functions which can be activated by the user independently, than $\Delta SIDHOW$ at least summarises each user function.

5. Δ SID-

If Δ SIDHOW is insufficiently brief, then for each function a describing " Δ SID-" could be added which explains the particulars of that function. So the syntax of each user functions $SIDxyz$ (which should at least briefly be described in " Δ SIDHOW") can be described more elaborately in Δ SIDxyz.

This documentation is the only elaborate one, because it is primarily intended for the novice to learn how to operate the various functions. Example: function $WHLSIMULATE$ has its syntax described in Δ WHLSIMULATE, function $RNAFOLD$ in Δ RNAFOLD.

 $\square + \Delta$ CPXPOLARHOW

Purpose: convert polar to cartesian coordinates

Syntax: $PL \leftarrow CPXPOLAR CRT$

$CPXPOLAR$ converts cartesian coordinates to polar coordinates. Its inverse program is $CPXCART$.

The right argument CRT specifies the cartesian coordinates. It can be a vector of pairs, or a 2-column table with X-Y values.

The result PL are the polar equivalents. The rank and size of PL are identical to those of CRT . Each pair is the modulus and the angle in radians.

Example	1	0	$\leftarrow$	$CPXPOLAR$	1	0
	1	0.78			0.71	0.71
	1	1.57			0	1

Comments

Use comments to describe what is done and why, but not how it is done (that's what APL is for). Any APL function should at least contain the following comments

- [1] Δ description what the function does
- [2] Δ R: description of the right argument
- [3] Δ L: description of the left argument
- [4] Δ Z: description of the result
- [5] Δ G: names of used globals and subglobals, if any
- [6] Δ P: name of programmer, dates of changes

Example:

```

[0] Z ← L VBL_DOCUMENT R;A;L•IO
[1] ## Document system and print to various devices.
[2] ## R[i;]: Character names of output devices i.
[3] ## L[]: Title to be printed.
[4] ## Z: '' if OK, else error message.
[5] ## G: VBLprint; VBLerr; VBLglobal <== bad design, many
[6] ## P: EvBatenburg; 6/83,3/84,8/84,2/85. (sub-)globals

```

Furthermore we suggest 2 types: section comment and line comment.

1. Section comments explain several lines of code. This type of comment uses a whole line in the function.
2. Line comments illuminate the code of a line. This type of comment trails at the right of that particular line. Example:

```

## This explains the following lines
.....code..... ## This explains the code at the left

```

Messages

Messages are still under discussion. We welcome anybody who has suggestions.

React immediately where the error is discovered. If a test discovers an error, this is the place where the first action should be taken. Avoid **printing** of error messages in sub-functions, but express the occurrence of an error in the explicit result. (For example: "(empty vector)" instead of computed array, 0 (zero) instead of computed array, error message instead of computed array)

Also avoid letting a sub-function halt the system by "+". The reason for both suggestions is that a sub-function may very well be unhappy with an exceptional condition, but its caller might better be suited to react to such a condition.

Miscellaneous

In this section a few remarks are made on APL programming practice. The size of an APL function should be within working conditions and preferably less. This means that if one is working with a screen that displays 20 lines of 80 characters each, the size of APL functions should preferably be less than, or equal to 20 lines of 80 characters.

Good APL functions...

1. are readable.
2. perform one conceptual task only.
3. can be used as building blocks.
4. are like APL primitive functions; so they will give a defined result for all parameter values.
5. work for missing left argument (default) and don't crash for empty arrays.
6. receive data in the right argument and control in the left argument.

Good APL functions are not necessarily (CPU) efficient. What is (CPU) efficient today may very well be (CPU) inefficient tomorrow (and vice versa).

Re-think again about your design if an APL function...

1. is bigger than your screen.
2. uses branches.
3. uses (sub-)global variables.

ASWI Standards Committee

FHDv.Batenburg, Institute for Theoretical Biology,
Kaiserstraat 63, 2311GP Leiden, the Netherlands.

TRC.Bonnema, University Twente, Postbus 217, 7500AE Enschede, the Netherlands.

Lv.Geldrop, I.P.Sharp-Reuter, Raiffeisenstraat 11, 5611CH Eindhoven, the
Netherlands.

Hv.Loos, ORIGIN/Nederland, Gebouw HKB-6, Kastanjelaan 1,
5616LH Eindhoven, the Netherlands.

B.Smoor, Adfee, Dorpstraat 50, 4128BZ Lexmond, the Netherlands.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.

Contents

Hackers' Corner: Fast Filing with $\Delta pack$	Adrian Smith	125
A Case Story About Moving an APL-application	Allan Zenth	127
Desk Top Publishing on the Cheap	Richard Hill	130
Training the Mouse (in APL*PLUS)	Alain Delmotte	132
How STSC's $\square MF$ can help in testing APL workspaces	Ray Cannon	135
A Look at Residues	Stephen Wynn	137



Cocking & Drury Ltd

**Your sole UK supplier
of the entire APL*PLUS software range**

including:

APL*PLUS PC

APL*PLUS II for the 386

APL*PLUS II for UNIX

APL*PLUS VAX/VMS

APL*PLUS Mainframe

**as well as Run-Time and Run-Time Developer Kits
and all delivered ex-stock!**

**The only dealer authorised to provide maintenance and support for APL*PLUS
All software prices include 12 months free maintenance, with free upgrades (except
APL*PLUS PC).**

**We also offer a wide range of courses in APL and the most experienced APL
consultancy service in the country.**

Whatever your APL needs - call the experts first.

Cocking & Drury Ltd 180 Tottenham Court Road, LONDON W1P 9LE

Phone 071-436 9481 Fax 071-436 0524

Hackers' Corner: Fast Filing with *Δpack*

by Adrian Smith

Health Warning

What follows is highly specific to APL*PLUS/PC; it won't even do you any good under PLUS II, which doesn't have the IPSA-style package utilities provided.

Motivation

For many years, my APL file system has been built around two almost trivial routines called *DUMP* and *GET*. These simply take a named list of APL objects (variables or functions) and sling them on file. The only subtlety is that *GET* will optionally pull back variables under a different name if you ask it to:

```
RC+'MYFILE' DUMP 'ΔCODE,ΔDESCR,anyΔfun'
RC+GET 'MYFILE'           A Simply retrieves the whole file
RC+'MYFILE' GET 'ΔDESCR'   A Selective retrieve
RC+'MYFILE' GET 'CHECK+ΔCODE' A ... with a change of name
```

This simple approach has the virtue of almost total portability (it even worked under VSPC using sequential text files), but when implemented on APL*PLUS/PC using the obvious strategy of one object per component (coincidentally the format turns out to be compatible with Statgraphics files, which is quite useful) it is horribly slow for collections of lots of small objects.

```
ρNL
112 12
RC+'SCRATCH' DUMP NL   A 11.2 sec to dump 20480 bytes
RC+GET 'SCRATCH'       A 1.3 sec to retrieve whole file
```

I had toyed with a variety of strategies for improving matters, and the best one I had come up with was to ravel like objects end to end (recording ranks and shapes in a header table) and then write out a maximum of four components. This speeds the writing process up quite a lot, but of course files are read more than they are written, and the slight reduction in reading speed brought about by all that indexing and reshaping was enough to put me off. So ... there matters rested until STSC quietly included a bunch of ASMFNS to emulate the Sharp Package concepts.

A Case Story About Moving an APL-application.

by Allan Zenth (Group Financial Control, Unidanmark A/s, Denmark)

When I read the Editorial in Vector Vol.7 no.4, about the differences between APL Dialects, I was inspired to tell the following story.

It all started back in October 1990, when I was asked about the possibility of moving an APL-application from one machine to another. Having already participated briefly in a similar move, I said that of course it was possible. I even added that there was a standard way of doing so. Little did I know the kind of problems that awaited me. I was told that the application used Digital's APLSF and ran on an old DEC20. This machine would soon be taken out of service and my job was to 'save' this application. The new platform for the application was to be APL2 under VM/CMS on one of the IBM-mainframes, since VSAPL was closed down just one month earlier.

As background information it may be mentioned that the application was located in one of three banks that were to be merged, but I in another where I had my APL-experience (mostly VSAPL). The application was a data base of real estate etc. owned by the bank together with taxation formulas.

Since I had no knowledge about APLSF or this application, I asked for an APL-manual on APLSF and the documentation. I got both at the beginning of December. From a friend I also got an important copy of APL-Quote-Quad vol.9 no.3 (March 1979) with the full WSISO-standard. In addition I had a copy of 4 pages from MicroAPL, with a good example of how to write the functions in the "old" system.

The APLSF-manual showed me a range of specialities:

- AV has 512 elements (9-bit per element).

- The files were component-files.

- Filehandling with quad-arrow-primitives as in APL.68000.

- A \$-format much alike □FMT.

- Monadic decode and epsilon as special executes (unquote).

- The ",'" separator.

- Printing as part of the session with special characters to start and stop the printer.

The documentation was quite good except that the latest changes in functions were missing. But I could see about 240 functions to move and the specialities above were heavily used in the functions. In addition there was an undocumented use of monadic omega as an index generator from booleans.

Late December I finally got access to the system at another office some 3 miles away. They had an old VDU (not cleaned for years) but there was no full-screen facility. Happily I started the old del-editor and I was soon able to do a first test only to get the nice message 'WS FULL'. As far as I could see the reasons were limited workspace and many space-consuming booleans (i.e. one boolean used 36 bits, or one word). According to the WSISO-standard the booleans were necessary because of the conversion from a 9-bit structure to a more common 8-bit. It was then necessary to rearrange the sending functions in a more space-economic way. I had to use a lot of time on other work but in mid-January 1991 I was told that the DEC20 would be closed on February 1st. This date was not the greatest problem as the final tapes with functions and data were shipped to the IBM-site on January 31st.

There were no real problems with reading functions or data to APL2 as $\square TF$ is very easy to work with. One minor problem was that every time the DEC20 delivered 36 bits I received 40 bits at the IBM-machine. It turned out to be necessary to throw away bits 33-36 and not the last four. Another problem was the first of my three translation-table errors. The high minus was translated to a normal minus although this was easy to handle for the data.

The case was put aside again for some months, but in May the process could continue. In advance of the real work a few decisions had to be made:

Format by example should replace the \$-format.

A locally developed "object-file system" using function and variable names as "keys" should replace the component files.

Printing through library-routines.

The omega function should be replaced by a subroutine.

The user should be able to use a terminal (PC) without the APL-symbols.

The work was time-consuming. Every function should be revised and with 240 functions I had to use about 8 full days for the first pass. The most time-consuming parts were file-handling and printing because no utilities were used. The three translation errors used considerable time as well.

Of course, I discovered some 'bugs' during the test-phase but nothing very serious. I will mention that negative numbers require that format by example is precisely handled.

The application is now running and appears as it did on the old machine. I am planning a change to a modern full-screen-interface.

Vector Back Numbers

Back numbers of Vector are available (in very limited quantities for Vols 1 and 4) from:

British APL Association
FREEPOST (SG923)
9 Oak Grove, HERTFORD
SG13 8BR

Price £10 per vol (UK); £14 (overseas), £20 (airmail)

Desk Top Publishing on the Cheap

by Richard Hill

After some considerable trouble (and after fruitless consultations with STSC and GSS) I have finally succeeded in importing graphics created by GSS drivers into Microsoft Word version 5.0. My approach takes account of the fact that I use a Hewlett Packard Laserjet but the principle should hold for most popular printers.

Before I fully explain the very simple procedure, I'll give you a brief description of what not to do.

Firstly, you might logically assume that the best way to export graphics is to use Computer Graphics Metafiles (CGM files). APL*PLUS PC & APL*PLUS II both supply the metafile driver META.SYS which to quote the GSS Manual 'manages metafiles, the computer graphics picture exchange and storage medium.' Wonderful stuff, but Microsoft Word 5.0 doesn't support metafiles. (I have used metafiles with WordPerfect and Harvard Graphics, but since I am obliged to use Word 5.0 I was forced to halt my research into metafiles. Word for Windows also supports the import of metafiles, although it does take a short while to convert the metafile to a readable format.)

The second approach I used was to save the charts to a HPGL plotter file using the GSS driver HPGLPLTR.SYS. The GSS Manual tells you that to write a graphic to a file, you need to type the following commands at the DOS prompt:

```
SET PLOTTER=HPGLPLTR  
SET HPGLPLTR=<filename>
```

Obviously you need also to include the HPGL driver in your CONFIG.SYS or (more conveniently) in the GSS configuration file CGI.CFG. I discovered that my machine also required 'SET FLAGGING=XONOFF' to communicate properly with the plotter (possibly due to the fact that I didn't have a plotter connected!). In order to avoid problems with filling the DOS environment space and also to avoid conflict with the contents of the GSS configuration file, I found it best to include the two commands above in CGI.CFG. Further tweaking can be done by reference to the GSS manual.

This method works in so much as it writes the graphic to a HPGL plotter file and Word 5.0 can interpret and print this on a Hewlett Packard laserjet printer. However the file created can be well over one megabyte in size for even a fairly

simple chart and although Word was capable of printing a chart on its own, as soon as any text was added the printer would misbehave - printing charts and text on different pages in spite of the page layout and also printing gibberish.

At this stage I was forced to refer once more to the GSS manual. (I bought my manual direct from America before STSC started to include the manual with APL*PLUS and consequently it contains descriptions of all of the GSS drivers rather than the select few that STSC supply.) This fact is important because the complete manual has details of the alternative Laserjet printer driver HPLASERP.SYS. This not only produces better quality output (especially at 300 dpi resolution) but more importantly offers the option of printing to a file. Unfortunately this file comes on the GSS Completer Kit which costs around £100 but you do receive all sorts of other drivers and font files (which I've still not worked out how to use!).

The output from the driver HPLASERP.SYS can be redirected to file by inserting 'HPLASERP=<filename>' into CGI.CFG. It is important to restrict the number of rows per page using 'PAGELENGTH=<rows per page>' if you wish to put Word text on the same page as an imported graphic. The option 'FF=' must be set to 'OFF' if you wish to type text after a graphic.

At this stage I discovered that the print file contains an escape sequence which makes the graphic start on a new page. The only way around this seems to be to delete the escape sequence in APL. This can be easily identified and removed in APL (treating the print file as a native file and reading its contents as a vector in APL) by reference to your printer manual. (The format of print files are very easy for an experienced APLer!)

The resulting print file can be imported into Word and although the graphic cannot be previewed or resized in Word (since it only has a recognisable form when printed) the preview option can be used to gauge the spacing and layout of a document. The option of whether to print to a file or to the printer cannot be interactively reset inside an APL session. I have got around this problem by having two GSS configuration files and using `⎕CMD` to overwrite CGI.CFG as required.

This method is a cheap and fairly efficient way of doing desk top publishing. Its major weaknesses are that the graphic cannot be viewed in Word and any resizing must be done in APL. This approach is useful to the NUFM Economics department because we have as a standard two main sizes of charts, thus there is no desire to resize charts. Obviously it would be preferable to import a true graphic image but until Word supports metafiles or GSS extend their selection of drivers to produce say Lotus PIC format then this approach is extremely viable.

Training the Mouse (in APL*PLUS)

by Alain Delmotte

In VECTOR 7.3 pp 130-135, Ray Cannon proposed some functions in APL*PLUS/PC to be able to use the mouse. These functions were very much welcome for me, but I slightly modified them as follows.

The mouse always works in pixels, that is the screen is divided in 200 lines of 640 columns even in text mode. In that case, the values jump by 8 (0 8 16 24 632 and 0 8 192). That is why Ray divides the cursor position values by 8 (*MSΔQ MSΔQCU MSΔQR MSΔWIN MSΔCU:0.125 x* or *8 x*) to return 25 lines and 80 columns (1 character "boxes").

But you can design different boxes. Suppose you have a menu bar at the top of the screen; it uses 8 pixel's lines (one text line) and you have 10 entries (each 64 pixel's columns). So the returned cursor position should always be 0 for the line and 0 1 2 or 9 for the columns (top left corner is 0 0). The modified functions allow to specify the size of the boxes. If two values are given they represent the y and x values (lines and columns), a single value will apply to both y and x. In text mode one will generally use 8 (as Ray did).

I have also introduced $\square IO = 1$ as local to the mouse functions. One could even go further by choosing the mouse shape and colour, but that is another story.

Finally I want to point out that I got some problems with the mouse in EGA graphic mode:

1. the cursor moves down to line 200 but not to line 357 or so of the EGA mode,
2. there is conflict with $\square GWRITE$ and
3. there is conflict with $\square GPRINT$. I haven't seen version 10 of APL*PLUS/PC, so I do not know if these problems exist with the builtin mouse support.

The function *GENERAL* uses the modified mouse functions. *barremenu* provides a screen with a top line menu: Help File Exit System Do and two information boxes. Selection is done either by clicking with the mouse left button or by typing the first letter of the choice. The mouse window is restricted to the first line.

```

    ▽ GENERAL;a;b;dum;HFESD;Δmouse
[1]   a Programme de cartographie climatique
[2]
[3]   □IO←0
[4]
[5]   a Help, File, System and exec are called functions
[6]   HFESD← 5 6 ρ 'Help File →0 Systemexec '
[7]
[8]   deb:
[9]   dum←0 □POKE 254 a □INKEY doesn't wait for input
[10]  MSΔINIT 0
[11]  0 0 1 80 MSΔWIN 7 8 a 7 8 is the size factor
[12]  0 0 25 80 □WPUT barremenu
[13]  MSΔSHOW
[14]  x: a check the mouse and the keyboard
[15]  a←0 MSΔQ 24 64 ◊ +((a[0]≠0)∧a[2]◊ 0 1 2 4 9)/go1
[16]  b←1+□INKEY,'x' ◊ b+□AV[b+32×97<b+□AV,b]
[17]  +(~b◊'HFESD')/x ◊ →go2
[18]  →x
[19]  go1:MSΔHIDE ◊ 1,(0 1 2 4 9 =a[2])≠HFESD ◊ →deb
[20]  go2:MSΔHIDE ◊ 1,('HFESD'=b)≠HFESD ◊ →deb
    ▽

    ▽ MSΔINIT cu;rc
[1]   a Initialize mouse ... global Δmouse+1 if OK ... else 0
[2]   a cu is the cursor type ... 0=mouse block; 1=hardware
[3]   rc←0 □INT 51 ◊ Δmouse+0×1+rc ◊ →Δmouse+0
[4]   →cu+0 ◊ rc← 10 1 □INT 51
    ▽

    ▽ MSΔSHOW;r
[1]   a Show mouse cursor
[2]   →Δmouse+0 ◊ r←1 □INT 51
    ▽

    ▽ MSΔHIDE;r
[1]   a Hide mouse cursor
[2]   →Δmouse+0 ◊ r←2 □INT 51
    ▽

```

```

▽ pos MSΔWIN sf;xy;rc;size;□IO
[1]  a Set mouse window ... as per □WIN i.e. offset,size
[2]  a sf is the size factor, sf[1] line, sf[2] column
[3]  □IO+1 ◊ →Δmouse+0
[4]  pos+4+,pos
[5]  a Set the window with a maximum of 639, 199
[6]  xy+ 639 199 [[φsf×2+pos ◊ size+ 639 199 [[φsf×2+pos
[7]  rc+(7,0,xy[1],xy[1]+size[1])□INT 51
[8]  rc+(8,0,xy[2],xy[2]+size[2])□INT 51
▽

▽ pos MSΔCU sf;xy;rc;□IO
[1]  a Place mouse cursor at row,col
[2]  a sf is the size factor, sf[1] line, sf[2] column
[3]  □IO+1 ◊ →Δmouse+0
[4]  xy+ 639 199 [[φsf×2+pos
[5]  rc+(4,0,xy)□INT 51
▽

▽ r+btn MSΔQ sf;rc;xy;ct;□IO
[1]  a Query number of presses and row/col for button <btn>
[2]  a btn+0 ... left, btn+1 ... right, btn+2 ... both together
[3]  a sf is the size factor, sf[1] line, sf[2] column
[4]  □IO+1 ◊ r+3p0 ◊ →Δmouse+0
[5]  rc+(5,btn)□INT 51
[6]  ct+rc[2] ◊ xy+rc[4 3] a note inversion of x, y
[7]  r+ct,[xy÷sf
▽

▽ r+MSΔQCU sf;rc;xy;□IO
[1]  a Query row/col for cursor
[2]  a sf is the size factor, sf[1] line, sf[2] column
[3]  □IO+1 ◊ r+ 0 0 ◊ →Δmouse+0
[4]  rc+3 □INT 51
[5]  xy+rc[4 3] a note inversion of x, y
[6]  r+[xy÷sf
▽

▽ r+btn MSΔQR sf;rc;xy;ct;□IO
[1]  a Query number of releases and cursor row/col for <btn>
[2]  a btn+0 ... left, btn+1 ... right, btn+2 ... both together
[3]  a sf is the size factor, sf[1] line, sf[2] column
[4]  □IO+1 ◊ r+3p0 ◊ →Δmouse+0
[5]  rc+(6,btn)□INT 51
[6]  ct+rc[2] ◊ xy+rc[4 3] a note inversion of x, y
[7]  r+ct,[xy÷sf
▽

```

How STSC's $\square MF$ can help in testing APL workspaces.

by Ray Cannon

The use of STSC's monitor function ($\square MF$) in the area of optimizations has been well documented in the past, e.g. Dave Ziemann's excellent article "Efficiency in APL" in Vector Vol.4 No.4, but I think $\square MF$ has a potentially more useful role to play in testing.

What $\square MF$ does is to record the amount of cpu time spent in each line of code executed in each function for which monitoring has been turned on. It also (and from my point of view, more importantly) records the number of times each line is entered.

I use $\square MF$ during testing to check that every line of code has been executed. Note, this does not show that every path through the code has been tested.

At its simplest, $\rangle XLOAD$ the WS to be tested, turn $\square MF$ on for all functions, run the WS using your test data, save a text matrix of the results of $\square MF$ alongside the $\square CR$ of each function being monitored to file. Check the normal test results. Print the $\square MF$ results file and check that every line of every function has been executed, and if some lines have not, modify the test data and repeat the process.

Sounds simple, and the idea is. However there are some problems.

First of all, not all the functions may be in the test workspace at the start and end of testing (dynamically loaded or fixed functions). The most difficult area in this respect is with function overlay. You may need to alter the overlay procedure to do the $\square MF$ recording automatically for you and then merge the results from all the overlays.

Secondly locked and compiled functions can't be tested in this way. Testing of these function needs to be done in their unlocked/pre-compiled state.

Thirdly, the resolution of $\square MF$ is only 1 line, not an APL statement. So $\square MF$ cannot show that all statements in lines with multiple, conditionally executed statements (e.g. lines with "DOIF") have been executed. (This is one reason I do not like "DOIF" and lines with statement separators.)

Fourthly, it is VERY difficult to create test data/input that executes every part of the system.

What $\square MF$ does show you more often than not, is that your testing is incomplete.

I recently had to test an archiving system being used at a large site. This had a great deal of error trapping designed to be as fail safe as possible. I used $\square MF$ to great effect in proving that every line of code had been tested. I then repeated the testing of the error trapped function, with a generated error being put on each line in turn. $\square MF$ was indispensable in showing what lines had been tested, and the path followed during error trapping.

The benefits of using $\square MF$ during testing are thus:

- Confidence that every line of code has been tested;
- Confidence that your test data contains at least a minimum subset;
- It supplies a simple discipline and challenge to testing;
- It provides a measurable first stage testing completion target.

This last point may be the most important from your manager's point of view.

If I may mis-quote the words of the famous beer commercial: " $\square MF$ shows the parts that other testing has not reached."

Happy testing!

A Look at Residues

by Stephen Wynn

One of the joys of APL is the ease with which it is possible to generate residues, for example the quadratic residues R of 13 are given by:

$$\boxed{+R+13|(1\ 12)*2}$$

1 4 9 3 12 10 10 12 3 9 4 1

In this article we will be concerned with cubic and higher order residues, especially as quadratic residues have peculiar properties not possessed by other residues.

Cubic Residues

Consider the sequence:

$$\boxed{+R+P|(1^{-1}+P+13)*3}$$

1 8 1 12 8 8 5 5 1 12 5 12

$$\boxed{+A+(R\ 1\ R)=1\ \rho\ R} \quad 1)$$

1 1 0 1 0 0 1 0 0 0 0 0

$$\boxed{+X+^{-1}1\ A/1\ \rho\ A}$$

7

For any prime P of the form $1+3k$, $p = 7\ 13\ 19\ 31 \dots$ we arrive at a number $X = 3\ 7\ 10\ 17 \dots$ which it turns out has not before been seen by science! What properties does it have? It is related to W defined by:

$$\boxed{+W+(1=R)/1\ \rho\ R}$$

1 3 9

An alternative definitions of X is:

$$\boxed{+X+\lceil / \lfloor P\ W \circ . \times R}$$

7

We have:

$$\begin{aligned} & \square \leftarrow V \leftarrow P \mid W \times X \\ & 7 \ 8 \ 11 \\ & P = 0.5 \times (3 \times + / V \times 2) - (+ / V) \times 2 \end{aligned} \quad (2)$$

We get the value unity for any of the primes $P = 7 \ 13 \ 19 \ 31 \dots$. If V is the vector (x, y, z) then 2) becomes the equation:

$$p = x^2 + y^2 + z^2 - xy - yz - xz \quad (3)$$

This result has been published [1].

A natural question is how many binomial coefficients, that is members of Pascal's Triangle, are residues. Binomial coefficients are given by $\iota K, + \setminus \iota K, + \setminus + \setminus \iota K$, etc. Continuing our APL:

$$\begin{aligned} & \square \leftarrow U \leftarrow + / (P \mid + \setminus \iota P) \in R \\ & 3 \\ & \square \leftarrow P \mid W \times U + (P + 2) \div 3 \\ & 8 \ 11 \ 7 \end{aligned}$$

Which we see has the same values as V except in a different order. This is true for any of the primes $7 \ 13 \ 19 \ 31 \dots$. Now consider the following:

$$\begin{aligned} & B \leftarrow P \mid + \setminus \iota P \quad \text{a binomial coefficients.} \\ & C \leftarrow 2 \\ & U1 \leftarrow + / B \in R \\ & U2 \leftarrow + / B \in P \mid R \times C \\ & U3 \leftarrow + / B \in P \mid R \times C \times 2 \\ & (U1, U2, U3) + (P + 2) \div 3 \\ & 8 \ 11 \ 7 \end{aligned}$$

Which again is the same as V to within permutations, and is for any C which is a cubic non-residue. e.g. for $C \leftarrow 4$ we get $8 \ 7 \ 11$.

This is also true for all $P = 7 \ 13 \ 19 \dots$

Higher Order Residues

We can now press on to look at quartics:

$$\begin{array}{l}
 R+13 \mid (1^{-1}+P+13)*4 \\
 B+P \mid + \setminus 1P \\
 C+2 \\
 U1++/B \in R \\
 U2++/B \in P \mid R \times C \\
 U3++/B \in P \mid R \times C * 2 \\
 U4++/B \in P \mid R \times C * 3 \\
 U1, U2, U3, U4 \\
 4 \quad 4 \quad 2 \quad 1
 \end{array}$$

Then we find:

$$P = ((U1 - U3) * 2) + (U2 - U4) * 2 \quad (4)$$

In this case C is arbitrary except that it needs to be a quadratic non-residue. This is true for any p of the form $1 + 4k$. There are many proofs of the fact that primes of the form $1 + 4k$ can be expressed as the sum of two squares $p = x^2 + y^2$ but these do not generally specify values for x and y .

This result generalises to primes of the form $1 + nk$ for any $n > 2$. Define a vector U length N where

$$\begin{array}{l}
 R+P \mid (1P-1)*N \\
 U[K]++/B \in P \mid R \times C * K-1 \quad \text{for } K = 1 \dots N-5
 \end{array}$$

Suppose that N is odd then

$$P = (+/U*2) - +/U \times (?N-1) \phi U \quad (6)$$

C is to a large extent arbitrary and can be any "primitive root" of P . Thus $U \times (?N-1) \phi U$ is constant. That is we have generated points round a circle such that when we rotate the circle, multiply coinciding points and add, we always get the same answer irrespective of the amount of the rotation.

For example if $P = 31, N = 5, C = 3$ we find $U = 6 \ 8 \ 2 \ 5 \ 8$.

$$31 \quad (+/U*2) - +/U*(74)\phi U$$

If N is even then we have:

$$P = (+/U*2) - +/U*(2*?(N-2)\div 2)\phi U \quad 7)$$

Instead of the above binomial coefficients given by B it apparently makes sense to look at further binomial coefficients, or more generally any polynomial and then use the same formula 5).

C F Gauss found formulae similar to 3) and 4). "Notions before notations" is a motto of his. However at that time APL had not yet been born. These investigations were prompted by the author from notation that is formula 1) which produces a Boolean string prompting the question where is the maximum unit value. We have a proof of 6) and 7) which are apparently original results.

This subject has connections with orthogonal experimental designs which are concerned with constructing orthogonal vectors of the same dimension, whose elements come from the same set of numbers. The vectors given by $K\phi U$ are quasi-orthogonal; that is the angle between any two of them is non-zero but is the same.

Reference

- [1] S Wynne, *Minimising a Form*, Problem 90.6, The Mathematical Intelligencer, Summer 1990.

Many people are asking: "What is *SIGAPL* today?" ...and we'd like to answer:

SIGAPL IS....

- LOCAL CHAPTERS

We have active chapters in several US cities, Europe (Switzerland and the Soviet Union) and Australia. Other *APL* groups are considering affiliation with us, and other *SIGAPL* chapters are forming. We work with **any** *APL* group on projects of common interest.

- CONFERENCES

We've sponsored most of the international *APL* conferences, and are planning future conferences. Our development, assistance, and participation in *APL* conferences has made many of them possible. We have a group of conference veterans working to insure that conferences continue to improve and diversify, while keeping costs reasonable.

- PUBLICATIONS

The *APL Quote Quad* is under new management, and is becoming a more dynamic, up-to-date publication. We welcome and publish *APL* technical articles of the highest calibre. Upcoming issues will expand to include product and meeting reviews, and news from our chapters. The Conference Proceedings is an issue of *Quote Quad*, and is an added benefit to members. Other works include our annual Index to *APL* Articles, Membership Roster, chapter newsletters, and project-related products.

- PROJECTS

Our Watch-and-Respond Project provides a united voice for *APL* advocates presenting facts regarding *APL* and *APL* use to the non-*APL* world. Our Software Library project gathers public domain software for review, commentary, and distribution. The *APL* Phrasebook project is developing a publication which clarifies the strengths, weaknesses, and differences among *APL* systems. We look forward to other project challenges, particularly relating to *APL* in education.

We're always interested in **more** participation, **more** activity, and **more** ideas.

Perhaps we've answered some questions about what *SIGAPL* is.

Are **you** ready for us?

If you are, **join** us!

It's time for you to fill out the membership form on the opposite page.

Membership forms and other requests for information may be sent to:

Pat McCarren, *SIGAPL* Program Director
Association for Computing Machinery
11 West 42nd Street New York, NY USA 10036

APPLICATION FOR **SIGAPL** MEMBERSHIP OR SUBSCRIPTION

Please enroll me as a member or subscriber of SIGAPL, as indicated:

I am already a member of ACM, but not yet a member of SIGAPL.

- ☐ Please add SIGAPL to my ACM membership. That increases my ACM dues by \$15/year (or \$7.50 for a student member of ACM).

My ACM member no: _____ or student no: _____

I am neither a member of ACM nor of SIGAPL:

- ☐ I wish to join both ACM *and* SIGAPL. I enclose an ACM application. Joining SIGAPL increases my ACM dues by \$15/year (or \$7.50 a year when I join as a student member of ACM).
- ☐ I wish to join SIGAPL without joining ACM, at \$40/yr.
- ☐ I wish to subscribe to APL Quote-Quad (the publication of SIGAPL) without joining either SIGAPL or ACM., at \$30/yr. (Members receive the publication automatically.)

AirMail Option: QuoteQuad is ordinarily mailed to members or subscribers by surface mail. The charge is the same regardless of destination. Members or subscribers outside North America may receive their publications more quickly by electing:

- ☐ Partial air: Bulk air to Amsterdam, surface from there, \$3/yr additional.
- ☐ Full air: \$15/yr additional:

Name and mailing address: _____

PLEASE PRINT _____

Signature: _____

Payment. If you are already an ACM member and are adding membership in SIGAPL, send no money now; ACM will bill you in its regular dues billing. All others: please include payment, by check or draft in US\$ on a US bank, or by credit card:

- ☐ Check enclosed. ☐ Visa. ☐ American Express. ☐ Master Card.

Card # _____

Expiration _____

Signature _____

Send application and payment to:

ACM Inc., PO Box 12115, Church Street Station, New York, NY 10249, USA

Index to Advertisers

APL People (half)	16
Cocking and Drury	124
Dyadic Systems Ltd	2,6
MicroAPL	54
SigAPL	141
Vector Back Numbers	129

All queries regarding advertising in VECTOR should be made to Alison Chatterton, at the address on the inside back cover.

Submitting Material to Vector

The Vector working group meets towards the end of the month in which Vector appears; we review material for issue $n+1$ and discuss themes for issues $n+2$ onwards. Please send the text of submitted articles (with diskette as appropriate) to the Editor:

Jonathan Barman,
Hill Top House,
East Garston,
NEWBURY, Berks RG16 7HD
Tel: 048839-575

(not after 10.00pm please!)

Camera-ready artwork (e.g. advertisements) and diskettes of 'standard' material (e.g. sustaining members news) should be sent to Vector Production, c/o Adrian Smith, Brook House, Gilling East, YORK Tel: 04393-385 (6.00pm - midnight).

Product Guide updates should continue to go to Alison Chatterton, as should requests for advertising space.

BAA: Membership Application Form

Membership of the British APL Association is open to anyone interested in APL. The membership year runs from 1st May to 30th April.

Name: _____
Address Line 1: _____
Address Line 2: _____
Address Line 3: _____
Post or zip code: _____
Country: _____
Telephone Number: _____

Membership category (please tick box): 91/92

UK private membership	£12	☐
Overseas private membership	£20	☐
Airmail supplement (not needed for Europe)	£8	☐
Corporate membership	£100	☐
Corporate membership overseas	£155	☐
Sustaining membership	£430	☐
Non-voting student membership (UK only)	£6	☐

I authorise you to debit my Visa/Mastercard account

Number: _____ Expiry date: ____ | ____

for the membership category indicated above,

- ☐ annually, at the prevailing rate, until further notice
☐ one year's subscription only

(please tick the required option above)

Signature: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK Sterling cheque to "The British APL Association", or you may quote your Access or Visa number. Send the completed form (no stamp required from the UK) to:

British APL Association, FREEPOST (SG923), 9 Oak Grove, HERTFORD, SG13 8BR

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1991/92 Committee

Chairman:	David Eastwood 071-922 8866	MicroAPL Ltd., South Bank Technopark, 90 London Road, LONDON SE1 6LN
Secretary:	Anthony Camacho 0727-860130	2 Blenheim Rd, ST ALBANS, Herts AL1 4NR.
Treasurer:	Nicholas Small 081-980 7870	8 Cardigan Road, LONDON E3 5HU
Journal Editor:	Jonathan Barman, 048839-575	Hill Top House, East Garston, NEWBURY, Berks RG16 7HD
Activities:	Dr Peter Branson 081-848 8989	Electronic Data Systems, Stockley Park, UXBRIDGE, Middx UB11 1BQ
Education:	Dr Alan Mayer 0792-295296	European Business Management School, Swansea University, Singleton Park, SWANSEA SA2 8PP
Technical:	Peter Donnelly 0256-811125	Dyadic Systems Ltd., Riverside View, Basing Road, Old Basing, BASINGSTOKE, Hants RG24 0AL
Projects:	John Searle 081-858 6811	4 Hawks Mews, Greenwich, LONDON SE10 8RA
Publicity:	Misha Jovanovic 0753-853141	99 Oxford Road, WINDSOR, Berks SL4 5DX
Recruitment:	Jill Moss 0225-462602	APL People Ltd, The Old Malthouse, Clarence St., BATH, Avon BA1 5NS
Administration:	Alison Chatterton 0992-552489	9 Oak Grove, HERTFORD, SG13 8AT

Journal Working Group

Editor:	Jonathan Barman	048839-575
Secretary:	Anthony Camacho	0727-860130
Production:	Adrian & Gill Smith	Brook House, Gilling East, YORK (04393-385)
Support Team:	John Searle (081-858 6811), Ray Cannon (0252-874697), Sylvia Camacho, Bridget Barman, Gill Smith	

Typeset by APL-385 with MS Word 5.0 and GoScript

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on most mainframe, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
The Old Malthouse
Clarence St, BATH, BA1 5NS
Tel:0225-462602

Dyadic Systems Ltd
Riverside View, Basing Road,
Old Basing, BASINGSTOKE,
Hants. RG24 0AL
Tel:0256-811125
Fax:0256-811130

Cocking & Drury Ltd
180 Tottenham Court Rd
LONDON, W1P 9LE
Tel:071-436 9481

Reuters Ltd
1-4 Singer St.
LONDON EC2A 4BQ
Tel:071-867 1166
Fax:071-867 9792

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN15 0PD
Tel: 0732-885126

Compass R&D Ltd
15 Frederick Sanger Rd
Surrey Research Park
GUILDFORD, Surrey GU2 5YD
Tel:0483-302249
Fax:0483-302279

HMW Trading Systems Ltd
Hamilton House,
1 Temple Avenue,
LONDON EC4Y 0HA
Tel:071-353 4212
Fax:071-353 3325

MicroAPL Ltd
South Bank Technopark
90 London Road
LONDON SE1 6LN
Tel:071-922 8866

Peter Cyrix Systems
12 Gloucester Place
LONDON W1H 3AW
Tel:071-935 2983



The British Computer Society, 13 Mansfield Street, London

The British Computer Society, 13 Mansfield Street, London W1M 0BD.